Generated from docs/generated/openapi.json
Generated at: 2026-06-07 09:00 UTC
Do not edit manually.
Run: npm run docs:generate

Gift App Backend — Full API Reference

Generated: 2026-06-07 09:00 UTC

This document is generated from the current OpenAPI for the Gift App backend. For each API, it includes allowed role/access, request payloads for write endpoints, and response bodies for read/write endpoints.

01 Auth

POST `/api/v1/auth/users/register`
Summary: Create Auth Users Register
Allowed role/access: PUBLIC
Notes: Access: PUBLIC.
Request payload(s):
payload

```
{
  "email": "<string>",
  "password": "<string>",
  "firstName": "<string>",
  "lastName": "<string>",
  "phone": "<string>",
  "referralCode": "SARAH-M"
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

POST `/api/v1/auth/providers/register`
Summary: Create Auth Providers Register
Allowed role/access: PUBLIC
Notes: Access: PUBLIC.
Request payload(s):
payload

```
{
  "email": "<string>",
  "password": "<string>",
  "firstName": "<string>",
  "lastName": "<string>",
  "phone": "<string>",
  "referralCode": "SARAH-M",
  "businessName": "<string>",
  "businessCategoryId": "<string>",
```

```
  "taxId": "<string>",
  "businessAddress": "<string>",
  "fulfillmentMethods": [
    "PICKUP"
  ],
  "autoAcceptOrders": true
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

POST `/api/v1/auth/guest/session`
Summary: Create guest browsing session
Allowed role/access: PUBLIC
Notes: Access: PUBLIC. Optional metadata-only request body. Guest capabilities are server-issued from Admin Guest Access Settings. Client-provided capabilities are ignored and will be removed in a future version. Guest sessions are for limited browsing and onboarding access only. Request body is optional metadata only. Guest capabilities are server-issued from Admin Guest Access Settings. Client-provided capabilities are ignored and will be removed in a future version. Guest sessions are for limited browsing and onboarding access only.
Request payload(s):
metadata

```
{
  "deviceId": "optional-device-id",
  "platform": "WEB",
  "appVersion": "1.0.0",
  "locale": "en",
  "timezone": "Asia/Karachi",
  "referrer": "landing-page"
}
```

empty

```
{}
```

Response body:

```
{
  "success": true,
  "data": {
    "guestSessionId": "guest_session_id",
    "accessToken": "guest_jwt",
    "tokenType": "Bearer",
    "role": "GUEST_USER",
    "capabilities": [
      "VIEW_ONBOARDING",
      "BROWSE_MARKETPLACE",
      "VIEW_GIFT_DETAILS",
      "VIEW_MARKETPLACE_FILTERS",
      "VIEW_DISCOUNTED_GIFTS"
    ],
    "expiresAt": "2026-05-18T12:00:00.000Z",
    "guestAccess": {
      "allowMarketplaceBrowsing": true,
```

```
      "allowMarketplaceHome": true,
      "allowGiftDetails": true,
      "allowDiscountedGifts": true,
      "allowFilterOptions": true,
      "allowWishlist": false,
      "allowCart": false,
      "allowCheckout": false
    }
  },
  "message": "Guest session created successfully."
}
```


POST /api/v1/auth/login
Summary: Create Auth Login
Allowed role/access: PUBLIC
Notes: Access: PUBLIC.
Request payload(s):
payload


```
{
  "email": "<string>",
  "password": "<string>"
}
```


Response body:


```
{
  "success": false,
  "error": {
    "code": "EMAIL_NOT_VERIFIED",
    "message": "Please verify your email before login",
    "user_verified": 0
  },
  "meta": {
    "statusCode": 403,
    "timestamp": "2026-05-18T11:04:05.524Z"
  }
}
```


POST /api/v1/auth/refresh
Summary: Create Auth Refresh
Allowed role/access: PUBLIC
Notes: Access: PUBLIC.
Request payload(s):
payload


```
{
  "refreshToken": "<string>"
}
```


Response body:


```
{
  "success": true,
  "data": "<response returned by endpoint>",
```

```
  "message": "Request completed successfully."
}
```

POST /api/v1/auth/logout
Summary: Create Auth Logout
Allowed role/access: Authenticated
Notes: Access: Authenticated. Authenticated JWT required.
Request payload(s):
payload

```
"<standard success envelope>"
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

POST /api/v1/auth/verify-email
Summary: Verify authenticated account email
Allowed role/access: Authenticated
Notes: Access: Authenticated. Authenticated JWT required. Verify the authenticated account email with the latest verification OTP.
Request payload(s):
payload

```
{
  "otp": "<string>"
}
```

Response body:

```
{
  "success": true,
  "data": {
    "id": "user_id",
    "email": "user@example.com",
    "isVerified": true
  },
  "message": "Email verified successfully"
}
```

POST /api/v1/auth/resend-otp
Summary: Resend authenticated email verification OTP
Allowed role/access: Authenticated
Notes: Access: Authenticated. Authenticated JWT required. Send a new verification OTP for the authenticated account when it is still unverified.
Request payload(s):
payload

```
"<standard success envelope>"
```

Response body:

```
{
  "success": true,
  "data": null,
  "message": "A verification OTP has been sent to your email address."
}
```

POST `/api/v1/auth/resend-verification-email`
Summary: Resend verification email
Allowed role/access: PUBLIC
Notes: Access: PUBLIC. Public endpoint. Returns the same success envelope for ineligible or missing accounts to avoid account enumeration. Reports delivery failure only when the mail provider fails for an eligible account.
Request payload(s):
payload

```
{
  "email": "user@example.com"
}
```

Response body:

```
{
  "success": true,
  "data": {
    "delivery": "OTP_SENT_IF_ELIGIBLE",
    "nextStep": "Use the 6-digit verification OTP to complete email verification."
  },
  "message": "A verification email has been sent to the email address you provided."
}
```

POST `/api/v1/auth/forgot-password`
Summary: Request password reset instructions
Allowed role/access: PUBLIC
Notes: Access: PUBLIC. Public endpoint. Returns the same success envelope for missing accounts to avoid account enumeration. Reports delivery failure only when the mail provider fails for an eligible account.
Request payload(s):
payload

```
{
  "email": "user@example.com"
}
```

Response body:

```
{
  "success": true,
  "message": "Reset instructions have been sent to the email address you provided."
}
```

POST `/api/v1/auth/verify-reset-otp`
Summary: Verify reset or verification OTP
Allowed role/access: PUBLIC
Notes: Access: PUBLIC. Verify OTP for password reset or unverified email flow.

Request payload(s):
payload

```
{
  "email": "user@example.com",
  "otp": "334018"
}
```

Response body:

```
{
  "success": true,
  "data": {
    "purpose": "EMAIL_VERIFICATION",
    "emailVerified": true
  },
  "message": "Email verified successfully"
}
```

POST `/api/v1/auth/reset-password`
Summary: Reset account password with OTP
Allowed role/access: PUBLIC
Notes: Access: PUBLIC. Public endpoint. Resets the password only when the supplied email and OTP are valid, without exposing whether the account exists.
Request payload(s):
payload

```
{
  "email": "user@example.com",
  "otp": "334018",
  "newPassword": "NewPassword@123"
}
```

Response body:

```
{
  "success": true,
  "message": "Password has been reset successfully."
}
```

PATCH `/api/v1/auth/change-password`
Summary: Update Auth Change Password
Allowed role/access: Authenticated
Notes: Access: Authenticated. Authenticated JWT required.
Request payload(s):
payload

```
{
  "currentPassword": "<string>",
  "newPassword": "<string>"
}
```

Response body:

```
{
  "success": true,
```

```
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET /api/v1/auth/me
Summary: List Auth Me
Allowed role/access: Authenticated
Notes: Access: Authenticated. Authenticated JWT required.
Response body:

```
{
  "success": true,
  "data": {
    "uid": "user_id",
    "role": "REGISTERED_USER",
    "email": "jane@example.com",
    "firstName": "Jane",
    "lastName": "Doe",
    "phone": "+923001234567",
    "avatarUrl": "https://cdn.yourdomain.com/user-avatars/jane.png",
    "permissions": null
  },
  "message": "Profile fetched successfully."
}
```

PATCH /api/v1/auth/me
Summary: Update Auth Me
Allowed role/access: Authenticated
Notes: Access: Authenticated. Authenticated JWT required.
Request payload(s):
payload

```
{
  "firstName": "Julian",
  "lastName": "Rivers",
  "phone": "+15551234567",
  "avatarUrl": "https://cdn.yourdomain.com/provider-avatars/julian.png"
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET /api/v1/auth/sessions
Summary: List Auth Sessions
Allowed role/access: Authenticated
Notes: Access: Authenticated. Authenticated JWT required.
Response body:

```
{
  "success": true,
```

```
  "data": [
    {
      "id": "session_id",
      "deviceName": "Chrome on Mac",
      "location": "Lahore, PK",
      "ipAddress": "203.0.113.10",
      "isCurrent": true,
      "lastActiveAt": "2026-04-08T11:45:00.000Z"
    }
  ],
  "message": "Active sessions fetched successfully."
}
```

POST `/api/v1/auth/sessions/logout-all`
Summary: Create Auth Sessions Logout All
Allowed role/access: Authenticated
Notes: Access: Authenticated. Authenticated JWT required.
Request payload(s):
payload

```
"<standard success envelope>"
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

DELETE `/api/v1/auth/sessions/{id}`
Summary: Delete Auth Sessions
Allowed role/access: Authenticated
Notes: Access: Authenticated. Authenticated JWT required.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload

```
"<standard success envelope>"
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

DELETE `/api/v1/auth/account`
Summary: Delete Auth Account
Allowed role/access: Authenticated
Notes: Access: Authenticated. Authenticated JWT required.

Request payload(s):
payload

```
"<standard success envelope>"
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

POST `/api/v1/auth/cancel-deletion`
Summary: Create Auth Cancel Deletion
Allowed role/access: Authenticated
Notes: Access: Authenticated. Authenticated JWT required.
Request payload(s):
payload

```
"<standard success envelope>"
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

01 Auth - Login Attempts

GET `/api/v1/login-attempts/stats`
Summary: List Login Attempts Stats
Allowed role/access: SUPER_ADMIN or ADMIN with loginAttempts.read
Notes: Access: SUPER_ADMIN or ADMIN with loginAttempts.read. SUPER_ADMIN or ADMIN with loginAttempts.read permission.
Parameters:

- `email` (query, optional, string)
- `status` (query, optional, string)
- `role` (query, optional, string)
- `page` (query, optional, number)
- `limit` (query, optional, number)
- `userId` (query, optional, string)
- `from` (query, optional, string)
- `to` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/login-attempts/export`
Summary: List Login Attempts Export
Allowed role/access: SUPER_ADMIN or ADMIN with loginAttempts.export
Notes: Access: SUPER_ADMIN or ADMIN with loginAttempts.export. SUPER_ADMIN or ADMIN with loginAttempts.export permission.
Parameters:

- `email` (query, optional, string)
- `status` (query, optional, string)
- `role` (query, optional, string)
- `page` (query, optional, number)
- `limit` (query, optional, number)
- `userId` (query, optional, string)
- `from` (query, optional, string)
- `to` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/login-attempts`
Summary: List Login Attempts
Allowed role/access: SUPER_ADMIN or ADMIN with loginAttempts.read
Notes: Access: SUPER_ADMIN or ADMIN with loginAttempts.read. SUPER_ADMIN or ADMIN with loginAttempts.read permission.
Parameters:

- `email` (query, optional, string)

- `status` (query, optional, string)
- `role` (query, optional, string)
- `page` (query, optional, number)
- `limit` (query, optional, number)
- `userId` (query, optional, string)
- `from` (query, optional, string)
- `to` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

02 Admin - Staff Management

GET `/api/v1/admins`
Summary: List admin staff users
Allowed role/access: SUPER_ADMIN
Notes: Access: SUPER_ADMIN. SUPER_ADMIN only. Admin staff management is controlled by Super Admin only. SUPER_ADMIN only. Returns User.role = ADMIN staff accounts only; SUPER_ADMIN accounts are intentionally excluded.
Parameters:

- `page` (query, optional, number)
- `limit` (query, optional, number)
- `search` (query, optional, string)
- `roleId` (query, optional, string)
- `role` (query, optional, string)
- `status` (query, optional, string)
- `sortBy` (query, optional, string)
- `sortOrder` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": [
    {
      "id": "admin_id",
      "firstName": "Operations",
      "lastName": "Staff",
      "fullName": "Operations Staff",
      "email": "staff@example.com",
      "phone": "+15550000002",
      "role": {
        "id": "admin_role_id",
        "name": "Gift Manager",
        "slug": "gift-manager"
      },
      "isActive": true,
      "isVerified": true,
      "createdAt": "2026-05-09T10:00:00.000Z",
      "lastLoginAt": null
    }
  ],
  "meta": {
    "page": 1,
    "limit": 10,
```

```
    "total": 1,
    "totalPages": 1
  },
  "message": "Admins fetched successfully"
}
```


POST /api/v1/admins
Summary: Create admin staff user
Allowed role/access: SUPER_ADMIN
Notes: Access: SUPER_ADMIN. SUPER_ADMIN only. Creates ADMIN staff users only. Creates an ADMIN staff user under Super Admin. The roleId field is the AdminRole ID that controls this staff user's permissions. This endpoint cannot create SUPER_ADMIN, REGISTERED_USER, PROVIDER, or GUEST_USER accounts.
Request payload(s):
payload


```
{
  "email": "staff@example.com",
  "temporaryPassword": "Temp@123456",
  "generateTemporaryPassword": false,
  "mustChangePassword": true,
  "firstName": "Operations",
  "lastName": "Staff",
  "phone": "+15550000002",
  "title": "Operations Manager",
  "roleId": "admin_role_id",
  "avatarUrl": "https://cdn.yourdomain.com/admin-avatars/staff.png",
  "isActive": true,
  "sendInviteEmail": true
}
```


Response body:


```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```


GET /api/v1/admins/{id}
Summary: Fetch Admins details
Allowed role/access: SUPER_ADMIN
Notes: Access: SUPER_ADMIN. SUPER_ADMIN only. Fetches ADMIN staff details.
Parameters:

- `id` (path, required, string)

Response body:


```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```


PATCH /api/v1/admins/{id}
Summary: Update admin staff profile or active status

Allowed role/access: SUPER_ADMIN
Notes: Access: SUPER_ADMIN. SUPER_ADMIN only. Updates ADMIN staff account details. SUPER_ADMIN only. Updates ADMIN staff profile, role assignment, and active/inactive status in one endpoint. Password updates are handled only by the dedicated password endpoint. SUPER_ADMIN self-disable and SUPER_ADMIN role updates are blocked.
Parameters:

- `id` (path, required, string)

Request payload(s):
updateAdminProfile

```
{
  "firstName": "Operations",
  "lastName": "Staff",
  "roleId": "admin_role_id"
}
```

activateAdmin

```
{
  "isActive": true,
  "reason": "Staff account re-enabled."
}
```

deactivateAdmin

```
{
  "isActive": false,
  "reason": "Staff account disabled after access review."
}
```

Response body:

```
{
  "success": true,
  "data": {
    "id": "admin_id",
    "firstName": "Operations",
    "lastName": "Staff",
    "role": {
      "id": "admin_role_id",
      "name": "Gift Manager"
    },
    "isActive": true
  },
  "message": "Admin updated successfully"
}
```

DELETE `/api/v1/admins/{id}`
Summary: Permanently delete admin staff user
Allowed role/access: SUPER_ADMIN
Notes: Access: SUPER_ADMIN. SUPER_ADMIN only. Permanently deletes an ADMIN staff account. DANGER: This endpoint permanently deletes an ADMIN staff account from the database. This is not a soft delete. Use only from Super Admin danger zone screens. SUPER_ADMIN accounts and self-delete are blocked.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload

```
"<standard success envelope>"
```

Response body:

```
{
  "success": true,
  "data": {
    "deletedAdminId": "admin_id"
  },
  "message": "Admin staff user permanently deleted successfully."
}
```

PATCH `/api/v1/admins/{id}/password`
Summary: Update Admins Password
Allowed role/access: SUPER_ADMIN
Notes: Access: SUPER_ADMIN. SUPER_ADMIN only. Changes ADMIN staff password from dashboard.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload

```
{
  "temporaryPassword": "<string>",
  "generateTemporaryPassword": true,
  "mustChangePassword": true,
  "sendEmail": true
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

02 Admin - Roles & Permissions

GET `/api/v1/admin-roles`
Summary: List Admin Roles
Allowed role/access: SUPER_ADMIN
Notes: Access: SUPER_ADMIN. SUPER_ADMIN only. Only SUPER_ADMIN can manage staff roles and permissions. Admin Roles / RBAC manages permission roles for ADMIN staff users only. SUPER_ADMIN has full immutable access and does not depend on AdminRole permissions.
Parameters:

- `search` (query, optional, string)
- `isSystem` (query, optional, boolean)
- `isActive` (query, optional, boolean)

Response body:

```
{
  "success": true,
```

```
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

POST `/api/v1/admin-roles`
Summary: Create Admin Roles
Allowed role/access: SUPER_ADMIN
Notes: Access: SUPER_ADMIN. SUPER_ADMIN only. ADMIN staff cannot create roles.
Request payload(s):
payload

```
{
  "name": "<string>",
  "description": "<string>",
  "permissions": {}
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/admin-roles/{id}`
Summary: Fetch Admin Roles details
Allowed role/access: SUPER_ADMIN
Notes: Access: SUPER_ADMIN. SUPER_ADMIN only. ADMIN staff cannot view role details.
Parameters:

- `id` (path, required, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

PATCH `/api/v1/admin-roles/{id}`
Summary: Update Admin Roles
Allowed role/access: SUPER_ADMIN
Notes: Access: SUPER_ADMIN. SUPER_ADMIN only. ADMIN staff cannot update roles.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload

```
{
  "name": "<string>",
  "description": "<string>",
```

```
  "isActive": true
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

DELETE `/api/v1/admin-roles/{id}`
Summary: Delete Admin Roles
Allowed role/access: SUPER_ADMIN
Notes: Access: SUPER_ADMIN. SUPER_ADMIN only. ADMIN staff cannot delete roles.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload

```
"<standard success envelope>"
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

PATCH `/api/v1/admin-roles/{id}/permissions`
Summary: Update Admin Roles Permissions
Allowed role/access: SUPER_ADMIN
Notes: Access: SUPER_ADMIN. SUPER_ADMIN only. ADMIN staff cannot update role permissions.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload

```
{
  "permissions": {}
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/permissions/catalog`
Summary: List Permissions Catalog
Allowed role/access: SUPER_ADMIN
Notes: Access: SUPER_ADMIN. SUPER_ADMIN only. Read-only backend permission catalog. Read-only list of backend-supported permission keys that can be assigned to admin roles.
Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

02 Admin - User Management

GET `/api/v1/users/export`
Summary: List Users Export
Allowed role/access: SUPER_ADMIN or ADMIN with users.export
Notes: Access: SUPER_ADMIN or ADMIN with users.export. SUPER_ADMIN or ADMIN with users.export permission.
Parameters:

- `search` (query, optional, string)
- `status` (query, optional, string)
- `registrationFrom` (query, optional, string)
- `registrationTo` (query, optional, string)
- `format` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/users`
Summary: List registered users
Allowed role/access: SUPER_ADMIN or ADMIN with users.read
Notes: Access: SUPER_ADMIN or ADMIN with users.read. SUPER_ADMIN or ADMIN with users.read permission. SUPER_ADMIN/ADMIN with users.read permission.
Parameters:

- `page` (query, optional, number)
- `limit` (query, optional, number)
- `search` (query, optional, string)
- `status` (query, optional, string)
- `registrationFrom` (query, optional, string)
- `registrationTo` (query, optional, string)
- `sortBy` (query, optional, string)
- `sortOrder` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": [
    {
```

```
      "id": "user_id",
      "email": "customer@example.com",
      "firstName": "Sarah",
      "lastName": "Johnson",
      "phone": "+923001234567",
      "role": "REGISTERED_USER",
      "isVerified": true,
      "isActive": true,
      "createdAt": "2026-05-09T10:00:00.000Z"
    }
  ],
  "meta": {
    "page": 1,
    "limit": 10,
    "total": 1,
    "totalPages": 1
  },
  "message": "Users fetched successfully"
}
```


GET `/api/v1/users/{id}`
Summary: Fetch Users details
Allowed role/access: SUPER_ADMIN or ADMIN with users.read
Notes: Access: SUPER_ADMIN or ADMIN with users.read. SUPER_ADMIN or ADMIN with users.read permission.
Parameters:

- `id` (path, required, string)

Response body:


```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```


PATCH `/api/v1/users/{id}`
Summary: Update Users
Allowed role/access: SUPER_ADMIN or ADMIN with users.update
Notes: Access: SUPER_ADMIN or ADMIN with users.update. SUPER_ADMIN or ADMIN with users.update permission.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload


```
{
  "firstName": "Alex",
  "lastName": "Johnson",
  "phone": "+15552345678",
  "avatarUrl": "https://<YOUR_PUBLIC_BUCKET_OR_CDN_URL>/user-avatars/avatar.jpg",
  "location": "New York, USA"
}
```


Response body:


```
{
  "success": true,
```

```
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

DELETE /api/v1/users/{id}
Summary: Permanently delete registered user
Allowed role/access: SUPER_ADMIN
Notes: Access: SUPER_ADMIN. SUPER_ADMIN only. Permanent delete danger-zone endpoint. DANGER: This endpoint permanently deletes/anonymizes the registered user and removes related non-financial data from the database. This is not a soft delete. Use only from Super Admin danger zone screens.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload

```
"<standard success envelope>"
```

Response body:

```
{
  "success": true,
  "data": {
    "deletedUserId": "user_id",
    "deletedRelatedRecords": true
  },
  "message": "User permanently deleted successfully."
}
```

PATCH /api/v1/users/{id}/status
Summary: Run registered user lifecycle action
Allowed role/access: SUPER_ADMIN or ADMIN with user lifecycle permission (UPDATE_STATUS/DISABLE/ENABLE=>users.status.update, SUSPEND=>users.suspend, UNSUSPEND=>users.unsuspend)
Notes: Access: SUPER_ADMIN or ADMIN with user lifecycle permission (UPDATE_STATUS/DISABLE/ENABLE=>users.status.update, SUSPEND=>users.suspend, UNSUSPEND=>users.unsuspend). SUPER_ADMIN or ADMIN with action-specific user lifecycle permission. UPDATE_STATUS, DISABLE, and ENABLE require users.status.update; SUSPEND requires users.suspend; UNSUSPEND requires users.unsuspend. SUPER_ADMIN or ADMIN with action-specific user lifecycle permission. UPDATE_STATUS, DISABLE, and ENABLE require 'users.status.update'; SUSPEND requires 'users.suspend'; UNSUSPEND requires 'users.unsuspend'.
Parameters:

- `id` (path, required, string)

Request payload(s):
updateStatus

```
{
  "action": "UPDATE_STATUS",
  "status": "ACTIVE",
  "reason": "OTHER",
  "comment": "Manual status update.",
  "notifyUser": true
}
```

suspendUser

```
{
  "action": "SUSPEND",
  "reason": "POLICY_VIOLATION",
  "comment": "User violated platform policy.",
  "notifyUser": true
}
```

unsuspendUser

```
{
  "action": "UNSUSPEND",
  "comment": "Account reviewed and restored.",
  "notifyUser": true
}
```

disableUser

```
{
  "action": "DISABLE",
  "reason": "OTHER",
  "comment": "Disabled after admin review.",
  "notifyUser": true
}
```

enableUser

```
{
  "action": "ENABLE",
  "comment": "Account enabled after verification.",
  "notifyUser": true
}
```

Response body:

```
{
  "success": true,
  "data": {
    "id": "user_id",
    "status": "SUSPENDED",
    "isActive": false,
    "suspension": {
      "reason": "POLICY_VIOLATION",
      "comment": "User violated platform policy.",
      "suspendedAt": "2026-05-25T10:00:00.000Z",
      "suspendedBy": "admin_id"
    }
  },
  "message": "User suspended successfully"
}
```

POST `/api/v1/users/{id}/reset-password`
Summary: Change registered user password
Allowed role/access: SUPER_ADMIN or ADMIN with users.resetPassword
Notes: Access: SUPER_ADMIN or ADMIN with users.resetPassword. SUPER_ADMIN or ADMIN with users.resetPassword permission. SUPER_ADMIN or ADMIN with users.resetPassword permission can change a REGISTERED_USER password from the dashboard. Optionally sends email and in-app notification to the user.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload

```
{
  "newPassword": "NewUser@123456",
  "sendEmail": true,
  "sendNotification": true,
  "reason": "Password changed by support request."
}
```

Response body:

```
{
  "success": true,
  "data": {
    "userId": "user_id",
    "email": "user@example.com",
    "emailSent": true,
    "notificationSent": true
  },
  "message": "User password changed successfully."
}
```

GET `/api/v1/users/{id}/activity`
Summary: Fetch Users Activity details
Allowed role/access: SUPER_ADMIN or ADMIN with users.read
Notes: Access: SUPER_ADMIN or ADMIN with users.read. SUPER_ADMIN or ADMIN with users.read permission.
Parameters:

- `id` (path, required, string)
- `page` (query, optional, number)
- `limit` (query, optional, number)
- `type` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/users/{id}/stats`
Summary: Fetch Users Stats details
Allowed role/access: SUPER_ADMIN or ADMIN with users.read
Notes: Access: SUPER_ADMIN or ADMIN with users.read. SUPER_ADMIN or ADMIN with users.read permission.
Parameters:

- `id` (path, required, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

02 Admin - Provider Management

GET `/api/v1/providers/export`
Summary: List Providers Export
Allowed role/access: SUPER_ADMIN or ADMIN with providers.export
Notes: Access: SUPER_ADMIN or ADMIN with providers.export. SUPER_ADMIN or ADMIN with providers.export permission.
Parameters:

- `search` (query, optional, string)
- `status` (query, optional, string)
- `approvalStatus` (query, optional, string)
- `format` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/providers/stats`
Summary: List Providers Stats
Allowed role/access: SUPER_ADMIN or ADMIN with providers.read
Notes: Access: SUPER_ADMIN or ADMIN with providers.read. SUPER_ADMIN or ADMIN with providers.read permission.
Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/providers`
Summary: List providers
Allowed role/access: SUPER_ADMIN or ADMIN with providers.read
Notes: Access: SUPER_ADMIN or ADMIN with providers.read. SUPER_ADMIN or ADMIN with providers.read permission. SUPER_ADMIN/ADMIN with providers.read permission.
Parameters:

- `page` (query, optional, number)
- `limit` (query, optional, number)
- `search` (query, optional, string)
- `status` (query, optional, string)
- `approvalStatus` (query, optional, string)
- `sortBy` (query, optional, string)
- `sortOrder` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": [
    {
      "id": "provider_id",
      "businessName": "Premium Gifts Co",
      "email": "provider@example.com",
      "phone": "+923001234567",
```

```
      "approvalStatus": "APPROVED",
      "isActive": true,
      "businessCategory": {
        "id": "category_id",
        "name": "Gift Supplier"
      },
      "createdAt": "2026-05-09T10:00:00.000Z"
    }
  ],
  "meta": {
    "page": 1,
    "limit": 10,
    "total": 1,
    "totalPages": 1
  },
  "message": "Providers fetched successfully"
}
```


POST `/api/v1/providers`

Summary: Create provider from admin dashboard

Allowed role/access: SUPER_ADMIN or ADMIN with providers.create

Notes: Access: SUPER_ADMIN or ADMIN with providers.create. SUPER_ADMIN or ADMIN with providers.create permission. SUPER_ADMIN or ADMIN with providers.create permission. Creates a PROVIDER account and provider business profile. Supports same business fields as provider self-registration, plus temporary password and invite email flow.

Request payload(s):

payload

```
{
  "email": "contact@giftsandblooms.com",
  "firstName": "Ali",
  "lastName": "Raza",
  "phone": "+15551234567",
  "businessName": "Gifts & Blooms Co. Ltd",
  "businessCategoryId": "provider_business_category_id",
  "taxId": "TAX-12345",
  "businessAddress": "123 Gift Street",
  "serviceArea": "New York, USA",
  "headquarters": "New York, USA",
  "fulfillmentMethods": [
    "PICKUP",
    "DELIVERY"
  ],
  "autoAcceptOrders": false,
  "documentUrls": [
    "https://cdn.yourdomain.com/provider-documents/license.pdf"
  ],
  "generateTemporaryPassword": true,
  "temporaryPassword": "Provider@123456",
  "mustChangePassword": true,
  "sendInviteEmail": true,
  "approvalStatus": "PENDING",
  "isActive": true
}
```

Response body:


```
{
  "success": true,
  "data": {
    "id": "provider_id",
    "userId": "provider_id",
```

```
    "email": "contact@giftsandblooms.com",
    "businessName": "Gifts & Blooms Co. Ltd",
    "approvalStatus": "PENDING",
    "isActive": true,
    "inviteEmailSent": true
  },
  "message": "Provider created successfully and invite email sent."
}
```


GET `/api/v1/providers/lookup`
Summary: List Providers Lookup
Allowed role/access: SUPER_ADMIN or ADMIN with providers.read
Notes: Access: SUPER_ADMIN or ADMIN with providers.read. SUPER_ADMIN or ADMIN with providers.read permission.
Parameters:

- `search` (query, optional, string)
- `approvalStatus` (query, optional, string)
- `isActive` (query, optional, boolean)
- `limit` (query, optional, number)

Response body:


```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```


GET `/api/v1/providers/{id}`
Summary: Fetch Providers details
Allowed role/access: SUPER_ADMIN or ADMIN with providers.read
Notes: Access: SUPER_ADMIN or ADMIN with providers.read. SUPER_ADMIN or ADMIN with providers.read permission.
Parameters:

- `id` (path, required, string)

Response body:


```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```


PATCH `/api/v1/providers/{id}`
Summary: Update Providers
Allowed role/access: SUPER_ADMIN or ADMIN with providers.update
Notes: Access: SUPER_ADMIN or ADMIN with providers.update. SUPER_ADMIN or ADMIN with providers.update permission.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload

```
{
  "businessName": "Gifts & Blooms Co. Ltd",
  "phone": "+15551234567",
  "serviceArea": "New York, USA",
  "headquarters": "New York, USA",
  "avatarUrl": "https://<YOUR_PUBLIC_BUCKET_OR_CDN_URL>/provider-logos/logo.png",
  "documentUrls": [
    "<string>"
  ]
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

DELETE `/api/v1/providers/{id}`
Summary: Permanently delete provider
Allowed role/access: SUPER_ADMIN
Notes: Access: SUPER_ADMIN. SUPER_ADMIN only. Permanent delete danger-zone endpoint. DANGER: This endpoint permanently deletes/anonymizes the provider and related provider data from the database. This is not a soft delete. Use only from Super Admin danger zone screens. Active processing orders block deletion.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload

```
"<standard success envelope>"
```

Response body:

```
{
  "success": true,
  "data": {
    "deletedProviderId": "provider_id",
    "deletedRelatedRecords": true
  },
  "message": "Provider permanently deleted successfully."
}
```

PATCH `/api/v1/providers/{id}/status`
Summary: Update provider lifecycle status
Allowed role/access: SUPER_ADMIN or ADMIN with provider lifecycle permission (APPROVE=>providers.approve, REJECT=>providers.reject, SUSPEND=>providers.suspend, UNSUSPEND=>providers.suspend, UPDATE_STATUS=>providers.updateStatus)
Notes: Access: SUPER_ADMIN or ADMIN with provider lifecycle permission (APPROVE=>providers.approve, REJECT=>providers.reject, SUSPEND=>providers.suspend, UNSUSPEND=>providers.suspend, UPDATE_STATUS=>providers.updateStatus). SUPER_ADMIN or ADMIN with lifecycle permission. APPROVE requires providers.approve; REJECT requires providers.reject; SUSPEND and UNSUSPEND require providers.suspend; UPDATE_STATUS requires providers.updateStatus. SUPER_ADMIN or ADMIN with provider lifecycle permission. APPROVE requires providers.approve, REJECT requires providers.reject, SUSPEND and UNSUSPEND require providers.suspend, UPDATE_STATUS requires providers.updateStatus. Uses action-based request body.

Parameters:

- `id` (path, required, string)

Request payload(s):

approveProvider

```
{
  "action": "APPROVE",
  "comment": "Documents verified successfully.",
  "notifyProvider": true
}
```

rejectProvider

```
{
  "action": "REJECT",
  "reason": "INCOMPLETE_DOCUMENTS",
  "comment": "Business license document is missing.",
  "notifyProvider": true
}
```

updateStatus

```
{
  "action": "UPDATE_STATUS",
  "status": "ACTIVE",
  "reason": "OTHER",
  "comment": "Provider account restored after review.",
  "notifyProvider": true
}
```

suspendProvider

```
{
  "action": "SUSPEND",
  "reason": "POLICY_VIOLATION",
  "comment": "Provider violated platform policy.",
  "notifyProvider": true
}
```

unsuspendProvider

```
{
  "action": "UNSUSPEND",
  "comment": "Provider account reviewed and restored.",
  "notifyProvider": true
}
```

Response body:

```
{
  "success": true,
  "data": {
    "id": "provider_id",
    "approvalStatus": "APPROVED",
    "status": "ACTIVE",
    "isActive": true
  },
```

```
  "message": "Provider approved successfully."
}
```

GET `/api/v1/providers/{id}/items`
Summary: Fetch Providers Items details
Allowed role/access: SUPER_ADMIN or ADMIN with providers.read
Notes: Access: SUPER_ADMIN or ADMIN with providers.read. SUPER_ADMIN or ADMIN with providers.read permission.
Parameters:

- `id` (path, required, string)
- `page` (query, optional, number)
- `limit` (query, optional, number)
- `search` (query, optional, string)
- `status` (query, optional, string)
- `sortBy` (query, optional, string)
- `sortOrder` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/providers/{id}/activity`
Summary: Fetch Providers Activity details
Allowed role/access: SUPER_ADMIN or ADMIN with providers.read
Notes: Access: SUPER_ADMIN or ADMIN with providers.read. SUPER_ADMIN or ADMIN with providers.read permission.
Parameters:

- `id` (path, required, string)
- `page` (query, optional, number)
- `limit` (query, optional, number)
- `type` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

POST `/api/v1/providers/{id}/message`
Summary: Create Providers Message
Allowed role/access: SUPER_ADMIN or ADMIN with providers.message
Notes: Access: SUPER_ADMIN or ADMIN with providers.message. SUPER_ADMIN or ADMIN with providers.message permission.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload

```
{
  "subject": "Account update",
  "message": "Please update your business documents.",
  "channel": "EMAIL"
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

02 Admin - Provider Business Categories

GET `/api/v1/provider-business-categories`
Summary: List provider business categories
Allowed role/access: PUBLIC
Notes: Access: PUBLIC. Lists provider business categories. By default returns all non-deleted categories. Use isActive=true or isActive=false to filter by active state. Lists provider business categories. By default returns all non-deleted categories. Use isActive=true or isActive=false to filter by active state.
Parameters:

- `page` (query, optional, number)
- `limit` (query, optional, number)
- `search` (query, optional, string)
- `isActive` (query, optional, boolean)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

POST `/api/v1/provider-business-categories`
Summary: Create provider business category
Allowed role/access: SUPER_ADMIN or ADMIN with providerBusinessCategories.create
Notes: Access: SUPER_ADMIN or ADMIN with providerBusinessCategories.create. SUPER_ADMIN or ADMIN with providerBusinessCategories.create permission. SUPER_ADMIN or ADMIN with providerBusinessCategories.create permission only.
Request payload(s):
payload

```
{
  "name": "<string>",
  "description": "<string>",
  "iconKey": "<string>",
  "sortOrder": 1.0,
  "isActive": true
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/provider-business-categories/lookup`
Summary: Lookup active provider business categories
Allowed role/access: PUBLIC
Notes: Access: PUBLIC. Provider signup dropdown. Returns active provider business categories only. Public/provider-signup dropdown. Returns active provider business categories only.
Parameters:

- `page` (query, optional, number)
- `limit` (query, optional, number)
- `search` (query, optional, string)
- `isActive` (query, optional, boolean)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/provider-business-categories/{id}`
Summary: Fetch provider business category details
Allowed role/access: SUPER_ADMIN or ADMIN with providerBusinessCategories.read
Notes: Access: SUPER_ADMIN or ADMIN with providerBusinessCategories.read. SUPER_ADMIN or ADMIN with providerBusinessCategories.read permission. SUPER_ADMIN or ADMIN with providerBusinessCategories.read permission only.
Parameters:

- `id` (path, required, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

PATCH `/api/v1/provider-business-categories/{id}`
Summary: Update provider business category
Allowed role/access: SUPER_ADMIN or ADMIN with providerBusinessCategories.update
Notes: Access: SUPER_ADMIN or ADMIN with providerBusinessCategories.update. SUPER_ADMIN or ADMIN with providerBusinessCategories.update permission. SUPER_ADMIN or ADMIN with providerBusinessCategories.update permission only.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload

```
{
  "name": "<string>",
  "description": "<string>",
  "iconKey": "<string>",
  "sortOrder": 1.0,
  "isActive": true
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

DELETE `/api/v1/provider-business-categories/{id}`
Summary: Delete provider business category
Allowed role/access: SUPER_ADMIN or ADMIN with providerBusinessCategories.delete
Notes: Access: SUPER_ADMIN or ADMIN with providerBusinessCategories.delete. SUPER_ADMIN or ADMIN with providerBusinessCategories.delete permission. SUPER_ADMIN or ADMIN with providerBusinessCategories.delete permission. Permanently deletes the category; refuses deletion when active providers are attached.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload

```
"<standard success envelope>"
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

02 Admin - Promotional Offers Management

GET `/api/v1/promotional-offers/stats`
Summary: List Promotional Offers Stats
Allowed role/access: SUPER_ADMIN or ADMIN with promotionalOffers.read
Notes: Access: SUPER_ADMIN or ADMIN with promotionalOffers.read. SUPER_ADMIN or ADMIN with promotionalOffers.read permission.
Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/promotional-offers/export`
Summary: List Promotional Offers Export
Allowed role/access: SUPER_ADMIN or ADMIN with promotionalOffers.export
Notes: Access: SUPER_ADMIN or ADMIN with promotionalOffers.export. SUPER_ADMIN or ADMIN with promotionalOffers.export permission.
Parameters:

- `page` (query, optional, number)
- `limit` (query, optional, number)
- `search` (query, optional, string)
- `status` (query, optional, string)
- `itemId` (query, optional, string)
- `sortBy` (query, optional, string)
- `sortOrder` (query, optional, string)
- `providerId` (query, optional, string)
- `approvalStatus` (query, optional, string)
- `discountType` (query, optional, string)
- `startFrom` (query, optional, string)
- `startTo` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/promotional-offers`
Summary: List Promotional Offers
Allowed role/access: SUPER_ADMIN or ADMIN with promotionalOffers.read
Notes: Access: SUPER_ADMIN or ADMIN with promotionalOffers.read. SUPER_ADMIN or ADMIN with promotionalOffers.read permission.
Parameters:

- `page` (query, optional, number)
- `limit` (query, optional, number)
- `search` (query, optional, string)
- `status` (query, optional, string)
- `itemId` (query, optional, string)
- `sortBy` (query, optional, string)
- `sortOrder` (query, optional, string)
- `providerId` (query, optional, string)
- `approvalStatus` (query, optional, string)
- `discountType` (query, optional, string)
- `startFrom` (query, optional, string)
- `startTo` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

POST `/api/v1/promotional-offers`
Summary: Create Promotional Offers
Allowed role/access: SUPER_ADMIN or ADMIN with promotionalOffers.create

Notes: Access: SUPER_ADMIN or ADMIN with promotionalOffers.create. SUPER_ADMIN or ADMIN with promotionalOffers.create permission.
Request payload(s):
payload

```
{
  "itemId": "<string>",
  "title": "<string>",
  "description": "<string>",
  "discountType": "PERCENTAGE",
  "discountValue": 1.0,
  "startDate": "<string>",
  "endDate": "<string>",
  "eligibilityRules": "<string>",
  "isActive": true,
  "providerId": "<string>",
  "approvalStatus": "PENDING"
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/promotional-offers/{id}`
Summary: Fetch Promotional Offers details
Allowed role/access: SUPER_ADMIN or ADMIN with promotionalOffers.read
Notes: Access: SUPER_ADMIN or ADMIN with promotionalOffers.read. SUPER_ADMIN or ADMIN with promotionalOffers.read permission.
Parameters:

- `id` (path, required, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

PATCH `/api/v1/promotional-offers/{id}`
Summary: Update Promotional Offers
Allowed role/access: SUPER_ADMIN or ADMIN with promotionalOffers.update
Notes: Access: SUPER_ADMIN or ADMIN with promotionalOffers.update. SUPER_ADMIN or ADMIN with promotionalOffers.update permission.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload

```
{
  "title": "<string>",
  "description": "<string>",
```

```
  "discountType": "PERCENTAGE",
  "discountValue": 1.0,
  "startDate": "<string>",
  "endDate": "<string>",
  "eligibilityRules": "<string>",
  "isActive": true,
  "status": "ACTIVE",
  "reason": "<string>"
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

DELETE `/api/v1/promotional-offers/{id}`
Summary: Delete Promotional Offers
Allowed role/access: SUPER_ADMIN or ADMIN with promotionalOffers.delete
Notes: Access: SUPER_ADMIN or ADMIN with promotionalOffers.delete. SUPER_ADMIN or ADMIN with promotionalOffers.delete permission.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload

```
"<standard success envelope>"
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

POST `/api/v1/promotional-offers/{id}/action`
Summary: Run promotional offer admin action
Allowed role/access: SUPER_ADMIN or ADMIN with promotional offer action permission (APPROVE=>promotionalOffers.approve, REJECT=>promotionalOffers.reject, ACTIVATE/DEACTIVATE=>promotionalOffers.status.update)
Notes: Access: SUPER_ADMIN or ADMIN with promotional offer action permission (APPROVE=>promotionalOffers.approve, REJECT=>promotionalOffers.reject, ACTIVATE/DEACTIVATE=>promotionalOffers.status.update). SUPER_ADMIN or ADMIN with action-specific promotional offer permission. APPROVE requires promotionalOffers.approve; REJECT requires promotionalOffers.reject; ACTIVATE and DEACTIVATE require promotionalOffers.status.update. SUPER_ADMIN or ADMIN with action-specific promotional offer permission. APPROVE requires 'promotionalOffers.approve'; REJECT requires 'promotionalOffers.reject'; ACTIVATE and DEACTIVATE require 'promotionalOffers.status.update'.
Parameters:

- `id` (path, required, string)

Request payload(s):
approve

```
{
  "action": "APPROVE",
  "comment": "Offer meets campaign rules.",
  "notifyProvider": true
}
```

reject

```
{
  "action": "REJECT",
  "reason": "INVALID_DISCOUNT",
  "comment": "Discount exceeds allowed campaign threshold.",
  "notifyProvider": true
}
```

activate

```
{
  "action": "ACTIVATE",
  "comment": "Offer is ready for publication.",
  "notifyProvider": true
}
```

deactivate

```
{
  "action": "DEACTIVATE",
  "reason": "OTHER",
  "comment": "Offer paused by admin.",
  "notifyProvider": true
}
```

Response body:

```
{
  "success": true,
  "data": {
    "id": "offer_id",
    "approvalStatus": "APPROVED",
    "status": "ACTIVE",
    "isActive": true
  },
  "message": "Promotional offer approved successfully"
}
```

02 Admin - Dashboard Overview

GET `/api/v1/admin/dashboard`
Summary: Fetch Super Admin dashboard data
Allowed role/access: SUPER_ADMIN or ADMIN with dashboard.read
Notes: Access: SUPER_ADMIN or ADMIN with dashboard.read. SUPER_ADMIN or ADMIN with dashboard.read permission. Read-only merged dashboard overview, trends, gift/payment distribution, provider performance, and recent disputes. Returns overview metrics, revenue trends, gift/payment distribution, provider performance, and recent disputes from real records only. Payment secrets, card data, provider bank data, dispute evidence, and private internal notes are never selected or returned.
Parameters:

- `range` (query, optional, string)

- `fromDate` (query, optional, string)
- `toDate` (query, optional, string)

Response body:


```
{
  "success": true,
  "data": {
    "overview": {
      "totalUsers": 128430,
      "totalUsersDeltaPercent": 12.5,
      "totalProviders": 1240,
      "totalProvidersDeltaPercent": 5.2,
      "transactions": 45200,
      "transactionsDeltaPercent": 18.1,
      "totalRevenue": 1240000,
      "totalRevenueDeltaPercent": 10.3
    },
    "revenueTrends": {
      "range": "LAST_12_MONTHS",
      "labels": [
        "Jan",
        "Feb",
        "Mar"
      ],
      "values": [
        12000,
        28000,
        16000
      ]
    },
    "giftVsPayment": {
      "giftCardsPercent": 65,
      "directPaymentsPercent": 35
    },
    "providerPerformance": [
      {
        "providerId": "provider_id",
        "providerName": "Gift Provider",
        "successRate": 99.2,
        "totalVolume": 450230
      }
    ],
    "recentDisputes": [
      {
        "id": "dispute_id",
        "caseId": "DISP-9021",
        "userName": "Marcus Wright",
        "reason": "Unauthorized transaction",
        "status": "HIGH_PRIORITY"
      }
    ]
  },
  "message": "Dashboard data fetched successfully."
}
```


02 Admin - Platform Analytics

`GET` `/api/v1/admin/platform-analytics/summary`
Summary: Fetch platform analytics summary

Allowed role/access: SUPER_ADMIN or ADMIN with analytics.read
Notes: Access: SUPER_ADMIN or ADMIN with analytics.read. SUPER_ADMIN or ADMIN with analytics.read permission. Uses real payment/subscription/user records. Calculates revenue from successful payments only and compares the selected range with the previous equivalent period.
Parameters:

- `range` (query, optional, string)
- `fromDate` (query, optional, string)
- `toDate` (query, optional, string)
- `categoryId` (query, optional, string)
- `providerId` (query, optional, string)
- `search` (query, optional, string)

Response body:


```
{
  "success": true,
  "data": {
    "totalRevenue": {
      "value": 154320,
      "changePercent": 8.1
    },
    "newSubscriptions": {
      "value": 215,
      "changePercent": 5.3
    },
    "churnRate": {
      "value": 2.9,
      "changePercent": 0.1
    },
    "activeUsers": {
      "value": 5120,
      "changePercent": -2.1
    }
  },
  "message": "Platform analytics summary fetched successfully."
}
```


GET `/api/v1/admin/platform-analytics/revenue-transactions`
Summary: List recent revenue transactions
Allowed role/access: SUPER_ADMIN or ADMIN with analytics.read
Notes: Access: SUPER_ADMIN or ADMIN with analytics.read. SUPER_ADMIN or ADMIN with analytics.read permission. Card/payment secrets are not returned. Uses real payment, order, provider order, gift, category, provider, and subscription records. Card/payment secrets are never selected or returned.
Parameters:

- `range` (query, optional, string)
- `fromDate` (query, optional, string)
- `toDate` (query, optional, string)
- `categoryId` (query, optional, string)
- `providerId` (query, optional, string)
- `search` (query, optional, string)
- `page` (query, optional, number)
- `limit` (query, optional, number)
- `status` (query, optional, string)
- `plan` (query, optional, string)
- `sortBy` (query, optional, string)
- `sortOrder` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": [
    {
      "id": "transaction_id",
      "date": "2023-09-17T00:00:00.000Z",
      "userEmail": "alex.rivera@gmail.com",
      "plan": "Pro",
      "amount": 150,
      "status": "COMPLETED",
      "currency": "PKR",
      "provider": {
        "id": "provider_id",
        "businessName": "Gift Provider"
      },
      "category": {
        "id": "category_id",
        "name": "Flowers"
      }
    }
  ],
  "meta": {
    "page": 1,
    "limit": 10,
    "total": 1,
    "totalPages": 1
  },
  "message": "Revenue transactions fetched successfully."
}
```


GET `/api/v1/admin/platform-analytics/filter-options`
Summary: Fetch platform analytics filter options
Allowed role/access: SUPER_ADMIN or ADMIN with analytics.read
Notes: Access: SUPER_ADMIN or ADMIN with analytics.read. SUPER_ADMIN or ADMIN with analytics.read permission. Categories are active/non-deleted, providers are active approved providers, and plans come from subscription plans.
Response body:


```
{
  "success": true,
  "data": {
    "categories": [
      {
        "id": "category_id",
        "name": "Flowers"
      }
    ],
    "providers": [
      {
        "id": "provider_id",
        "businessName": "Gift Provider"
      }
    ],
    "plans": [
      "Free",
      "Pro",
      "Enterprise"
    ],
    "statuses": [
      "COMPLETED",
      "PENDING",
      "FAILED",
```

```
      "REFUNDED"
    ]
  },
  "message": "Platform analytics filter options fetched successfully."
}
```


GET `/api/v1/admin/platform-analytics/report`
Summary: Generate platform analytics report
Allowed role/access: SUPER_ADMIN or ADMIN with analytics.export
Notes: Access: SUPER_ADMIN or ADMIN with analytics.export. SUPER_ADMIN or ADMIN with analytics.export permission. Export excludes payment/card/bank secrets. Streams a CSV report using the same transaction filters and excludes card numbers, CVV, Stripe secrets, payment client secrets, and bank details.
Parameters:

- `range` (query, optional, string)
- `fromDate` (query, optional, string)
- `toDate` (query, optional, string)
- `categoryId` (query, optional, string)
- `providerId` (query, optional, string)
- `search` (query, optional, string)
- `format` (query, optional, string)
- `status` (query, optional, string)

Response body:


```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```


02 Admin - Commission & Payout Settings

GET `/api/v1/admin/payout-settings`
Summary: Fetch commission and payout settings
Allowed role/access: SUPER_ADMIN or ADMIN with payoutSettings.read
Notes: Access: SUPER_ADMIN or ADMIN with payoutSettings.read. SUPER_ADMIN or ADMIN with payoutSettings.read permission. Returns global platform commission, payout threshold/schedule, and active provider commission tiers. Settings affect future payout calculations only and do not rewrite historical payouts.
Response body:


```
{
  "success": true,
  "data": {
    "platformRatePercent": 5,
    "minimumPayoutThreshold": 100,
    "currency": "USD",
    "payoutSchedule": "MONTHLY_LAST_DAY",
    "payoutTimeUtc": "00:00",
    "autoPayoutEnabled": true,
    "lastUpdatedAt": "2026-05-16T10:00:00.000Z",
    "commissionTiers": [
      {
        "id": "tier_standard",
        "name": "Standard Tier",
        "commissionRatePercent": 15,
        "orderVolumeThreshold": 0,
```

```
        "sortOrder": 1,
        "isActive": true
      },
      {
        "id": "tier_silver",
        "name": "Silver Partner",
        "commissionRatePercent": 12.5,
        "orderVolumeThreshold": 5000,
        "sortOrder": 2,
        "isActive": true
      }
    ]
  },
  "message": "Payout settings fetched successfully."
}
```


PATCH `/api/v1/admin/payout-settings`
Summary: Update commission and payout settings
Allowed role/access: SUPER_ADMIN or ADMIN with payoutSettings.update
Notes: Access: SUPER_ADMIN or ADMIN with payoutSettings.update. SUPER_ADMIN or ADMIN with payoutSettings.update permission. Applies to future payout calculations only. Updates global settings for future payout calculations only; existing historical payouts are not rewritten or recalculated.
Request payload(s):
update


```
{
  "platformRatePercent": 5,
  "minimumPayoutThreshold": 100,
  "currency": "USD",
  "payoutSchedule": "MONTHLY_LAST_DAY",
  "payoutTimeUtc": "00:00",
  "autoPayoutEnabled": true
}
```


Response body:


```
{
  "success": true,
  "data": {
    "platformRatePercent": 5,
    "minimumPayoutThreshold": 100,
    "currency": "USD",
    "payoutSchedule": "MONTHLY_LAST_DAY",
    "payoutTimeUtc": "00:00",
    "autoPayoutEnabled": true,
    "lastUpdatedAt": "2026-05-16T10:00:00.000Z"
  },
  "message": "Payout settings updated successfully."
}
```


GET `/api/v1/admin/payout-settings/commission-tiers`
Summary: List commission tiers
Allowed role/access: SUPER_ADMIN or ADMIN with payoutSettings.read
Notes: Access: SUPER_ADMIN or ADMIN with payoutSettings.read. SUPER_ADMIN or ADMIN with payoutSettings.read permission. Returns active commission tiers ordered by sort order and threshold.
Response body:

```
{
  "success": true,
  "data": [
    {
      "id": "tier_standard",
      "name": "Standard Tier",
      "commissionRatePercent": 15,
      "orderVolumeThreshold": 0,
      "sortOrder": 1,
      "isActive": true
    }
  ],
  "message": "Commission tiers fetched successfully."
}
```


POST `/api/v1/admin/payout-settings/commission-tiers`
Summary: Create commission tier
Allowed role/access: SUPER_ADMIN or ADMIN with payoutSettings.update
Notes: Access: SUPER_ADMIN or ADMIN with payoutSettings.update. SUPER_ADMIN or ADMIN with payoutSettings.update permission. Creates a provider commission tier for future billing/payout cycles only. Thresholds must be unique among active tiers.
Request payload(s):
create


```
{
  "name": "Gold Elite",
  "commissionRatePercent": 10,
  "orderVolumeThreshold": 15000,
  "sortOrder": 3,
  "isActive": true
}
```


Response body:


```
{
  "success": true,
  "data": {
    "id": "tier_gold",
    "name": "Gold Elite",
    "commissionRatePercent": 10,
    "orderVolumeThreshold": 15000,
    "sortOrder": 3,
    "isActive": true
  },
  "message": "Commission tier created successfully."
}
```


PATCH `/api/v1/admin/payout-settings/commission-tiers/{id}`
Summary: Update commission tier
Allowed role/access: SUPER_ADMIN or ADMIN with payoutSettings.update
Notes: Access: SUPER_ADMIN or ADMIN with payoutSettings.update. SUPER_ADMIN or ADMIN with payoutSettings.update permission. Applies next billing/payout cycle. Tier changes take effect at the start of the next billing/payout cycle and do not rewrite historical payouts.
Parameters:

- `id` (path, required, string)

Request payload(s):

update

```
{
  "name": "Gold Elite",
  "commissionRatePercent": 10,
  "orderVolumeThreshold": 15000,
  "sortOrder": 3,
  "isActive": true
}
```

Response body:

```
{
  "success": true,
  "data": {
    "id": "tier_gold",
    "name": "Gold Elite",
    "commissionRatePercent": 10,
    "orderVolumeThreshold": 15000,
    "sortOrder": 3,
    "isActive": true
  },
  "message": "Commission tier updated successfully."
}
```

DELETE `/api/v1/admin/payout-settings/commission-tiers/{id}`
Summary: Delete commission tier
Allowed role/access: SUPER_ADMIN or ADMIN with payoutSettings.update
Notes: Access: SUPER_ADMIN or ADMIN with payoutSettings.update. SUPER_ADMIN or ADMIN with payoutSettings.update permission. Permanently deletes the tier record. Permanently deletes the tier record.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload

```
"<standard success envelope>"
```

Response body:

```
{
  "success": true,
  "data": {
    "id": "tier_gold",
    "deleted": true
  },
  "message": "Commission tier deleted successfully."
}
```

GET `/api/v1/admin/payout-settings/audit-logs`
Summary: List payout settings audit logs
Allowed role/access: SUPER_ADMIN or ADMIN with payoutSettings.read
Notes: Access: SUPER_ADMIN or ADMIN with payoutSettings.read. SUPER_ADMIN or ADMIN with payoutSettings.read permission. Returns audit logs for payout settings and commission tier changes, including sanitized before/after JSON.
Parameters:

- `page` (query, optional, number)

- `limit` (query, optional, number)
- `fromDate` (query, optional, string)
- `toDate` (query, optional, string)

Response body:


```
{
  "success": true,
  "data": [
    {
      "id": "audit_log_id",
      "action": "PAYOUT_SETTINGS_UPDATED",
      "actor": {
        "id": "admin_id",
        "name": "Alex Rivera"
      },
      "targetType": "PAYOUT_SETTINGS",
      "before": {
        "platformRatePercent": 5
      },
      "after": {
        "platformRatePercent": 6
      },
      "createdAt": "2026-05-16T10:00:00.000Z"
    }
  ],
  "meta": {
    "page": 1,
    "limit": 10,
    "total": 1,
    "totalPages": 1
  },
  "message": "Payout settings audit logs fetched successfully."
}
```

02 Admin - Provider Payouts

GET `/api/v1/admin/provider-payouts/stats`
Summary: Fetch provider payout dashboard stats
Allowed role/access: SUPER_ADMIN or ADMIN with providerPayouts.read
Notes: Access: SUPER_ADMIN or ADMIN with providerPayouts.read. SUPER_ADMIN or ADMIN with providerPayouts.read permission. Aggregates from provider payout records only; no frontend totals are trusted.
Response body:

```
{
  "success": true,
  "data": {
    "totalPayoutsThisMonth": 128430,
    "totalPayoutsDeltaPercent": 12.5,
    "pendingPayouts": 12250,
    "pendingPayoutsDeltaPercent": -2.4,
    "completedPayouts": 116180,
    "completedPayoutsDeltaPercent": 14.2,
    "platformRevenue": 19264.5,
    "platformRevenueDeltaPercent": 8.1,
    "currency": "USD"
  },
  "message": "Provider payout stats fetched successfully."
}
```

GET `/api/v1/admin/provider-payouts/trends`
Summary: Fetch monthly provider payout trend
Allowed role/access: SUPER_ADMIN or ADMIN with providerPayouts.read
Notes: Access: SUPER_ADMIN or ADMIN with providerPayouts.read. SUPER_ADMIN or ADMIN with providerPayouts.read permission. Returns monthly payout totals for the selected 3, 6, or 12 month range from provider payout records.
Parameters:

- `range` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": {
    "range": "LAST_12_MONTHS",
    "labels": [
      "Jan",
      "Feb",
      "Mar"
    ],
    "values": [
      12000,
      28000,
      16000
    ],
    "currency": "USD"
  },
  "message": "Provider payout trends fetched successfully."
}
```

GET `/api/v1/admin/provider-payouts/earning-distribution`
Summary: Fetch earning distribution by provider tier

Allowed role/access: SUPER_ADMIN or ADMIN with providerPayouts.read
Notes: Access: SUPER_ADMIN or ADMIN with providerPayouts.read. SUPER_ADMIN or ADMIN with providerPayouts.read permission. Uses provider earnings ledger totals and active commission tier thresholds.
Response body:


```
{
  "success": true,
  "data": [
    {
      "tierId": "tier_silver",
      "tierName": "Silver Partner",
      "providerCount": 18,
      "totalEarnings": 450230,
      "currency": "USD"
    }
  ],
  "message": "Provider earning distribution fetched successfully."
}
```


GET `/api/v1/admin/provider-payouts/export`
Summary: Export provider payouts
Allowed role/access: SUPER_ADMIN or ADMIN with providerPayouts.export
Notes: Access: SUPER_ADMIN or ADMIN with providerPayouts.export. SUPER_ADMIN or ADMIN with providerPayouts.export permission. Uses list filters and excludes full bank account numbers. Applies the same filters as list and never exports full bank account numbers.
Parameters:

- `page` (query, optional, number)
- `limit` (query, optional, number)
- `search` (query, optional, string)
- `status` (query, optional, string)
- `providerId` (query, optional, string)
- `fromDate` (query, optional, string)
- `toDate` (query, optional, string)
- `sortBy` (query, optional, string)
- `sortOrder` (query, optional, string)
- `format` (query, optional, string)

Response body:


```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```


POST `/api/v1/admin/provider-payouts/bulk-action`
Summary: Run bulk provider payout action
Allowed role/access: SUPER_ADMIN or ADMIN with providerPayouts.approve
Notes: Access: SUPER_ADMIN or ADMIN with providerPayouts.approve. SUPER_ADMIN or ADMIN with providerPayouts.approve permission. APPROVE is currently the only enabled bulk action and returns per-item idempotent results. SUPER_ADMIN or ADMIN with action-specific providerPayouts permission. APPROVE requires providerPayouts.approve and is currently the only enabled bulk action. HOLD and REJECT are reserved for future bulk logic and return a validation error for now. Approval remains idempotent per payout: already PROCESSING/COMPLETED payouts are returned as successful idempotent items and are not processed twice.
Request payload(s):
approve

```
{
  "action": "APPROVE",
  "payoutIds": [
    "payout_id_1",
    "payout_id_2"
  ],
  "reason": "COMPLIANCE_REVIEW",
  "comment": "Bulk processed after finance review.",
  "notifyProvider": true
}
```

holdUnsupported

```
{
  "action": "HOLD",
  "payoutIds": [
    "payout_id_1"
  ],
  "reason": "COMPLIANCE_REVIEW",
  "comment": "Reserved for future bulk hold support.",
  "notifyProvider": true
}
```

rejectUnsupported

```
{
  "action": "REJECT",
  "payoutIds": [
    "payout_id_1"
  ],
  "reason": "COMPLIANCE_REJECTED",
  "comment": "Reserved for future bulk reject support.",
  "notifyProvider": true
}
```

Response body:

```
{
  "success": true,
  "data": [
    {
      "payoutId": "payout_id_1",
      "status": "PROCESSING",
      "success": true,
      "idempotent": false
    },
    {
      "payoutId": "payout_id_2",
      "status": "PROCESSING",
      "success": true,
      "idempotent": true
    }
  ],
  "message": "Bulk payout action processed."
}
```

GET `/api/v1/admin/provider-payouts`
Summary: List provider payouts
Allowed role/access: SUPER_ADMIN or ADMIN with providerPayouts.read

Notes: Access: SUPER_ADMIN or ADMIN with providerPayouts.read. SUPER_ADMIN or ADMIN with providerPayouts.read permission. Supports status/provider/date/search filters and sorting. Bank account data is masked only.
Parameters:

- `page` (query, optional, number)
- `limit` (query, optional, number)
- `search` (query, optional, string)
- `status` (query, optional, string)
- `providerId` (query, optional, string)
- `fromDate` (query, optional, string)
- `toDate` (query, optional, string)
- `sortBy` (query, optional, string)
- `sortOrder` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": [
    {
      "id": "payout_id",
      "provider": {
        "id": "provider_id",
        "businessName": "TechSolutions Inc.",
        "providerCode": "PRV-90210",
        "avatarUrl": "https://cdn.example.com/provider.png"
      },
      "pendingAmount": 3420,
      "currency": "USD",
      "lastPayoutDate": "2023-10-12T00:00:00.000Z",
      "nextPayoutDate": "2023-11-12T00:00:00.000Z",
      "status": "COMPLETED"
    }
  ],
  "meta": {
    "page": 1,
    "limit": 10,
    "total": 1,
    "totalPages": 1
  },
  "message": "Provider payouts fetched successfully."
}
```

GET `/api/v1/admin/provider-payouts/{id}/breakdown`
Summary: Fetch pending payout transaction breakdown
Allowed role/access: SUPER_ADMIN or ADMIN with providerPayouts.read
Notes: Access: SUPER_ADMIN or ADMIN with providerPayouts.read. SUPER_ADMIN or ADMIN with providerPayouts.read permission. Shows gross amount, platform fee, processing fee, net payout, and recent ledger transactions without full bank details.
Parameters:

- `id` (path, required, string)

Response body:

```
{
  "success": true,
  "data": {
    "payoutId": "payout_id",
    "provider": {
      "id": "provider_id",
      "businessName": "CloudTech Solutions",
```

```
      "merchantId": "MER-29401-2023"
    },
    "grossAmount": 4200,
    "platformFee": 420,
    "platformFeePercent": 10,
    "processingFee": 12.5,
    "netPayout": 3767.5,
    "currency": "USD",
    "recentTransactions": [
      {
        "orderNumber": "88392",
        "description": "Subscription - Professional Plan",
        "amount": 199
      }
    ]
  },
  "message": "Payout breakdown fetched successfully."
}
```


POST `/api/v1/admin/provider-payouts/{id}/action`

Summary: Run provider payout action

Allowed role/access: SUPER_ADMIN or ADMIN with action-specific providerPayouts permission

Notes: Access: SUPER_ADMIN or ADMIN with action-specific providerPayouts permission. APPROVE requires providerPayouts.approve, HOLD requires providerPayouts.hold, REJECT requires providerPayouts.reject. APPROVE requires providerPayouts.approve and moves PENDING/ON_HOLD payouts to PROCESSING. HOLD requires providerPayouts.hold, is allowed from PENDING, keeps ledger balance locked, and requires reason. REJECT requires providerPayouts.reject, is allowed from PENDING/ON_HOLD, requires reason, and releases locked PAYOUT_PENDING ledger balance back to AVAILABLE.

Parameters:

- `id` (path, required, string)

Request payload(s):

approve


```
{
  "action": "APPROVE",
  "comment": "Approved after verification.",
  "notifyProvider": true
}
```


hold


```
{
  "action": "HOLD",
  "reason": "BANK_VERIFICATION_PENDING",
  "comment": "Bank verification required.",
  "notifyProvider": true
}
```


reject


```
{
  "action": "REJECT",
  "reason": "INVALID_BANK_ACCOUNT",
  "comment": "Bank details are invalid.",
  "notifyProvider": true
}
```


Response body:

```
{
  "success": true,
  "data": {
    "id": "payout_id",
    "status": "PROCESSING"
  },
  "message": "Payout action processed successfully."
}
```


GET `/api/v1/admin/provider-payouts/{id}`
Summary: Fetch provider payout details
Allowed role/access: SUPER_ADMIN or ADMIN with providerPayouts.read
Notes: Access: SUPER_ADMIN or ADMIN with providerPayouts.read. SUPER_ADMIN or ADMIN with providerPayouts.read permission. Payout destination is masked. Returns payout details, provider display data, and masked payout destination only.
Parameters:

- `id` (path, required, string)

Response body:


```
{
  "success": true,
  "data": {
    "id": "payout_id",
    "provider": {
      "id": "provider_id",
      "businessName": "TechSolutions Inc.",
      "providerCode": "PRV-90210",
      "avatarUrl": "https://cdn.example.com/provider.png"
    },
    "pendingAmount": 3420,
    "amount": 3420,
    "processingFee": 42,
    "totalToReceive": 3378,
    "currency": "USD",
    "status": "COMPLETED",
    "destination": {
      "id": "method_id",
      "bankName": "Chase Bank",
      "maskedAccount": "****1234",
      "last4": "1234",
      "verificationStatus": "VERIFIED"
    },
    "createdAt": "2023-10-12T00:00:00.000Z"
  },
  "message": "Provider payout details fetched successfully."
}
```


02 Admin - Transaction Monitoring

GET `/api/v1/admin/transactions/stats`
Summary: Fetch transaction monitoring stats
Allowed role/access: SUPER_ADMIN or ADMIN with transactions.read
Notes: Access: SUPER_ADMIN or ADMIN with transactions.read. SUPER_ADMIN or ADMIN with transactions.read permission. Stats are calculated from real payment/transaction records and return zeros when no records match.

Parameters:

- `range` (query, optional, string)
- `fromDate` (query, optional, string)
- `toDate` (query, optional, string)
- `transactionType` (query, optional, string)
- `status` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": {
    "totalVolume": 124500,
    "totalVolumeDeltaPercent": 12,
    "successRate": 98.2,
    "successRateDeltaPercent": 2.1,
    "pendingReview": 14,
    "failedToday": 3,
    "failedTodayDeltaPercent": -5,
    "currency": "PKR"
  },
  "message": "Transaction stats fetched successfully."
}
```

GET `/api/v1/admin/transactions/export`
Summary: Export admin transactions
Allowed role/access: SUPER_ADMIN or ADMIN with transactions.export
Notes: Access: SUPER_ADMIN or ADMIN with transactions.export. SUPER_ADMIN or ADMIN with transactions.export permission. Excludes card and payment secrets. Applies the same filters as the list API and excludes raw card numbers, CVV, client secrets, and Stripe secret fields.
Parameters:

- `range` (query, optional, string)
- `fromDate` (query, optional, string)
- `toDate` (query, optional, string)
- `transactionType` (query, optional, string)
- `status` (query, optional, string)
- `page` (query, optional, number)
- `limit` (query, optional, number)
- `search` (query, optional, string)
- `gatewayProvider` (query, optional, string)
- `userId` (query, optional, string)
- `providerId` (query, optional, string)
- `minAmount` (query, optional, number)
- `maxAmount` (query, optional, number)
- `sortBy` (query, optional, string)
- `sortOrder` (query, optional, string)
- `format` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/admin/transactions`
Summary: List admin transactions

Allowed role/access: SUPER_ADMIN or ADMIN with transactions.read
Notes: Access: SUPER_ADMIN or ADMIN with transactions.read. SUPER_ADMIN or ADMIN with transactions.read permission. Admin-side financial monitoring endpoint; customer transaction history remains under /api/v1/customer/transactions. Search supports transaction ID, order number, customer name/email, gateway reference, and provider business name.
Parameters:

- `range` (query, optional, string)
- `fromDate` (query, optional, string)
- `toDate` (query, optional, string)
- `transactionType` (query, optional, string)
- `status` (query, optional, string)
- `page` (query, optional, number)
- `limit` (query, optional, number)
- `search` (query, optional, string)
- `gatewayProvider` (query, optional, string)
- `userId` (query, optional, string)
- `providerId` (query, optional, string)
- `minAmount` (query, optional, number)
- `maxAmount` (query, optional, number)
- `sortBy` (query, optional, string)
- `sortOrder` (query, optional, string)

Response body:


```
{
  "success": true,
  "data": [
    {
      "id": "transaction_id",
      "transactionId": "TXN-882194",
      "user": {
        "id": "user_id",
        "name": "Sarah Jenkins",
        "avatarUrl": "https://cdn.yourdomain.com/user-avatars/sarah.png"
      },
      "gatewayProvider": "STRIPE",
      "type": "PAYMENT",
      "amount": 1250,
      "currency": "PKR",
      "status": "SUCCESS",
      "createdAt": "2026-10-24T14:20:00.000Z"
    }
  ],
  "meta": {
    "page": 1,
    "limit": 10,
    "total": 1,
    "totalPages": 1
  },
  "message": "Transactions fetched successfully."
}
```


GET `/api/v1/admin/transactions/{id}/timeline`
Summary: Fetch transaction timeline
Allowed role/access: SUPER_ADMIN or ADMIN with transactions.read
Notes: Access: SUPER_ADMIN or ADMIN with transactions.read. SUPER_ADMIN or ADMIN with transactions.read permission. Returns ordered payment, refund, dispute, notification, and audit events without card/payment secrets.
Parameters:

- `id` (path, required, string)

Response body:

```
{
  "success": true,
  "data": [
    {
      "status": "INITIATED",
      "title": "Initiated",
      "description": "Checkout session started by user via mobile application.",
      "source": "User Session",
      "timestamp": "2026-11-24T14:30:02.000Z"
    },
    {
      "status": "COMPLETED",
      "title": "Completed",
      "description": "Funds successfully transferred to the merchant escrow account.",
      "source": "System Auto-Update",
      "timestamp": "2026-11-24T14:32:10.000Z"
    }
  ],
  "message": "Transaction timeline fetched successfully."
}
```


GET `/api/v1/admin/transactions/{id}/receipt`
Summary: Download transaction receipt
Allowed role/access: SUPER_ADMIN or ADMIN with transactions.receipt.download
Notes: Access: SUPER_ADMIN or ADMIN with transactions.receipt.download. SUPER_ADMIN or ADMIN with transactions.receipt.download permission. Returns a PDF-compatible receipt file with transaction ID, order ID, customer display info, amount breakdown, gateway provider, and masked payment method only.
Parameters:

- `id` (path, required, string)

Response body:


```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```


POST `/api/v1/admin/transactions/{id}/action`
Summary: Run transaction action
Allowed role/access: SUPER_ADMIN or ADMIN with transaction action permission (REFUND=>transactions.refund, OPEN_DISPUTE=>transactions.openDispute, NOTIFY_USER=>transactions.notifyUser)
Notes: Access: SUPER_ADMIN or ADMIN with transaction action permission (REFUND=>transactions.refund, OPEN_DISPUTE=>transactions.openDispute, NOTIFY_USER=>transactions.notifyUser). SUPER_ADMIN or ADMIN with action-specific transaction permission. Refund amount is server-validated, duplicate open disputes are blocked, and notifications use NotificationDispatchService. SUPER_ADMIN or ADMIN with action-specific transaction permission. REFUND requires 'transactions.refund' and keeps existing server-side refund validation. OPEN_DISPUTE requires 'transactions.openDispute' and keeps duplicate dispute prevention. NOTIFY_USER requires 'transactions.notifyUser' and dispatches through NotificationDispatchService. Raw card numbers, CVV, Stripe secret keys, and payment intent client secrets are never exposed.
Parameters:

- `id` (path, required, string)

Request payload(s):
refund

```
{
  "action": "REFUND",
  "refundType": "FULL",
  "refundAmount": 1281.25,
  "reason": "CUSTOMER_REQUEST",
  "comment": "Refund approved by support.",
  "notifyUser": true
}
```

openDispute

```
{
  "action": "OPEN_DISPUTE",
  "reason": "PRODUCT_NOT_RECEIVED",
  "priority": "HIGH",
  "claimDetails": "Dispute opened from transaction detail screen.",
  "assignToId": "admin_id"
}
```

notifyUser

```
{
  "action": "NOTIFY_USER",
  "channel": "EMAIL",
  "subject": "Transaction update",
  "message": "Your transaction has been successfully processed.",
  "includeReceipt": true
}
```

Response body:

```
{
  "success": true,
  "data": {
    "transactionId": "TRX-982341",
    "refundId": "RF-45678",
    "refundAmount": 1281.25,
    "currency": "PKR",
    "status": "REFUNDED"
  },
  "message": "Transaction refunded successfully."
}
```

GET `/api/v1/admin/transactions/{id}`
Summary: Fetch transaction details
Allowed role/access: SUPER_ADMIN or ADMIN with transactions.read
Notes: Access: SUPER_ADMIN or ADMIN with transactions.read. SUPER_ADMIN or ADMIN with transactions.read permission. Card/payment secrets are masked. Shows payment breakdown, gateway information, customer info, related records, and refund state. Raw card numbers, CVV, Stripe secret keys, and payment intent client secrets are never exposed.
Parameters:

- `id` (path, required, string)

Response body:

```
{
  "success": true,
  "data": {
    "id": "transaction_id",
```

```
    "transactionId": "TRX-982341",
    "status": "SUCCESS",
    "type": "PAYMENT",
    "currency": "PKR",
    "paymentBreakdown": {
      "subtotal": 1250,
      "processingFee": 31.25,
      "processingFeePercent": 2.5,
      "totalAmount": 1281.25
    },
    "gatewayInformation": {
      "provider": "STRIPE",
      "gatewayReference": "REF-XP-382341",
      "paymentMethod": "Visa **** 4242",
      "settlementStatus": "CLEARED",
      "processorAuthCode": "AUTH-9921-X"
    },
    "customer": {
      "id": "user_id",
      "name": "Julianne Doe",
      "email": "julianne.doe@example.com",
      "location": "San Francisco, CA, USA",
      "kycStatus": "KYC Level 2 Verified"
    },
    "relatedRecords": {
      "orderId": "order_id",
      "orderNumber": "ORD-88421",
      "paymentId": "payment_id",
      "subscriptionId": null,
      "moneyGiftId": null,
      "walletLedgerId": null
    },
    "refund": {
      "isRefundable": true,
      "refundedAmount": 0,
      "remainingRefundableAmount": 1281.25,
      "refundWindowEndsAt": "2026-11-24T00:00:00.000Z"
    },
    "createdAt": "2026-10-24T14:20:00.000Z"
  },
  "message": "Transaction details fetched successfully."
}
```


02 Admin - Message Moderation

GET `/api/v1/admin/message-moderation/conversations`
Summary: List flagged message moderation conversations
Allowed role/access: SUPER_ADMIN or ADMIN with messageModeration.read
Notes: Access: SUPER_ADMIN or ADMIN with messageModeration.read. SUPER_ADMIN or ADMIN with messageModeration.read permission. Harmful message previews are redacted. Supports page, limit, search, chatType, status, severity, flagReason, senderRole, participantType, assignedToAdminId, fromDate, toDate, sortBy, sortOrder.
Parameters:

- `page` (query, optional, number)
- `limit` (query, optional, number)
- `search` (query, optional, string)
- `chatType` (query, optional, string)
- `status` (query, optional, string)
- `severity` (query, optional, string)
- `flagReason` (query, optional, string)

- `senderRole` (query, optional, string)
- `participantType` (query, optional, string)
- `assignedToAdminId` (query, optional, string)
- `source` (query, optional, string)
- `flagType` (query, optional, string)
- `assignedToId` (query, optional, string)
- `fromDate` (query, optional, string)
- `toDate` (query, optional, string)
- `sortBy` (query, optional, string)
- `sortOrder` (query, optional, string)

Response body:


```
{
  "success": true,
  "data": [
    {
      "id": "conversation_id",
      "chatType": "BUYER_PROVIDER",
      "status": "PENDING_REVIEW",
      "severity": "HIGH",
      "flaggedMessageCount": 2,
      "lastFlaggedAt": "2026-05-18T10:00:00.000Z",
      "participants": [
        {
          "id": "customer_id",
          "type": "REGISTERED_USER",
          "name": "Sarah Johnson",
          "avatarUrl": null
        },
        {
          "id": "provider_id",
          "type": "PROVIDER",
          "name": "Dcodax Gifts",
          "avatarUrl": null
        }
      ],
      "lastMessagePreview": "Please pay outside the platform...",
      "flagReasons": [
        "OFF_PLATFORM_PAYMENT",
        "AUTO_FLAG_KEYWORD"
      ],
      "assignedTo": null
    }
  ],
  "meta": {
    "page": 1,
    "limit": 10,
    "total": 1,
    "totalPages": 1
  },
  "message": "Flagged message conversations fetched successfully."
}
```


GET `/api/v1/admin/message-moderation/stats`
Summary: Fetch message moderation stats
Allowed role/access: SUPER_ADMIN or ADMIN with messageModeration.read
Notes: Access: SUPER_ADMIN or ADMIN with messageModeration.read. SUPER_ADMIN or ADMIN with messageModeration.read permission. Calculated from real moderation records.
Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/admin/message-moderation/filter-options`
Summary: Fetch message moderation filter options
Allowed role/access: SUPER_ADMIN or ADMIN with messageModeration.read
Notes: Access: SUPER_ADMIN or ADMIN with messageModeration.read. SUPER_ADMIN or ADMIN with messageModeration.read permission.
Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/admin/message-moderation/export`
Summary: Export message moderation rows
Allowed role/access: SUPER_ADMIN or ADMIN with messageModeration.export
Notes: Access: SUPER_ADMIN or ADMIN with messageModeration.export. SUPER_ADMIN or ADMIN with messageModeration.export permission. Export is redacted. Export contains moderation-safe redacted fields only, not full private conversation history.
Parameters:

- `page` (query, optional, number)
- `limit` (query, optional, number)
- `search` (query, optional, string)
- `chatType` (query, optional, string)
- `status` (query, optional, string)
- `severity` (query, optional, string)
- `flagReason` (query, optional, string)
- `senderRole` (query, optional, string)
- `participantType` (query, optional, string)
- `assignedToAdminId` (query, optional, string)
- `source` (query, optional, string)
- `flagType` (query, optional, string)
- `assignedToId` (query, optional, string)
- `fromDate` (query, optional, string)
- `toDate` (query, optional, string)
- `sortBy` (query, optional, string)
- `sortOrder` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/admin/message-moderation/audit-logs`
Summary: Fetch message moderation audit logs
Allowed role/access: SUPER_ADMIN or ADMIN with messageModeration.auditLogs.read

Notes: Access: SUPER_ADMIN or ADMIN with messageModeration.auditLogs.read. SUPER_ADMIN or ADMIN with messageModeration.auditLogs.read permission.
Parameters:

- `page` (query, optional, number)
- `limit` (query, optional, number)
- `conversationId` (query, optional, string)
- `messageId` (query, optional, string)
- `actorAdminId` (query, optional, string)
- `action` (query, optional, string)
- `fromDate` (query, optional, string)
- `toDate` (query, optional, string)
- `sortBy` (query, optional, string)
- `sortOrder` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/admin/message-moderation/conversations/{id}`
Summary: Fetch message moderation conversation detail
Allowed role/access: SUPER_ADMIN or ADMIN with messageModeration.read
Notes: Access: SUPER_ADMIN or ADMIN with messageModeration.read. SUPER_ADMIN or ADMIN with messageModeration.read permission. Flagged body is null by default; redactedBody is returned. Returns participants, flagged messages, internal notes, and action history.
Parameters:

- `id` (path, required, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/admin/message-moderation/conversations/{id}/history`
Summary: Fetch paginated message moderation conversation history
Allowed role/access: SUPER_ADMIN or ADMIN with messageModeration.read
Notes: Access: SUPER_ADMIN or ADMIN with messageModeration.read. SUPER_ADMIN or ADMIN with messageModeration.read permission. Defaults to paginated scoped history around flagged messages and masks sensitive payment/contact data.
Parameters:

- `id` (path, required, string)
- `beforeMessageId` (query, optional, string)
- `afterMessageId` (query, optional, string)
- `page` (query, optional, number)
- `limit` (query, optional, number)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
```

```
  "message": "Request completed successfully."
}
```

POST `/api/v1/admin/message-moderation/messages/{messageId}/action`
Summary: Run message moderation action
Allowed role/access: SUPER_ADMIN or ADMIN with message moderation action permission (HIDE_MESSAGE/RESTORE_MESSAGE/DISMISS_FLAG=>messageModeration.moderate, WARN_SENDER=>messageModeration.warn, SUSPEND_SENDER=>messageModeration.suspend, ADD_NOTE=>messageModeration.notes.create, REPROCESS=>messageModeration.reprocess, ESCALATE=>messageModeration.escalate)
Notes: Access: SUPER_ADMIN or ADMIN with message moderation action permission (HIDE_MESSAGE/RESTORE_MESSAGE/DISMISS_FLAG=>messageModeration.moderate, WARN_SENDER=>messageModeration.warn, SUSPEND_SENDER=>messageModeration.suspend, ADD_NOTE=>messageModeration.notes.create, REPROCESS=>messageModeration.reprocess, ESCALATE=>messageModeration.escalate). SUPER_ADMIN or ADMIN with action-specific message moderation permission. HIDE_MESSAGE, RESTORE_MESSAGE, and DISMISS_FLAG require messageModeration.moderate; WARN_SENDER requires messageModeration.warn; SUSPEND_SENDER requires messageModeration.suspend; ADD_NOTE requires messageModeration.notes.create; REPROCESS requires messageModeration.reprocess; ESCALATE requires messageModeration.escalate. SUPER_ADMIN or ADMIN with action-specific message moderation permission. HIDE_MESSAGE, RESTORE_MESSAGE, and DISMISS_FLAG require 'messageModeration.moderate'; WARN_SENDER requires 'messageModeration.warn'; SUSPEND_SENDER requires 'messageModeration.suspend'; ADD_NOTE requires 'messageModeration.notes.create'; REPROCESS requires 'messageModeration.reprocess'; ESCALATE requires 'messageModeration.escalate'.
Parameters:

- `messageId` (path, required, string)

Request payload(s):
hideMessage

```
{
  "action": "HIDE_MESSAGE",
  "reason": "OFF_PLATFORM_PAYMENT",
  "comment": "Message asks the customer to pay outside the platform.",
  "notifySender": true,
  "notifyParticipants": true
}
```

restoreMessage

```
{
  "action": "RESTORE_MESSAGE",
  "reason": "FALSE_POSITIVE",
  "comment": "Message was incorrectly flagged.",
  "notifyParticipants": true
}
```

warnSender

```
{
  "action": "WARN_SENDER",
  "reason": "POLICY_WARNING",
  "comment": "Sender warned for policy violation.",
  "notifySender": true,
  "severity": "MEDIUM"
}
```

suspendSender

```
{
  "action": "SUSPEND_SENDER",
  "reason": "REPEATED_POLICY_VIOLATION",
```

```
  "comment": "Sender suspended after repeated violations.",
  "notifySender": true
}
```

dismissFlag

```
{
  "action": "DISMISS_FLAG",
  "reason": "NOT_A_VIOLATION",
  "comment": "Flag dismissed after manual review."
}
```

addNote

```
{
  "action": "ADD_NOTE",
  "comment": "Internal note added for future reviews."
}
```

reprocess

```
{
  "action": "REPROCESS",
  "reason": "POLICY_UPDATED",
  "comment": "Re-run scanner using the current policy."
}
```

escalate

```
{
  "action": "ESCALATE",
  "reason": "SECURITY_REVIEW_REQUIRED",
  "comment": "Escalated for security review.",
  "severity": "HIGH",
  "assignToAdminId": "admin_id"
}
```

Response body:

```
{
  "success": true,
  "data": {
    "messageId": "message_id",
    "status": "ACTION_TAKEN"
  },
  "message": "Message hidden successfully."
}
```

02 Admin - Social Moderation

GET `/api/v1/admin/social-moderation/stats`
Summary: Fetch social moderation stats
Allowed role/access: SUPER_ADMIN or ADMIN with socialModeration.read
Notes: Access: SUPER_ADMIN or ADMIN with socialModeration.read. SUPER_ADMIN or ADMIN with socialModeration.read permission.

Parameters:

- `range` (query, optional, string)
- `fromDate` (query, optional, string)
- `toDate` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": {
    "totalFlaggedPosts": 245,
    "pendingReports": 38,
    "highSeverityReports": 12,
    "removedPosts": 19,
    "hiddenPosts": 44,
    "warningsSent": 28
  },
  "message": "Social moderation stats fetched successfully."
}
```

GET `/api/v1/admin/social-moderation/export`
Summary: Export social moderation log
Allowed role/access: SUPER_ADMIN or ADMIN with socialModeration.export
Notes: Access: SUPER_ADMIN or ADMIN with socialModeration.export. SUPER_ADMIN or ADMIN with socialModeration.export permission. Exports moderation-safe fields only.
Parameters:

- `page` (query, optional, number)
- `limit` (query, optional, number)
- `search` (query, optional, string)
- `reportType` (query, optional, string)
- `status` (query, optional, string)
- `severity` (query, optional, string)
- `fromDate` (query, optional, string)
- `toDate` (query, optional, string)
- `sortBy` (query, optional, string)
- `sortOrder` (query, optional, string)
- `format` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/admin/social-moderation/reports`
Summary: List social moderation reports
Allowed role/access: SUPER_ADMIN or ADMIN with socialModeration.read
Notes: Access: SUPER_ADMIN or ADMIN with socialModeration.read. SUPER_ADMIN or ADMIN with socialModeration.read permission. Search supports post content, user name, username, report ID, and post ID.
Parameters:

- `page` (query, optional, number)
- `limit` (query, optional, number)
- `search` (query, optional, string)
- `reportType` (query, optional, string)
- `status` (query, optional, string)

- `severity` (query, optional, string)
- `fromDate` (query, optional, string)
- `toDate` (query, optional, string)
- `sortBy` (query, optional, string)
- `sortOrder` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/admin/social-moderation/reports/{id}`
Summary: Fetch social report inspection details
Allowed role/access: SUPER_ADMIN or ADMIN with socialModeration.read
Notes: Access: SUPER_ADMIN or ADMIN with socialModeration.read. SUPER_ADMIN or ADMIN with socialModeration.read permission. Returns post inspection drawer data and report history.
Parameters:

- `id` (path, required, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

POST `/api/v1/admin/social-moderation/reports/{id}/action`
Summary: Run social moderation action
Allowed role/access: SUPER_ADMIN or ADMIN with socialModeration.moderate
Notes: Access: SUPER_ADMIN or ADMIN with socialModeration.moderate. SUPER_ADMIN or ADMIN with socialModeration.moderate permission. HIDE/REMOVE/WARN_USER create moderation logs and audit logs; posts are not physically deleted.
Parameters:

- `id` (path, required, string)

Request payload(s):
hide

```
{
  "action": "HIDE",
  "reason": "SPAM",
  "comment": "Post hidden due to deceptive link reports.",
  "notifyUser": true
}
```

remove

```
{
  "action": "REMOVE",
  "reason": "DECEPTIVE_LINK",
  "comment": "Post removed after manual review.",
```

```
  "notifyUser": true
}
```

warn

```
{
  "action": "WARN_USER",
  "reason": "INAPPROPRIATE_BEHAVIOR",
  "comment": "Warning issued for repeated spam behavior.",
  "notifyUser": true
}
```

reviewed

```
{
  "action": "MARK_REVIEWED",
  "comment": "Reviewed and no additional action required.",
  "notifyUser": false
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

02 Admin - Social Reporting Rules

GET `/api/v1/admin/social-reporting-rules/stats`
Summary: Fetch social reporting rule stats
Allowed role/access: SUPER_ADMIN or ADMIN with socialReportingRules.read
Notes: Access: SUPER_ADMIN or ADMIN with socialReportingRules.read. SUPER_ADMIN or ADMIN with socialReportingRules.read permission.
Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/admin/social-reporting-rules/export`
Summary: Export social reporting rules
Allowed role/access: SUPER_ADMIN or ADMIN with socialReportingRules.export
Notes: Access: SUPER_ADMIN or ADMIN with socialReportingRules.export. SUPER_ADMIN or ADMIN with socialReportingRules.export permission.
Parameters:

- `format` (query, optional, string)
- `isActive` (query, optional, boolean)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/admin/social-reporting-rules`
Summary: List social reporting rules
Allowed role/access: SUPER_ADMIN or ADMIN with socialReportingRules.read
Notes: Access: SUPER_ADMIN or ADMIN with socialReportingRules.read. SUPER_ADMIN or ADMIN with socialReportingRules.read permission.
Parameters:

- `page` (query, optional, number)
- `limit` (query, optional, number)
- `search` (query, optional, string)
- `isActive` (query, optional, boolean)
- `reportCategory` (query, optional, string)
- `sortBy` (query, optional, string)
- `sortOrder` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

POST `/api/v1/admin/social-reporting-rules`
Summary: Create social reporting rule
Allowed role/access: SUPER_ADMIN or ADMIN with socialReportingRules.create
Notes: Access: SUPER_ADMIN or ADMIN with socialReportingRules.create. SUPER_ADMIN or ADMIN with socialReportingRules.create permission.
Request payload(s):
payload

```
{
  "reportCategory": "<string>",
  "label": "<string>",
  "description": "<string>",
  "iconKey": "<string>",
  "autoFlagThreshold": 1.0,
  "escalationRule": "AUTO_HIDE_CONTENT",
  "isActive": true
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/admin/social-reporting-rules/{id}`
Summary: Fetch social reporting rule details
Allowed role/access: SUPER_ADMIN or ADMIN with socialReportingRules.read
Notes: Access: SUPER_ADMIN or ADMIN with socialReportingRules.read. SUPER_ADMIN or ADMIN with socialReportingRules.read permission.
Parameters:

- `id` (path, required, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

PATCH `/api/v1/admin/social-reporting-rules/{id}`
Summary: Update social reporting rule
Allowed role/access: SUPER_ADMIN or ADMIN with socialReportingRules.update
Notes: Access: SUPER_ADMIN or ADMIN with socialReportingRules.update. SUPER_ADMIN or ADMIN with socialReportingRules.update permission. Standard rule fields and isActive status changes both go through this endpoint.
Parameters:

- `id` (path, required, string)

Request payload(s):
updateRule

```
{
  "label": "Spam",
  "description": "Spam or advertising content.",
  "iconKey": "spam",
  "autoFlagThreshold": 10,
  "escalationRule": "MANUAL_REVIEW"
}
```

activateRule

```
{
  "label": "Spam",
  "isActive": true,
  "reason": "Rule enabled for campaign period."
}
```

deactivateRule

```
{
  "label": "Spam",
  "isActive": false,
  "reason": "Rule paused after campaign period."
}
```

Response body:

```
{
  "success": true,
  "data": {
    "id": "rule_1",
```

```
    "label": "Spam",
    "isActive": true,
    "autoFlagThreshold": 10,
    "escalationRule": "MANUAL_REVIEW"
  },
  "message": "Social reporting rule updated successfully."
}
```


DELETE `/api/v1/admin/social-reporting-rules/{id}`
Summary: Delete social reporting rule
Allowed role/access: SUPER_ADMIN or ADMIN with socialReportingRules.delete
Notes: Access: SUPER_ADMIN or ADMIN with socialReportingRules.delete. SUPER_ADMIN or ADMIN with socialReportingRules.delete permission. Permanently deletes the rule record.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload


```
"<standard success envelope>"
```


Response body:


```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```


02 Admin - Notification Delivery Monitoring

GET `/api/v1/admin/notification-delivery/stats`
Summary: Fetch notification delivery stats
Allowed role/access: Authenticated
Notes: Access: Authenticated. Authenticated JWT required. SUPER_ADMIN or ADMIN with notifications.read.
Response body:


```
{
  "success": true,
  "data": {
    "total": 12,
    "queued": 0,
    "delivered": 10,
    "failed": 1,
    "skipped": 1,
    "retried": 2
  },
  "message": "Notification delivery stats fetched successfully."
}
```


GET `/api/v1/admin/notification-delivery/logs`
Summary: List notification delivery logs
Allowed role/access: Authenticated

Notes: Access: Authenticated. Authenticated JWT required. SUPER_ADMIN or ADMIN with notifications.read.
Parameters:

- `page` (query, optional, number)
- `limit` (query, optional, number)
- `status` (query, optional, string)
- `recipientType` (query, optional, string)
- `notificationType` (query, optional, string)
- `recipientId` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/admin/notification-delivery/logs/{id}`
Summary: Fetch notification delivery log detail
Allowed role/access: Authenticated
Notes: Access: Authenticated. Authenticated JWT required. SUPER_ADMIN or ADMIN with notifications.read.
Parameters:

- `id` (path, required, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

POST `/api/v1/admin/notification-delivery/logs/{id}/retry`
Summary: Retry notification delivery
Allowed role/access: Authenticated
Notes: Access: Authenticated. Authenticated JWT required. SUPER_ADMIN or ADMIN with notifications.delivery.retry. Retries non-IN_APP delivery channels without duplicating the existing in-app notification.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload

```
"<standard success envelope>"
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

02 Admin - Referral Settings

`GET` `/api/v1/referral-settings`
Summary: Fetch referral settings
Allowed role/access: SUPER_ADMIN or ADMIN with referralSettings.read
Notes: Access: SUPER_ADMIN or ADMIN with referralSettings.read. SUPER_ADMIN or ADMIN with referralSettings.read permission. Customer referral APIs consume these settings. Pending referrals use the settings snapshot stored at referral creation.
Response body:


```
{
  "success": true,
  "data": {
    "isActive": true,
    "referrerRewardAmount": 25,
    "newUserRewardAmount": 10,
    "rewardCurrency": "USD",
    "minimumTransactionAmount": 50,
    "referralExpirationValue": 30,
    "referralExpirationUnit": "DAYS",
    "allowSelfReferrals": false,
    "qualificationRule": "FIRST_SUCCESSFUL_PURCHASE",
    "updatedAt": "2026-05-09T10:00:00.000Z"
  },
  "message": "Referral settings fetched successfully."
}
```


`PATCH` `/api/v1/referral-settings`
Summary: Update referral settings
Allowed role/access: SUPER_ADMIN
Notes: Access: SUPER_ADMIN. SUPER_ADMIN only. Changes apply to future referral snapshots. SUPER_ADMIN only. Changes apply to future referral snapshots and do not recalculate already-earned rewards.
Request payload(s):
payload


```
{
  "referrerRewardAmount": 25,
  "newUserRewardAmount": 10,
  "rewardCurrency": "USD",
  "minimumTransactionAmount": 50,
  "referralExpirationValue": 30,
  "referralExpirationUnit": "DAYS",
  "allowSelfReferrals": false,
  "qualificationRule": "FIRST_SUCCESSFUL_PURCHASE"
}
```


Response body:


```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```


`PATCH` `/api/v1/referral-settings/status`
Summary: Update referral program status

Allowed role/access: SUPER_ADMIN
Notes: Access: SUPER_ADMIN. SUPER_ADMIN only. isActive=true activates the referral program; isActive=false deactivates it. SUPER_ADMIN only. isActive=true activates the referral program; isActive=false deactivates it. This status endpoint is restricted to Super Admin, and earned rewards remain redeemable.
Request payload(s):
activate

```
{
  "isActive": true,
  "reason": "Seasonal referral campaign enabled."
}
```

deactivate

```
{
  "isActive": false,
  "reason": "Referral campaign paused for budget review."
}
```

Response body:

```
{
  "success": true,
  "data": {
    "isActive": true,
    "updatedAt": "2026-05-25T10:00:00.000Z"
  },
  "message": "Referral program status updated successfully."
}
```

GET `/api/v1/referral-settings/stats`
Summary: Fetch referral stats
Allowed role/access: SUPER_ADMIN or ADMIN with referralSettings.read
Notes: Access: SUPER_ADMIN or ADMIN with referralSettings.read. SUPER_ADMIN or ADMIN with referralSettings.read permission.
Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/referral-settings/audit-logs`
Summary: List referral settings audit logs
Allowed role/access: SUPER_ADMIN
Notes: Access: SUPER_ADMIN. SUPER_ADMIN only. Referral settings audit logs. SUPER_ADMIN only.
Parameters:

- `page` (query, optional, number)
- `limit` (query, optional, number)
- `fromDate` (query, optional, string)
- `toDate` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

02 Admin - Refund Policy Settings

GET `/api/v1/admin/refund-policy-settings`
Summary: Fetch refund policy settings
Allowed role/access: SUPER_ADMIN or ADMIN with refundPolicies.read
Notes: Access: SUPER_ADMIN or ADMIN with refundPolicies.read. SUPER_ADMIN or ADMIN with refundPolicies.read permission. Settings feed refund eligibility, cancellation deduction tiers, dispute, and provider refund workflows. These global settings are used by refund eligibility, cancellation deduction tiers, dispute decisions, and provider refund workflows.
Response body:

```
{
  "success": true,
  "data": {
    "allowRefund": true,
    "noteText": "Refunds are processed according to cancellation policy.",
    "refundWindowDays": 30,
    "autoRefundThresholdAmount": 50,
    "cancellationTiers": [
      {
        "id": "tier_id",
        "daysBeforeCheckIn": 5,
        "deductionPercent": 10,
        "label": "Early"
      }
    ],
    "lastUpdatedAt": "2026-05-25T10:00:00.000Z",
    "lastUpdatedBy": {
      "id": "admin_id",
      "name": "Super Admin"
    }
  },
  "message": "Refund policy settings fetched successfully."
}
```

PATCH `/api/v1/admin/refund-policy-settings`
Summary: Update refund policy settings
Allowed role/access: SUPER_ADMIN
Notes: Access: SUPER_ADMIN. SUPER_ADMIN only. Updates global refund policy settings used by customer refund request eligibility, provider refund handling, cancellation deduction tiers, and admin/provider dispute workflows. SUPER_ADMIN only. Updates global refund policy settings used by customer refund request eligibility, provider refund handling, cancellation deduction tiers, and admin/provider dispute workflows.
Request payload(s):
enableRefunds

```
{
  "allowRefund": true,
  "noteText": "Refunds are processed according to cancellation policy.",
  "refundWindowDays": 30,
  "autoRefundThresholdAmount": 50,
  "cancellationTiers": [
```

```
    {
      "daysBeforeCheckIn": 5,
      "deductionPercent": 10,
      "label": "Early"
    }
  ]
}
```

disableRefunds

```
{
  "allowRefund": false,
  "noteText": "Refunds are temporarily disabled."
}
```

updateCancellationTiers

```
{
  "cancellationTiers": [
    {
      "daysBeforeCheckIn": 5,
      "deductionPercent": 10,
      "label": "Early"
    },
    {
      "daysBeforeCheckIn": 2,
      "deductionPercent": 25,
      "label": "Late"
    }
  ]
}
```

Response body:

```
{
  "success": true,
  "data": {
    "allowRefund": true,
    "noteText": "Refunds are processed according to cancellation policy.",
    "refundWindowDays": 30,
    "autoRefundThresholdAmount": 50,
    "cancellationTiers": [
      {
        "id": "tier_id",
        "daysBeforeCheckIn": 5,
        "deductionPercent": 10,
        "label": "Early"
      }
    ],
    "lastUpdatedAt": "2026-05-25T10:00:00.000Z",
    "lastUpdatedBy": {
      "id": "admin_id",
      "name": "Super Admin"
    }
  },
  "message": "Refund policy settings updated successfully."
}
```

GET `/api/v1/admin/refund-policy-settings/logs`
Summary: List refund policy audit logs

Allowed role/access: SUPER_ADMIN
Notes: Access: SUPER_ADMIN. SUPER_ADMIN only. Refund policy settings audit logs. SUPER_ADMIN only. Returns compliance logs for REFUND_POLICY_SETTINGS_UPDATED changes, including before/after policy JSON.
Parameters:

- `page` (query, optional, number)
- `limit` (query, optional, number)
- `fromDate` (query, optional, string)
- `toDate` (query, optional, string)

Response body:


```
{
  "success": true,
  "data": [
    {
      "id": "audit_log_id",
      "action": "REFUND_POLICY_SETTINGS_UPDATED",
      "actor": {
        "id": "admin_id",
        "name": "Alex Rivera"
      },
      "before": {
        "allowRefund": true,
        "noteText": "Refunds are processed according to cancellation policy.",
        "refundWindowDays": 30,
        "autoRefundThresholdAmount": 50,
        "cancellationTiers": []
      },
      "after": {
        "allowRefund": false,
        "noteText": "Refunds are temporarily disabled.",
        "refundWindowDays": 30,
        "autoRefundThresholdAmount": 50,
        "cancellationTiers": []
      },
      "createdAt": "2026-05-14T10:00:00.000Z"
    }
  ],
  "meta": {
    "page": 1,
    "limit": 10,
    "total": 1,
    "totalPages": 1
  },
  "message": "Refund policy audit logs fetched successfully."
}
```


02 Admin - Media Upload Policy

GET `/api/v1/media-upload-policy`
Summary: Fetch global media upload policy
Allowed role/access: SUPER_ADMIN or ADMIN with mediaPolicy.read
Notes: Access: SUPER_ADMIN or ADMIN with mediaPolicy.read. SUPER_ADMIN or ADMIN with mediaPolicy.read permission. uploads/presigned-url enforces this policy before issuing upload URLs.
Response body:


```
{
  "success": true,
  "data": {
```

```
    "allowedFileTypes": {
      "jpeg": true,
      "jpg": true,
      "png": true,
      "gif": false,
      "mp4": true,
      "mov": true,
      "mp3": true,
      "wav": false,
      "svg": false
    },
    "maxImageSizeMb": 10,
    "maxVideoSizeMb": 500,
    "maxAudioSizeMb": 50,
    "scanUploads": true,
    "blockSvgUploads": true,
    "updatedAt": "2026-05-09T10:00:00.000Z",
    "updatedBy": {
      "id": "admin_id",
      "name": "Alex Rivera"
    }
  },
  "message": "Media upload policy fetched successfully."
}
```


PATCH `/api/v1/media-upload-policy`

Summary: Update global media upload policy

Allowed role/access: SUPER_ADMIN

Notes: Access: SUPER_ADMIN. SUPER_ADMIN only. Updates global media upload policy. SUPER_ADMIN only. Does not expose AWS secrets or bucket credentials.

Request payload(s):

payload


```
{
  "allowedFileTypes": {
    "jpeg": true,
    "jpg": true,
    "png": true,
    "gif": false,
    "mp4": true,
    "mov": true,
    "mp3": true,
    "wav": false,
    "svg": false
  },
  "maxImageSizeMb": 10,
  "maxVideoSizeMb": 500,
  "maxAudioSizeMb": 50,
  "scanUploads": true,
  "blockSvgUploads": true
}
```


Response body:


```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/media-upload-policy/audit-logs`
Summary: List media upload policy audit logs
Allowed role/access: SUPER_ADMIN
Notes: Access: SUPER_ADMIN. SUPER_ADMIN only. Media upload policy audit logs. SUPER_ADMIN only.
Parameters:

- `page` (query, optional, number)
- `limit` (query, optional, number)
- `fromDate` (query, optional, string)
- `toDate` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

02 Admin - System Settings

GET `/api/v1/admin/system-settings`
Summary: Fetch system settings
Allowed role/access: SUPER_ADMIN or ADMIN with systemSettings.read
Notes: Access: SUPER_ADMIN or ADMIN with systemSettings.read. SUPER_ADMIN or ADMIN with systemSettings.read permission. SMTP secrets are never returned. SMTP secrets are never returned.
Response body:

```
{
  "success": true,
  "data": {
    "platformInfo": {
      "applicationName": "Gift App",
      "supportEmail": "support@giftapp.com",
      "platformLogoUrl": "https://cdn.example.com/logo.png"
    },
    "security": {
      "sessionTimeoutMinutes": 30,
      "adminMfaRequired": true,
      "passwordPolicy": {
        "minLength": 8,
        "requireUppercase": true,
        "requireLowercase": true,
        "requireNumber": true,
        "requireSymbol": true
      }
    },
    "payments": {
      "defaultCurrency": "USD",
      "transactionFeePercent": 2.5
    },
    "notifications": {
      "pushNotificationsEnabled": true,
      "emailNotificationsEnabled": true,
      "smtpConfigured": true
    }
  },
```

```
  "message": "System settings fetched successfully."
}
```


PATCH `/api/v1/admin/system-settings`
Summary: Update system settings
Allowed role/access: SUPER_ADMIN or ADMIN with systemSettings.update
Notes: Access: SUPER_ADMIN or ADMIN with systemSettings.update. SUPER_ADMIN or ADMIN with systemSettings.update permission. Session timeout and payment fee changes apply to future sessions/transactions only.
Request payload(s):
payload


```
{
  "platformInfo": {
    "applicationName": "Gift App",
    "supportEmail": "support@giftapp.com",
    "platformLogoUrl": "https://cdn.example.com/logo.png"
  },
  "security": {
    "sessionTimeoutMinutes": 30,
    "adminMfaRequired": true,
    "passwordPolicy": {
      "minLength": 8,
      "requireUppercase": true,
      "requireLowercase": true,
      "requireNumber": true,
      "requireSymbol": true
    }
  },
  "payments": {
    "defaultCurrency": "USD",
    "transactionFeePercent": 2.5
  },
  "notifications": {
    "pushNotificationsEnabled": true,
    "emailNotificationsEnabled": true
  }
}
```


Response body:


```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```


POST `/api/v1/admin/system-settings/logo`
Summary: Update system logo URL/reference
Allowed role/access: SUPER_ADMIN or ADMIN with systemSettings.update
Notes: Access: SUPER_ADMIN or ADMIN with systemSettings.update. SUPER_ADMIN or ADMIN with systemSettings.update permission. Stores logo URL/reference only. Provide a completed Storage upload reference when available; only URL/reference is stored.
Request payload(s):
payload


```
{
  "platformLogoUrl": "https://cdn.example.com/logo.png",
```

```
  "uploadId": "upload_id"
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

POST `/api/v1/admin/system-settings/smtp/test`
Summary: Send SMTP test email
Allowed role/access: SUPER_ADMIN or ADMIN with systemSettings.update
Notes: Access: SUPER_ADMIN or ADMIN with systemSettings.update. SUPER_ADMIN or ADMIN with systemSettings.update permission. Does not expose SMTP secrets. Uses configured mailer and does not expose SMTP password or secrets.
Request payload(s):
payload

```
{
  "to": "admin@giftapp.com"
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/admin/system-settings/audit-logs`
Summary: List system settings audit logs
Allowed role/access: SUPER_ADMIN or ADMIN with systemSettings.read
Notes: Access: SUPER_ADMIN or ADMIN with systemSettings.read. SUPER_ADMIN or ADMIN with systemSettings.read permission.
Parameters:

- `page` (query, optional, number)
- `limit` (query, optional, number)
- `fromDate` (query, optional, string)
- `toDate` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

02 Admin - Guest Access Settings

GET `/api/v1/admin/guest-access-settings`
Summary: Fetch guest access settings
Allowed role/access: SUPER_ADMIN or ADMIN with guestAccessSettings.read
Notes: Access: SUPER_ADMIN or ADMIN with guestAccessSettings.read. SUPER_ADMIN or ADMIN with guestAccessSettings.read permission.
Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

PATCH `/api/v1/admin/guest-access-settings`
Summary: Update guest access settings
Allowed role/access: SUPER_ADMIN or ADMIN with guestAccessSettings.update
Notes: Access: SUPER_ADMIN or ADMIN with guestAccessSettings.update. SUPER_ADMIN or ADMIN with guestAccessSettings.update permission. Creates audit logs.
Request payload(s):
payload

```
{
  "guestAccessEnabled": true,
  "allowMarketplaceBrowsing": true,
  "allowMarketplaceHome": true,
  "allowGiftDetails": true,
  "allowDiscountedGifts": true,
  "allowFilterOptions": true,
  "allowProviderPreview": true,
  "allowWishlist": true,
  "allowCart": true,
  "allowCheckout": true,
  "sessionTtlMinutes": 1.0,
  "maxRequestsPerMinute": 1.0,
  "showExactStockToGuests": true,
  "showSkuToGuests": true
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/admin/guest-access-settings/audit-logs`
Summary: List guest access settings audit logs
Allowed role/access: SUPER_ADMIN or ADMIN with guestAccessSettings.read
Notes: Access: SUPER_ADMIN or ADMIN with guestAccessSettings.read. SUPER_ADMIN or ADMIN with guestAccessSettings.read permission.
Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

02 Admin - System Logs & Audit Trail

GET `/api/v1/audit-logs/stats`
Summary: Fetch audit log stats
Allowed role/access: SUPER_ADMIN
Notes: Access: SUPER_ADMIN. SUPER_ADMIN only. System log summary cards are restricted to Super Admin. SUPER_ADMIN only. Returns system log summary cards and security/system metrics.
Parameters:

- `fromDate` (query, optional, string)
- `toDate` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": {
    "criticalAlerts24h": 12,
    "dailyAverageActions": 4200,
    "uptimeStatus": 99.98,
    "totalLogs": 1240,
    "successCount": 1100,
    "failedCount": 140
  },
  "message": "Audit log stats fetched successfully."
}
```

GET `/api/v1/audit-logs/action-types`
Summary: Fetch audit log action types
Allowed role/access: SUPER_ADMIN
Notes: Access: SUPER_ADMIN. SUPER_ADMIN only. Audit log action filter options are restricted to Super Admin. SUPER_ADMIN only. Returns action type dropdown options for System Logs filters.
Response body:

```
{
  "success": true,
  "data": [
    {
      "key": "PROVIDER_APPROVED",
      "label": "Provider Approved"
    },
    {
      "key": "FAILED_LOGIN_ATTEMPT",
      "label": "Failed Login Attempt"
    }
  ],
  "message": "Audit log action types fetched successfully."
}
```

GET `/api/v1/audit-logs/users`
Summary: Fetch audit log user selector options
Allowed role/access: SUPER_ADMIN
Notes: Access: SUPER_ADMIN. SUPER_ADMIN only. Audit log actor selector options are restricted to Super Admin. SUPER_ADMIN only. Returns actor/user selector options for System Logs filters.
Parameters:

- `search` (query, optional, string)
- `role` (query, optional, string)
- `limit` (query, optional, number)

Response body:


```
{
  "success": true,
  "data": [
    {
      "id": "system",
      "name": "System",
      "email": null,
      "role": "SYSTEM",
      "label": "System Automated Action"
    },
    {
      "id": "admin_id",
      "name": "Sarah Chen",
      "email": "sarah@example.com",
      "role": "ADMIN",
      "label": "Sarah Chen — Compliance Officer"
    }
  ],
  "message": "Audit log users fetched successfully."
}
```


GET `/api/v1/audit-logs/export`
Summary: Export audit logs CSV
Allowed role/access: SUPER_ADMIN
Notes: Access: SUPER_ADMIN. SUPER_ADMIN only. Audit log export is restricted to Super Admin. SUPER_ADMIN only. Exports sanitized audit log records using the same filters as the list API.
Parameters:

- `page` (query, optional, number)
- `limit` (query, optional, number)
- `actorId` (query, optional, string)
- `userId` (query, optional, string)
- `targetId` (query, optional, string)
- `action` (query, optional, string)
- `actionType` (query, optional, string)
- `targetType` (query, optional, string)
- `module` (query, optional, string)
- `status` (query, optional, string)
- `environment` (query, optional, string)
- `sourceIp` (query, optional, string)
- `from` (query, optional, string)
- `to` (query, optional, string)
- `fromDate` (query, optional, string)
- `toDate` (query, optional, string)
- `sortBy` (query, optional, string)
- `sortOrder` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```


GET `/api/v1/audit-logs`
Summary: List audit logs
Allowed role/access: SUPER_ADMIN
Notes: Access: SUPER_ADMIN. SUPER_ADMIN only. Audit logs are restricted to Super Admin. SUPER_ADMIN only. Supports filtering by action type, actor, status, date range, module, and source IP.
Parameters:

- `page` (query, optional, number)
- `limit` (query, optional, number)
- `actorId` (query, optional, string)
- `userId` (query, optional, string)
- `targetId` (query, optional, string)
- `action` (query, optional, string)
- `actionType` (query, optional, string)
- `targetType` (query, optional, string)
- `module` (query, optional, string)
- `status` (query, optional, string)
- `environment` (query, optional, string)
- `sourceIp` (query, optional, string)
- `from` (query, optional, string)
- `to` (query, optional, string)
- `fromDate` (query, optional, string)
- `toDate` (query, optional, string)
- `sortBy` (query, optional, string)
- `sortOrder` (query, optional, string)

Response body:


```
{
  "success": true,
  "data": [
    {
      "id": "audit_log_id",
      "eventId": "EV-90210",
      "logReference": "789042",
      "timestamp": "2023-10-27T14:32:01.000Z",
      "actor": {
        "id": "admin_id",
        "name": "Sarah Chen",
        "role": "Compliance Officer",
        "avatarInitials": "SC"
      },
      "action": "PROVIDER_APPROVED",
      "actionLabel": "Provider Approved",
      "module": "Provider Management",
      "sourceIp": "192.168.1.45",
      "environment": "Production-Cluster-A",
      "status": "SUCCESS",
      "target": {
        "id": "provider_id",
        "type": "PROVIDER"
      },
      "createdAt": "2023-10-27T14:32:01.000Z"
    }
  ],
```

```
  "meta": {
    "page": 1,
    "limit": 10,
    "total": 1,
    "totalPages": 1
  },
  "message": "Audit logs fetched successfully"
}
```


GET `/api/v1/audit-logs/{id}`
Summary: Fetch audit log detail
Allowed role/access: SUPER_ADMIN
Notes: Access: SUPER_ADMIN. SUPER_ADMIN only. Audit log details are restricted to Super Admin. SUPER_ADMIN only. Returns sanitized raw JSON payloads, system response preview, and technical metadata.
Parameters:

- `id` (path, required, string)

Response body:


```
{
  "success": true,
  "data": {
    "id": "audit_log_id",
    "logReference": "789042",
    "eventId": "EV-90210",
    "timestamp": "2023-10-27T14:32:01.000Z",
    "status": "SUCCESS",
    "category": "AUDIT_TRAIL",
    "actor": {
      "id": "admin_id",
      "username": "super_admin_fintech",
      "name": "Super Admin",
      "role": "SUPER_ADMIN"
    },
    "actionType": "TRANSACTION_AUTHORIZATION",
    "module": "Transactions",
    "sourceIp": "192.168.1.104",
    "environment": "Production-Cluster-A",
    "target": {
      "id": "transaction_id",
      "type": "TRANSACTION"
    },
    "requestPayload": {
      "authorization": "[REDACTED]",
      "request_id": "req_550e8400"
    },
    "systemResponse": {
      "statusCode": 200,
      "durationMs": 142,
      "message": "Transaction authorized successfully."
    },
    "createdAt": "2023-10-27T14:32:01.000Z"
  },
  "message": "Audit log details fetched successfully"
}
```

02 Admin - Dispute Manager

`GET` `/api/v1/admin/disputes/stats`
Summary: Fetch dispute dashboard stats
Allowed role/access: SUPER_ADMIN or ADMIN with disputes.read
Notes: Access: SUPER_ADMIN or ADMIN with disputes.read. SUPER_ADMIN or ADMIN with disputes.read permission. Supports TODAY, LAST_7_DAYS, LAST_30_DAYS, and CUSTOM ranges.
Parameters:

- `range` (query, optional, string)
- `fromDate` (query, optional, string)
- `toDate` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": {
    "openCases": 12,
    "openCasesDelta": 2,
    "awaitingAction": 5,
    "escalated": 2,
    "resolvedThisWeek": 24,
    "resolvedDeltaPercent": 12,
    "currency": "PKR"
  },
  "message": "Dispute stats fetched successfully."
}
```

`GET` `/api/v1/admin/disputes/export`
Summary: Export dispute cases
Allowed role/access: SUPER_ADMIN or ADMIN with disputes.export
Notes: Access: SUPER_ADMIN or ADMIN with disputes.export. SUPER_ADMIN or ADMIN with disputes.export permission. Sensitive card/payment secrets are never exported.
Parameters:

- `status` (query, optional, string)
- `priority` (query, optional, string)
- `fromDate` (query, optional, string)
- `toDate` (query, optional, string)
- `format` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

`GET` `/api/v1/admin/disputes`
Summary: List dispute queue
Allowed role/access: SUPER_ADMIN or ADMIN with disputes.read
Notes: Access: SUPER_ADMIN or ADMIN with disputes.read. SUPER_ADMIN or ADMIN with disputes.read permission. Used by Dispute & Refund Cases queue with filters and sorting.
Parameters:

- `range` (query, optional, string)

- `fromDate` (query, optional, string)
- `toDate` (query, optional, string)
- `page` (query, optional, number)
- `limit` (query, optional, number)
- `search` (query, optional, string)
- `status` (query, optional, string)
- `priority` (query, optional, string)
- `sortBy` (query, optional, string)
- `sortOrder` (query, optional, string)

Response body:


```
{
  "success": true,
  "data": [
    {
      "id": "dispute_id",
      "caseId": "DIS-9842",
      "customer": {
        "id": "customer_id",
        "name": "Eleanor Pena",
        "email": "eleanor.p@gmail.com"
      },
      "transactionId": "TRX-78229410",
      "orderId": "order_id",
      "orderNumber": "ORD-88421",
      "amount": 492,
      "currency": "PKR",
      "priority": "HIGH",
      "status": "ESCALATED",
      "daysOpen": 8,
      "reason": "PRODUCT_NOT_RECEIVED",
      "createdAt": "2026-04-01T10:00:00.000Z"
    }
  ],
  "meta": {
    "page": 1,
    "limit": 10,
    "total": 1,
    "totalPages": 1
  },
  "message": "Disputes fetched successfully."
}
```


GET `/api/v1/admin/disputes/{id}/timeline`
Summary: Fetch dispute timeline
Allowed role/access: SUPER_ADMIN or ADMIN with disputes.read
Notes: Access: SUPER_ADMIN or ADMIN with disputes.read. SUPER_ADMIN or ADMIN with disputes.read permission. SUPER_ADMIN or ADMIN with disputes.timeline.read. Returns timeline preview events in chronological order.
Parameters:

- `id` (path, required, string)

Response body:


```
{
  "success": true,
  "data": [
    {
      "id": "timeline_id",
      "type": "DISPUTE_CREATED",
      "title": "Dispute Created",
```

```
      "description": "Customer created dispute for product not received.",
      "actor": {
        "type": "CUSTOMER",
        "name": "Jane Doe"
      },
      "createdAt": "2026-04-05T09:15:00.000Z"
    }
  ],
  "message": "Dispute timeline fetched successfully."
}
```


GET `/api/v1/admin/disputes/{id}/internal-data`
Summary: Fetch internal transaction data
Allowed role/access: SUPER_ADMIN or ADMIN with disputes.read
Notes: Access: SUPER_ADMIN or ADMIN with disputes.read. SUPER_ADMIN or ADMIN with disputes.read permission. Includes payment status, refund eligibility, auth code, and order/provider transaction history without card secrets.
Parameters:

- `id` (path, required, string)

Response body:


```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```


GET `/api/v1/admin/disputes/{id}/notes`
Summary: Fetch internal dispute notes
Allowed role/access: SUPER_ADMIN or ADMIN with disputes.read
Notes: Access: SUPER_ADMIN or ADMIN with disputes.read. SUPER_ADMIN or ADMIN with disputes.read permission. Returns internal notes only.
Parameters:

- `id` (path, required, string)

Response body:


```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```


POST `/api/v1/admin/disputes/{id}/notes`
Summary: Add internal dispute note
Allowed role/access: SUPER_ADMIN or ADMIN with disputes.notes.create
Notes: Access: SUPER_ADMIN or ADMIN with disputes.notes.create. SUPER_ADMIN or ADMIN with disputes.notes.create permission. Notes are internal-only and create audit/timeline entries.
Parameters:

- `id` (path, required, string)

Request payload(s):
internal

```
{
  "note": "Customer tracking shows pending status for over 14 days. Investigating
merchant log.",
  "visibility": "INTERNAL"
}
```


Response body:


```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```


GET `/api/v1/admin/disputes/{id}`
Summary: Fetch dispute details and evidence review summary
Allowed role/access: SUPER_ADMIN or ADMIN with disputes.read
Notes: Access: SUPER_ADMIN or ADMIN with disputes.read. SUPER_ADMIN or ADMIN with disputes.read permission. SLA remaining text is computed from slaDeadlineAt; approaching deadline is true within 24 hours.
Parameters:

- `id` (path, required, string)

Response body:


```
{
  "success": true,
  "data": {
    "id": "dispute_id",
    "caseId": "DSP-1024",
    "status": "IN_REVIEW",
    "priority": "HIGH",
    "reason": "PRODUCT_NOT_RECEIVED",
    "amount": 129.99,
    "currency": "PKR",
    "sla": {
      "deadlineAt": "2026-04-12T10:00:00.000Z",
      "remainingText": "22h 14m remaining",
      "isApproachingDeadline": true
    },
    "customer": {
      "id": "customer_id",
      "name": "Jane Doe",
      "email": "jane.doe@example.com"
    },
    "transaction": {
      "id": "transaction_id",
      "transactionId": "TXN-789012",
      "paymentStatus": "CAPTURED",
      "processorAuthCode": "AUTH-9921-X",
      "amount": 129.99,
      "currency": "PKR"
    },
    "refund": {
      "eligible": true,
      "eligibleReason": "Within refund window",
      "maxRefundAmount": 129.99
    },
    "claimDetails": "Item never arrived. Tracking shows delivered but not at my
address.",
```

```
    "createdAt": "2026-04-05T09:15:00.000Z",
    "lastUpdatedAt": "2026-04-08T11:45:00.000Z"
  },
  "message": "Dispute details fetched successfully."
}
```


02 Admin - Dispute Evidence

GET `/api/v1/admin/disputes/{id}/evidence`
Summary: Fetch dispute evidence
Allowed role/access: SUPER_ADMIN or ADMIN with disputes.read
Notes: Access: SUPER_ADMIN or ADMIN with disputes.read. SUPER_ADMIN or ADMIN with disputes.read permission. SUPER_ADMIN or ADMIN with disputes.evidence.read. Returns only evidence rows linked to this dispute, from customer/admin uploads in dispute-evidence folder.
Parameters:

- `id` (path, required, string)

Response body:


```
{
  "success": true,
  "data": [
    {
      "id": "evidence_id",
      "fileName": "Order confirmation.pdf",
      "fileUrl": "https://cdn.yourdomain.com/dispute-evidence/order-confirmation.pdf",
      "contentType": "application/pdf",
      "uploadedBy": "CUSTOMER",
      "createdAt": "2026-04-05T09:30:00.000Z"
    }
  ],
  "message": "Dispute evidence fetched successfully."
}
```


02 Admin - Dispute Linkage

GET `/api/v1/admin/disputes/{id}/linkage`
Summary: Fetch current dispute transaction linkage state
Allowed role/access: SUPER_ADMIN or ADMIN with disputes.read
Notes: Access: SUPER_ADMIN or ADMIN with disputes.read. SUPER_ADMIN or ADMIN with disputes.read permission. Shows dispute summary, linked transaction, and current refund selection without card secrets.
Parameters:

- `id` (path, required, string)

Response body:


```
{
  "success": true,
  "data": {
    "dispute": {
      "id": "dispute_id",
      "caseId": "DSP-1024",
      "customer": {
        "id": "customer_id",
```

```
        "name": "Jane Doe"
      },
      "disputeAmount": 129.99,
      "currency": "PKR",
      "claimDetails": "Item never arrived. Tracking shows delivered but not at my
address.",
      "status": "IN_REVIEW"
    },
    "linkedTransaction": {
      "id": "transaction_id",
      "transactionId": "TXN-789012",
      "orderId": "order_id",
      "orderDate": "2026-04-01T00:00:00.000Z",
      "paymentMethod": "VISA **** 1234",
      "amount": 129.99,
      "currency": "PKR",
      "status": "SETTLED",
      "refundEligible": true,
      "eligibilityText": "Eligible within 30-day window"
    },
    "refundSelection": {
      "type": "FULL",
      "amount": 129.99,
      "recommended": true
    }
  },
  "message": "Dispute linkage fetched successfully."
}
```


GET `/api/v1/admin/disputes/{id}/transaction-search`
Summary: Search original transaction for a dispute
Allowed role/access: SUPER_ADMIN or ADMIN with disputes.read
Notes: Access: SUPER_ADMIN or ADMIN with disputes.read. SUPER_ADMIN or ADMIN with disputes.read permission. Search is scoped to the dispute customer where possible and never exposes card secrets.
Parameters:

- `id` (path, required, string)
- `query` (query, optional, string)
- `recentOnly` (query, optional, boolean)
- `limit` (query, optional, number)

Response body:


```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```


POST `/api/v1/admin/disputes/{id}/link-transaction`
Summary: Confirm dispute transaction linkage
Allowed role/access: SUPER_ADMIN or ADMIN with disputes.linkTransaction
Notes: Access: SUPER_ADMIN or ADMIN with disputes.linkTransaction. SUPER_ADMIN or ADMIN with disputes.linkTransaction permission. Stores linked transaction/payment/order and refund selection, creates timeline/audit records, and does not process refunds.
Parameters:

- `id` (path, required, string)

Request payload(s):

full

```
{
  "transactionId": "transaction_id",
  "refundType": "FULL",
  "refundAmount": 129.99,
  "confirmCorrectTransaction": true
}
```

none

```
{
  "transactionId": "transaction_id",
  "refundType": "NONE",
  "confirmCorrectTransaction": true
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

POST `/api/v1/admin/disputes/{id}/refund-preview`
Summary: Preview dispute refund selection
Allowed role/access: SUPER_ADMIN or ADMIN with disputes.refund.evaluate
Notes: Access: SUPER_ADMIN or ADMIN with disputes.refund.evaluate. SUPER_ADMIN or ADMIN with disputes.refund.evaluate permission. Validates requested refunds against paid amount, prior refunds, and refund window without processing a refund.
Parameters:

- `id` (path, required, string)

Request payload(s):
full

```
{
  "transactionId": "transaction_id",
  "refundType": "FULL",
  "refundAmount": 129.99
}
```

partial

```
{
  "transactionId": "transaction_id",
  "refundType": "PARTIAL",
  "refundAmount": 50
}
```

none

```
{
  "transactionId": "transaction_id",
  "refundType": "NONE"
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

02 Admin - Dispute Decisions

GET `/api/v1/admin/disputes/{id}/decision-summary`
Summary: Fetch dispute decision summary
Allowed role/access: SUPER_ADMIN or ADMIN with disputes.decide
Notes: Access: SUPER_ADMIN or ADMIN with disputes.decide. SUPER_ADMIN or ADMIN with disputes.decide permission. Summarizes customer, transaction, refund eligibility, and case history before final decision.
Parameters:

- `id` (path, required, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

POST `/api/v1/admin/disputes/{id}/decision`
Summary: Submit final dispute decision
Allowed role/access: SUPER_ADMIN or ADMIN with disputes.decide plus action-specific permission (approve/reject/escalate)
Notes: Access: SUPER_ADMIN or ADMIN with disputes.decide plus action-specific permission (approve/reject/escalate). SUPER_ADMIN or ADMIN with disputes.decide and the action-specific permission as applicable. SUPER_ADMIN or ADMIN with dispute decision permissions. APPROVE validates linked transaction/refund selection and creates a refund record; REJECT never creates a refund; ESCALATE assigns supervisor and resets SLA. Stripe refunds are represented by refund tracking records and no card/Stripe secrets are exposed.
Parameters:

- `id` (path, required, string)

Request payload(s):
approve

```
{
  "decision": "APPROVE",
  "comment": "Customer evidence validates missing delivery.",
  "notifyCustomer": true
}
```

reject

```
{
  "decision": "REJECT",
  "reason": "INSUFFICIENT_EVIDENCE",
  "comment": "Tracking evidence confirms delivery.",
```

```
  "notifyCustomer": true
}
```

escalate

```
{
  "decision": "ESCALATE",
  "assignedToId": "admin_supervisor_id",
  "escalationReason": "Policy ambiguity requires supervisor intervention.",
  "notifyCustomer": false
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/admin/disputes/{id}/confirmation`
Summary: Fetch decision confirmation
Allowed role/access: SUPER_ADMIN or ADMIN with disputes.decide
Notes: Access: SUPER_ADMIN or ADMIN with disputes.decide. SUPER_ADMIN or ADMIN with disputes.decide permission. Returns refund, processor, protocol, and customer notification confirmation.
Parameters:

- `id` (path, required, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

02 Admin - Dispute Tracking

GET `/api/v1/admin/disputes/{id}/tracking-log/export`
Summary: Export full dispute tracking log
Allowed role/access: SUPER_ADMIN or ADMIN with disputes.tracking.export
Notes: Access: SUPER_ADMIN or ADMIN with disputes.tracking.export. SUPER_ADMIN or ADMIN with disputes.tracking.export permission. Includes timeline, decision, refund, notifications, and internal notes without card or Stripe secrets.
Parameters:

- `id` (path, required, string)
- `format` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
```

```
  "message": "Request completed successfully."
}
```


GET /api/v1/admin/disputes/{id}/tracking-log
Summary: Fetch full dispute tracking log
Allowed role/access: SUPER_ADMIN or ADMIN with disputes.tracking.read
Notes: Access: SUPER_ADMIN or ADMIN with disputes.tracking.read. SUPER_ADMIN or ADMIN with disputes.tracking.read permission. Returns secure audit timeline, customer notifications, and internal notes.
Parameters:

- `id` (path, required, string)

Response body:


```
{
  "success": true,
  "data": {
    "caseId": "DSP-1024",
    "customer": {
      "name": "Jane Doe"
    },
    "finalStatus": "APPROVED",
    "lastUpdatedAt": "2026-04-08T11:45:00.000Z",
    "secureAuditActive": true,
    "timeline": [
      {
        "id": "timeline_id",
        "type": "REFUND_PROCESSED",
        "title": "System Automated Action",
        "description": "Refund processed successfully.",
        "amount": 129.99,
        "refundId": "RF-45678",
        "createdAt": "2026-04-08T11:45:00.000Z"
      }
    ],
    "customerNotifications": [
      {
        "type": "REFUND_CONFIRMATION_EMAIL",
        "status": "DELIVERED",
        "deliveredAt": "2026-04-08T11:46:00.000Z"
      }
    ],
    "internalNotes": [
      {
        "id": "note_id",
        "author": "Alex Morgan",
        "note": "Customer tracking shows pending status for over 14 days.",
        "createdAt": "2026-04-08T10:00:00.000Z"
      }
    ]
  },
  "message": "Case tracking log fetched successfully."
}
```


POST /api/v1/admin/disputes/{id}/follow-up-notes
Summary: Add dispute follow-up note
Allowed role/access: SUPER_ADMIN or ADMIN with disputes.notes.create
Notes: Access: SUPER_ADMIN or ADMIN with disputes.notes.create. SUPER_ADMIN or ADMIN with disputes.notes.create permission. Adds internal note, tracking timeline entry, and notifies assigned admin when present.

Parameters:

- `id` (path, required, string)

Request payload(s):
internal

```
{
  "note": "Followed up with provider for missing dispatch log.",
  "visibility": "INTERNAL"
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

02 Admin - Provider Dispute Manager

GET `/api/v1/admin/provider-disputes/stats`
Summary: Fetch provider dispute dashboard stats
Allowed role/access: SUPER_ADMIN or ADMIN with providerDisputes.read
Notes: Access: SUPER_ADMIN or ADMIN with providerDisputes.read. SUPER_ADMIN or ADMIN with providerDisputes.read permission. Reuses Provider Orders, Payments, Notifications, Audit Logs, and dispute patterns.
Parameters:

- `range` (query, optional, string)
- `fromDate` (query, optional, string)
- `toDate` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": {
    "criticalOpenCases": 8,
    "evidencePhase": 3,
    "underReview": 4,
    "escalations": 1,
    "resolvedThisWeek": 14,
    "averageClosureTimeDays": 4.2,
    "topConflictSource": {
      "providerName": "Acme Corp",
      "category": "Quality Disputes",
      "percentOfTotal": 65
    },
    "systemHealth": {
      "status": "STABLE",
      "message": "All nodes stable",
      "apiLatencyMs": 42
    }
  },
  "message": "Provider dispute stats fetched successfully."
}
```

GET `/api/v1/admin/provider-disputes/export`
Summary: Export provider dispute queue
Allowed role/access: SUPER_ADMIN or ADMIN with providerDisputes.export
Notes: Access: SUPER_ADMIN or ADMIN with providerDisputes.export. SUPER_ADMIN or ADMIN with providerDisputes.export permission. Does not expose card secrets or unrelated uploads.
Parameters:

- `status` (query, optional, string)
- `severity` (query, optional, string)
- `category` (query, optional, string)
- `fromDate` (query, optional, string)
- `toDate` (query, optional, string)
- `format` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/admin/provider-disputes`
Summary: List provider dispute queue
Allowed role/access: SUPER_ADMIN or ADMIN with providerDisputes.read
Notes: Access: SUPER_ADMIN or ADMIN with providerDisputes.read. SUPER_ADMIN or ADMIN with providerDisputes.read permission. Used by Provider Dispute Case Queue with filters and sorting.
Parameters:

- `range` (query, optional, string)
- `fromDate` (query, optional, string)
- `toDate` (query, optional, string)
- `page` (query, optional, number)
- `limit` (query, optional, number)
- `search` (query, optional, string)
- `providerId` (query, optional, string)
- `category` (query, optional, string)
- `severity` (query, optional, string)
- `status` (query, optional, string)
- `sortBy` (query, optional, string)
- `sortOrder` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": [
    {
      "id": "provider_dispute_id",
      "caseId": "PRV-101",
      "provider": {
        "id": "provider_id",
        "businessName": "Acme Corp",
        "contactName": "John Smith",
        "tier": "Gold Partner"
      },
      "customer": {
        "id": "customer_id",
        "name": "Michael Chen"
      },
      "transaction": {
```

```
        "id": "transaction_id",
        "transactionId": "TXN-998",
        "status": "VERIFIED"
      },
      "category": "NON_DELIVERY",
      "amount": 650,
      "currency": "PKR",
      "status": "RULING_PENDING",
      "priority": "HIGH",
      "riskAssessment": "HIGH",
      "daysOpen": 5,
      "createdAt": "2026-04-05T10:00:00.000Z"
    }
  ],
  "meta": {
    "page": 1,
    "limit": 10,
    "total": 1,
    "totalPages": 1
  },
  "message": "Provider disputes fetched successfully."
}
```


GET `/api/v1/admin/provider-disputes/{id}/timeline`
Summary: Fetch provider dispute timeline
Allowed role/access: SUPER_ADMIN or ADMIN with providerDisputes.read
Notes: Access: SUPER_ADMIN or ADMIN with providerDisputes.read. SUPER_ADMIN or ADMIN with providerDisputes.read permission. Includes provider dispute creation, evidence submission, requests, and review actions.
Parameters:

- `id` (path, required, string)

Response body:


```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```


GET `/api/v1/admin/provider-disputes/{id}/notes`
Summary: Fetch provider dispute internal notes
Allowed role/access: SUPER_ADMIN or ADMIN with providerDisputes.read
Notes: Access: SUPER_ADMIN or ADMIN with providerDisputes.read. SUPER_ADMIN or ADMIN with providerDisputes.read permission. Returns internal reviewer notes only.
Parameters:

- `id` (path, required, string)

Response body:


```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

POST `/api/v1/admin/provider-disputes/{id}/notes`
Summary: Add provider dispute internal note
Allowed role/access: SUPER_ADMIN or ADMIN with providerDisputes.notes.create
Notes: Access: SUPER_ADMIN or ADMIN with providerDisputes.notes.create. SUPER_ADMIN or ADMIN with providerDisputes.notes.create permission. Creates internal note, timeline entry, and audit log.
Parameters:

- `id` (path, required, string)

Request payload(s):
internal

```
{
  "note": "Provider failed to submit required photographic proof.",
  "visibility": "INTERNAL"
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/admin/provider-disputes/{id}`
Summary: Fetch provider dispute details
Allowed role/access: SUPER_ADMIN or ADMIN with providerDisputes.read
Notes: Access: SUPER_ADMIN or ADMIN with providerDisputes.read. SUPER_ADMIN or ADMIN with providerDisputes.read permission. Reuses Provider Orders, Customer Orders, Payments, Notifications, Storage, and Audit Logs. No card/payment secrets are exposed.
Parameters:

- `id` (path, required, string)

Response body:

```
{
  "success": true,
  "data": {
    "id": "provider_dispute_id",
    "caseId": "PD-2047",
    "status": "EVIDENCE_PHASE",
    "priority": "MEDIUM",
    "category": "NON_DELIVERY",
    "reason": "Missing delivery evidence",
    "claimType": "Non-Delivery",
    "amount": 89.99,
    "currency": "PKR",
    "provider": {
      "id": "provider_id",
      "businessName": "FreshGrocer Supplies",
      "providerCode": "PRV-8923",
      "tier": "Gold Partner",
      "currentPayoutBalance": -127.5,
      "disputeCount": 4,
      "winRate": 50
    },
    "customer": {
      "id": "customer_id",
      "name": "Michael Chen",
```

```
      "email": "michael@example.com"
    },
    "order": {
      "id": "order_id",
      "orderNumber": "ORD-45678"
    },
    "transaction": {
      "id": "transaction_id",
      "transactionId": "TXN-789012",
      "grossTransaction": 89.99,
      "providerShare": 67.49,
      "platformFee": 22.5,
      "refundEligible": true,
      "eligibilityText": "Within the standard 14-day resolution window."
    },
    "customerStatement": "I stayed home all day waiting for the delivery. I got a
notification saying it was delivered, but nothing was at my door.",
    "riskAlert": {
      "enabled": true,
      "message": "FreshGrocer Supplies has a 60% dispute rate over the last 30 days."
    },
    "createdAt": "2026-04-05T10:00:00.000Z"
  },
  "message": "Provider dispute details fetched successfully."
}
```


02 Admin - Provider Dispute Evidence

GET `/api/v1/admin/provider-disputes/{id}/evidence`
Summary: Fetch provider dispute evidence exchange
Allowed role/access: SUPER_ADMIN or ADMIN with providerDisputes.read
Notes: Access: SUPER_ADMIN or ADMIN with providerDisputes.read. SUPER_ADMIN or ADMIN with providerDisputes.read permission. Returns customer/provider evidence linked to providerDisputeId only, reusing Storage and media policy.
Parameters:

- `id` (path, required, string)

Response body:


```
{
  "success": true,
  "data": {
    "caseId": "PD-2047",
    "reviewStatus": {
      "startedBy": "A. Marcus",
      "startedAt": "2026-04-08T09:00:00.000Z",
      "isComplete": false
    },
    "customerEvidence": {
      "submittedAt": "2026-04-05T10:00:00.000Z",
      "status": "RECEIVED",
      "narrative": "The package was never delivered to my doorstep despite tracking
saying otherwise.",
      "files": [
        {
          "id": "evidence_file_id",
          "fileName": "Order confirmation screenshot.pdf",
          "fileUrl": "https://cdn.yourdomain.com/provider-dispute-evidence/order-
confirmation.pdf",
          "contentType": "application/pdf",
```

```
          "sizeText": "1.2 MB",
          "category": "PDF Document"
        }
      ]
    },
    "providerEvidence": {
      "submittedAt": "2026-04-07T10:00:00.000Z",
      "status": "RECEIVED_LATE",
      "lateText": "+2 days past deadline",
      "narrative": "Delivery was completed at 2:14 PM. GPS coordinates and driver logs
confirm arrival.",
      "files": [
        {
          "id": "evidence_file_id_4",
          "fileName": "GPS coordinates at delivery.json",
          "fileUrl": "https://cdn.yourdomain.com/provider-dispute-evidence/gps.json",
          "contentType": "application/json",
          "sizeText": "4 KB",
          "category": "Metadata File"
        }
      ]
    },
    "internalReviewerNotes": "Document your findings here. These notes are only visible
to internal staff."
  },
  "message": "Provider dispute evidence fetched successfully."
}
```


POST `/api/v1/admin/provider-disputes/{id}/evidence/request`
Summary: Request additional provider dispute evidence
Allowed role/access: SUPER_ADMIN or ADMIN with providerDisputes.evidence.request
Notes: Access: SUPER_ADMIN or ADMIN with providerDisputes.evidence.request. SUPER_ADMIN or ADMIN with providerDisputes.evidence.request permission. Creates timeline and optional notifications without changing final ruling.
Parameters:

- `id` (path, required, string)

Request payload(s):
provider


```
{
  "target": "PROVIDER",
  "message": "Please upload photographic proof of delivery and driver log.",
  "dueAt": "2026-04-09T18:00:00.000Z",
  "notifyTarget": true
}
```


Response body:


```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```


POST `/api/v1/admin/provider-disputes/{id}/evidence/mark-reviewed`
Summary: Mark provider dispute evidence review complete
Allowed role/access: SUPER_ADMIN or ADMIN with providerDisputes.update

Notes: Access: SUPER_ADMIN or ADMIN with providerDisputes.update. SUPER_ADMIN or ADMIN with providerDisputes.update permission. Marks evidence review complete, moves case to RULING_PENDING, creates timeline and audit log.
Parameters:

- `id` (path, required, string)

Request payload(s):
review

```
{
  "reviewerNotes": "Provider evidence is incomplete. GPS data was submitted late and
lacks photo confirmation.",
  "nextStep": "RULING"
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

02 Admin - Provider Dispute Rulings

GET `/api/v1/admin/provider-disputes/{id}/ruling-summary`
Summary: Fetch provider dispute ruling summary
Allowed role/access: SUPER_ADMIN or ADMIN with providerDisputes.ruling.read
Notes: Access: SUPER_ADMIN or ADMIN with providerDisputes.ruling.read. SUPER_ADMIN or ADMIN with providerDisputes.ruling.read permission. Shows ruling options, evidence summary, and financial starting point.
Parameters:

- `id` (path, required, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

POST `/api/v1/admin/provider-disputes/{id}/ruling`
Summary: Save provider dispute ruling
Allowed role/access: SUPER_ADMIN or ADMIN with providerDisputes.ruling.create
Notes: Access: SUPER_ADMIN or ADMIN with providerDisputes.ruling.create. SUPER_ADMIN or ADMIN with providerDisputes.ruling.create permission. Stores ruling and reason, but final financial execution remains gated behind final status update/attestation.
Parameters:

- `id` (path, required, string)

Request payload(s):
full

```
{
  "ruling": "CUSTOMER_WINS_FULL_REFUND",
  "rulingReason": "Provider failed to provide required proof of delivery.",
  "refundAmount": 89.99,
  "applyPenalty": true,
  "penaltyAmount": 25,
  "penaltyReason": "Repeat offense",
  "saveAsDraft": false
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

02 Admin - Provider Financial Adjustments

GET `/api/v1/admin/provider-disputes/{id}/financial-impact`
Summary: Fetch provider dispute financial impact
Allowed role/access: SUPER_ADMIN or ADMIN with providerDisputes.financial.read
Notes: Access: SUPER_ADMIN or ADMIN with providerDisputes.financial.read. SUPER_ADMIN or ADMIN with providerDisputes.financial.read permission. Server calculates provider share, fee reversal, refund, and penalty impact.
Parameters:

- `id` (path, required, string)

Response body:

```
{
  "success": true,
  "data": {
    "caseId": "PD-2047",
    "ruling": "CUSTOMER_WINS_FULL_REFUND",
    "providerAccountPreview": {
      "currentBalance": 340.5,
      "pendingPayout": 210.0,
      "newBalance": 225.51,
      "currency": "PKR"
    },
    "breakdown": [
      {
        "lineItem": "Order Total",
        "adjustment": 89.99,
        "runningTotal": 89.99
      },
      {
        "lineItem": "Customer Refund",
        "adjustment": -89.99,
        "runningTotal": 0
      },
      {
        "lineItem": "Provider Lost Earnings",
        "adjustment": -67.49,
        "runningTotal": -67.49
      },
      {
```

```
        "lineItem": "Platform Fee Reversal",
        "adjustment": -22.5,
        "runningTotal": -89.99
      },
      {
        "lineItem": "Penalty Fee",
        "adjustment": -25.0,
        "runningTotal": -114.99
      }
    ],
    "totalProviderDeduction": 114.99
  },
  "message": "Financial impact fetched successfully."
}
```


POST `/api/v1/admin/provider-disputes/{id}/payout-penalty-linkage`
Summary: Link payout and penalty adjustments
Allowed role/access: SUPER_ADMIN or ADMIN with providerDisputes.financial.link
Notes: Access: SUPER_ADMIN or ADMIN with providerDisputes.financial.link. SUPER_ADMIN or ADMIN with providerDisputes.financial.link permission. Creates provider financial adjustment ledgers; final financial execution is still deferred.
Parameters:

- `id` (path, required, string)

Request payload(s):
deduct


```
{
  "adjustmentType": "DEDUCT_FROM_NEXT_PAYOUT",
  "confirmFinancialAccuracy": true,
  "sendProviderSummary": true
}
```


Response body:


```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```


POST `/api/v1/admin/provider-disputes/{id}/final-attestation`
Summary: Complete final financial attestation
Allowed role/access: SUPER_ADMIN or ADMIN with providerDisputes.ruling.update
Notes: Access: SUPER_ADMIN or ADMIN with providerDisputes.ruling.update. SUPER_ADMIN or ADMIN with providerDisputes.ruling.update permission. Confirms line items and marks case ready for final status update.
Parameters:

- `id` (path, required, string)

Request payload(s):
confirm


```
{
  "confirmFinancialLineItems": true,
  "sendAutomatedFinancialSummary": true,
```

```
  "comment": "All financial line items confirmed as accurate."
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

02 Admin - Provider Dispute Resolution

POST `/api/v1/admin/provider-disputes/{id}/finalize`
Summary: Finalize provider dispute
Allowed role/access: SUPER_ADMIN or ADMIN with providerDisputes.resolve
Notes: Access: SUPER_ADMIN or ADMIN with providerDisputes.resolve. SUPER_ADMIN or ADMIN with providerDisputes.resolve permission. Executes final refund/deduction application, updates immutable resolution state, creates financial and communication logs, and opens provider appeal window.
Parameters:

- `id` (path, required, string)

Request payload(s):
finalize

```
{
  "notifyCustomer": true,
  "notifyProvider": true,
  "executeFinancialAdjustments": true,
  "comment": "Final ruling confirmed and financial adjustments approved."
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/admin/provider-disputes/{id}/resolution`
Summary: Fetch provider dispute resolution summary
Allowed role/access: SUPER_ADMIN or ADMIN with providerDisputes.resolve
Notes: Access: SUPER_ADMIN or ADMIN with providerDisputes.resolve. SUPER_ADMIN or ADMIN with providerDisputes.resolve permission. Returns final ruling, financial execution, notification status, refund timing, and appeal window.
Parameters:

- `id` (path, required, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
```

```
  "message": "Request completed successfully."
}
```

POST `/api/v1/admin/provider-disputes/{id}/notify-again`
Summary: Resend provider dispute notifications
Allowed role/access: SUPER_ADMIN or ADMIN with providerDisputes.notify
Notes: Access: SUPER_ADMIN or ADMIN with providerDisputes.notify. SUPER_ADMIN or ADMIN with providerDisputes.notify permission. Resends email/in-app notifications, creates communication log entries, and writes a timeline event.
Parameters:

- `id` (path, required, string)

Request payload(s):
provider

```
{
  "target": "PROVIDER",
  "channels": [
    "EMAIL",
    "IN_APP"
  ],
  "message": "Reminder: Your dispute resolution is available in the provider portal."
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

02 Admin - Provider Dispute Logs

GET `/api/v1/admin/provider-disputes/{id}/resolution-log/export`
Summary: Export provider dispute resolution log
Allowed role/access: SUPER_ADMIN or ADMIN with providerDisputes.logs.export
Notes: Access: SUPER_ADMIN or ADMIN with providerDisputes.logs.export. SUPER_ADMIN or ADMIN with providerDisputes.logs.export permission. Includes lifecycle timeline, financial audit log, communication log, ruling, and final status without Stripe/card secrets.
Parameters:

- `id` (path, required, string)
- `format` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/admin/provider-disputes/{id}/resolution-log`

Summary: Fetch provider dispute resolution log
Allowed role/access: SUPER_ADMIN or ADMIN with providerDisputes.logs.read
Notes: Access: SUPER_ADMIN or ADMIN with providerDisputes.logs.read. SUPER_ADMIN or ADMIN with providerDisputes.logs.read permission. Returns lifecycle timeline, financial audit log, communication log, and performance impact snapshot.
Parameters:

- `id` (path, required, string)

Response body:


```
{
  "success": true,
  "data": {
    "caseId": "PD-2047",
    "finalRuling": "CUSTOMER_WINS",
    "closedAt": "2026-10-24T00:00:00.000Z",
    "description": "Complete audit trail of provider dispute, financial adjustments, and
communications.",
    "lifecycleTimeline": [
      {
        "type": "PENALTY_APPLIED",
        "title": "Penalty Applied",
        "description": "System-generated penalty of 500.00 applied to provider.",
        "createdAt": "2026-10-24T14:22:00.000Z"
      }
    ],
    "financialAuditLog": [
      {
        "transactionId": "TXN_29048-REV",
        "action": "Reversal Execution",
        "amount": -1240.0,
        "currency": "PKR",
        "status": "SUCCESS"
      }
    ],
    "communicationLog": [
      {
        "type": "EMAIL",
        "title": "Resolution Decision Sent",
        "to": "claims@provider.com",
        "bodyPreview": "The evidence provided failed to meet...",
        "createdAt": "2026-10-24T10:00:00.000Z"
      }
    ],
    "providerPerformanceImpact": {
      "winRateBefore": 50,
      "winRateAfter": 40,
      "penaltyPoints": 15,
      "tierStatus": "Silver At Risk"
    }
  },
  "message": "Provider dispute resolution log fetched successfully."
}
```


03 Provider - Dashboard

GET `/api/v1/provider/dashboard`
Summary: Fetch mobile provider dashboard
Allowed role/access: Authenticated

Notes: Access: Authenticated. Authenticated JWT required. PROVIDER only. providerId is derived from JWT. Pending, rejected, inactive, or suspended providers cannot access the dashboard.
Response body:


```
{
  "success": true,
  "data": {
    "provider": {
      "id": "provider_id",
      "businessName": "Global Logistics Solutions",
      "avatarUrl": "https://cdn.yourdomain.com/provider-avatars/provider.png",
      "approvalStatus": "APPROVED",
      "status": "ACTIVE"
    },
    "operationalSummary": {
      "todayOrders": 24,
      "pendingOrders": 12,
      "activeOffers": 5,
      "totalItems": 128
    },
    "performance": {
      "range": "WEEKLY",
      "labels": [
        "Mon",
        "Tue",
        "Wed",
        "Thu",
        "Fri",
        "Sat",
        "Sun"
      ],
      "values": [
        120,
        180,
        150,
        110,
        190,
        260,
        220
      ],
      "currency": "PKR"
    },
    "recentOrders": [
      {
        "id": "provider_order_id",
        "orderNumber": "ORD-8821",
        "itemName": "Nike Air Max 270",
        "imageUrl": "https://cdn.yourdomain.com/gifts/shoe.png",
        "amount": 120,
        "currency": "PKR",
        "status": "PAID",
        "createdAgoText": "2m ago"
      }
    ]
  },
  "message": "Provider dashboard fetched successfully."
}
```

03 Provider - Earnings

GET `/api/v1/provider/earnings/summary`
Summary: Fetch own provider earnings summary
Allowed role/access: PROVIDER
Notes: Access: PROVIDER. PROVIDER only. providerId is derived from JWT; provider can access only own inventory, offers, orders, analytics, and messages. PROVIDER only. providerId is derived from JWT.
Parameters:

- `range` (query, optional, string)
- `fromDate` (query, optional, string)
- `toDate` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/provider/earnings/chart`
Summary: Fetch own provider earnings chart
Allowed role/access: PROVIDER
Notes: Access: PROVIDER. PROVIDER only. providerId is derived from JWT; provider can access only own inventory, offers, orders, analytics, and messages. PROVIDER only. Uses provider earnings ledger.
Parameters:

- `range` (query, optional, string)
- `fromDate` (query, optional, string)
- `toDate` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/provider/earnings/ledger`
Summary: List own provider earnings ledger
Allowed role/access: PROVIDER
Notes: Access: PROVIDER. PROVIDER only. providerId is derived from JWT; provider can access only own inventory, offers, orders, analytics, and messages. PROVIDER only. Does not accept providerId query/body.
Parameters:

- `range` (query, optional, string)
- `fromDate` (query, optional, string)
- `toDate` (query, optional, string)
- `page` (query, optional, number)
- `limit` (query, optional, number)
- `type` (query, optional, string)
- `status` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

03 Provider - Business Info

GET `/api/v1/provider/business-info`
Summary: Fetch own provider business information
Allowed role/access: PROVIDER
Notes: Access: PROVIDER. PROVIDER only. providerId is derived from JWT; provider can access only own inventory, offers, orders, analytics, and messages. PROVIDER only. providerId is derived from JWT.
Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

PATCH `/api/v1/provider/business-info`
Summary: Update own provider business information
Allowed role/access: PROVIDER
Notes: Access: PROVIDER. PROVIDER only. providerId is derived from JWT; provider can access only own inventory, offers, orders, analytics, and messages. PROVIDER only. Cannot set approvalStatus/isActive; material business changes require verification review.
Request payload(s):
payload

```
{
  "businessName": "Global Logistics Solutions",
  "legalName": "Global Logistics Solutions LLC",
  "taxId": "XX-XXXXXXX",
  "businessCategoryId": "category_id",
  "email": "ops@globallogistics.com",
  "phone": "+1 (555) 012-3456",
  "businessAddress": "123 Main Street",
  "serviceArea": "New York, USA",
  "headquarters": "New York, USA",
  "website": "https://www.sylviabond.com",
  "storeAddress": {
    "line1": "842 Industrial Way, Suite 102",
    "city": "San Francisco",
    "state": "CA",
    "country": "USA",
    "postalCode": "94107",
    "latitude": 37.7749,
    "longitude": -122.4194
  },
  "businessHours": [
    {
      "day": "MONDAY",
      "isOpen": true,
      "openTime": "09:00",
      "closeTime": "18:00"
```

```
    }
  ],
  "fulfillmentMethods": [
    "PICKUP"
  ],
  "autoAcceptOrders": false
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

03 Provider - Reviews

GET `/api/v1/provider/reviews/summary`
Summary: Fetch provider rating summary
Allowed role/access: PROVIDER
Notes: Access: PROVIDER. PROVIDER only. Provider can access only own reviews. PROVIDER only. Uses shared Review records visible to provider/customer/admin modules.
Response body:

```
{
  "success": true,
  "data": {
    "averageRating": 4.8,
    "reviewCount": 128,
    "distribution": {
      "1": 1,
      "2": 2,
      "3": 5,
      "4": 12,
      "5": 80
    }
  },
  "message": "Review summary fetched successfully."
}
```

GET `/api/v1/provider/reviews/filter-options`
Summary: Fetch provider review filter options
Allowed role/access: PROVIDER
Notes: Access: PROVIDER. PROVIDER only. PROVIDER only. Declared before /provider/reviews/:id.
Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/provider/reviews`
Summary: List provider reviews
Allowed role/access: PROVIDER
Notes: Access: PROVIDER. PROVIDER only. Provider can access only own reviews. PROVIDER only. Shows only reviews for own provider account/orders and excludes hidden/removed reviews.
Parameters:

- `page` (query, optional, number)
- `limit` (query, optional, number)
- `rating` (query, optional, number)
- `hasResponse` (query, optional, boolean)
- `search` (query, optional, string)
- `sortBy` (query, optional, string)
- `sortOrder` (query, optional, string)

Response body:


```
{
  "success": true,
  "data": [
    {
      "id": "review_id",
      "orderId": "order_id",
      "orderNumber": "ORD-45678",
      "customer": {
        "id": "customer_id",
        "name": "Michael Chen",
        "avatarUrl": null
      },
      "rating": 5,
      "comment": "Great packaging and timely delivery.",
      "createdAt": "2026-04-08T10:00:00.000Z",
      "isNew": true,
      "likesCount": 3,
      "response": {
        "id": "response_id",
        "body": "Thank you for your feedback!",
        "createdAt": "2026-04-08T11:00:00.000Z"
      }
    }
  ],
  "meta": {
    "page": 1,
    "limit": 10,
    "total": 1,
    "totalPages": 1
  },
  "message": "Reviews fetched successfully."
}
```


GET `/api/v1/provider/reviews/{id}`
Summary: Fetch Provider Reviews details
Allowed role/access: PROVIDER
Notes: Access: PROVIDER. PROVIDER only. Provider can access only own reviews.
Parameters:

- `id` (path, required, string)

Response body:

```
{
  "success": true,
  "data": {
    "id": "review_id",
    "rating": 5,
    "comment": "Great packaging and timely delivery.",
    "customer": {
      "id": "customer_id",
      "name": "Michael Chen",
      "avatarUrl": null
    },
    "order": {
      "id": "order_id",
      "orderNumber": "ORD-45678",
      "createdAt": "2026-04-08T09:00:00.000Z"
    },
    "response": {
      "id": "response_id",
      "body": "Thank you for your feedback!",
      "createdAt": "2026-04-08T11:00:00.000Z"
    }
  },
  "message": "Review fetched successfully."
}
```


POST `/api/v1/provider/reviews/{id}/response`
Summary: Post public review response
Allowed role/access: PROVIDER
Notes: Access: PROVIDER. PROVIDER only. Provider can respond only to own reviews. PROVIDER only. Provider can respond only to own review. Only one active public response per review.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload


```
{
  "body": "Thank you for your kind words, Sarah. We are happy you loved the packaging."
}
```


Response body:


```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```


PATCH `/api/v1/provider/reviews/{id}/response`
Summary: Update public review response
Allowed role/access: PROVIDER
Notes: Access: PROVIDER. PROVIDER only. Provider can update only own review responses. PROVIDER only. Updates only provider's own active response; customer review content is never modified.
Parameters:

- `id` (path, required, string)

Request payload(s):

payload

```
{
  "body": "Thank you for your kind words, Sarah. We are happy you loved the packaging."
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

DELETE `/api/v1/provider/reviews/{id}/response`
Summary: Delete public review response
Allowed role/access: PROVIDER
Notes: Access: PROVIDER. PROVIDER only. Provider can delete only own review responses. PROVIDER only. Deletes only the provider's own response and does not delete the original customer review.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload

```
"<standard success envelope>"
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

03 Provider - Inventory

GET `/api/v1/provider/inventory`
Summary: List provider inventory items
Allowed role/access: PROVIDER
Notes: Access: PROVIDER. PROVIDER only. providerId is derived from JWT; provider can access only own inventory, offers, orders, analytics, and messages. PROVIDER only. Pending providers cannot access inventory. Provider inventory items do not require separate admin approval; visibility depends on approved/active provider plus manually managed item status and not deleted.
Parameters:

- `page` (query, optional, number)
- `limit` (query, optional, number)
- `search` (query, optional, string)
- `status` (query, optional, string)
- `categoryId` (query, optional, string)
- `sortBy` (query, optional, string)
- `sortOrder` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": [
    {
      "id": "gift_id",
      "name": "Luxury Perfume",
      "price": 99.99,
      "currency": "PKR",
      "status": "ACTIVE",
      "moderationStatus": "NOT_REQUIRED",
      "category": {
        "id": "category_id",
        "name": "Perfumes"
      },
      "variants": [
        {
          "id": "variant_id",
          "name": "50ml",
          "price": 129.99,
          "isDefault": true
        }
      ]
    }
  ],
  "meta": {
    "page": 1,
    "limit": 10,
    "total": 1,
    "totalPages": 1
  },
  "message": "Provider inventory fetched successfully"
}
```


POST `/api/v1/provider/inventory`

Summary: Create provider inventory item with optional nested variants

Allowed role/access: PROVIDER

Notes: Access: PROVIDER. PROVIDER only. providerId is derived from JWT; provider can access only own inventory, offers, orders, analytics, and messages. PROVIDER only. Pending providers cannot access this module. Provider inventory items do not require separate admin approval; approved active providers create ACTIVE inventory directly and can manually change status later.

Request payload(s):

withVariants

```
{
  "name": "Luxury Perfume",
  "description": "Long-lasting premium fragrance.",
  "shortDescription": "Premium fragrance gift.",
  "categoryId": "gift_category_id",
  "price": 99.99,
  "currency": "PKR",
  "imageUrls": [
    "https://cdn.yourdomain.com/gift-images/perfume.png"
  ],
  "variants": [
    {
      "name": "30ml",
      "price": 89.99,
      "originalPrice": 119.99,
      "isPopular": false,
      "isDefault": false,
      "sortOrder": 1,
```

```
      "isActive": true
    },
    {
      "name": "50ml",
      "price": 129.99,
      "originalPrice": 159.99,
      "isPopular": true,
      "isDefault": true,
      "sortOrder": 2,
      "isActive": true
    }
  ]
}
```


Response body:


```
{
  "success": true,
  "data": {
    "id": "gift_id",
    "name": "Luxury Perfume",
    "price": 99.99,
    "currency": "PKR",
    "status": "ACTIVE",
    "moderationStatus": "NOT_REQUIRED",
    "variants": [
      {
        "id": "variant_id",
        "name": "50ml",
        "price": 129.99,
        "originalPrice": 159.99,
        "isDefault": true,
        "isActive": true
      }
    ]
  },
  "message": "Inventory item created successfully"
}
```


GET `/api/v1/provider/inventory/stats`
Summary: Fetch provider inventory stats
Allowed role/access: PROVIDER
Notes: Access: PROVIDER. PROVIDER only. providerId is derived from JWT; provider can access only own inventory, offers, orders, analytics, and messages.
Response body:


```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```


GET `/api/v1/provider/inventory/lookup`
Summary: Lookup active provider inventory items
Allowed role/access: PROVIDER
Notes: Access: PROVIDER. PROVIDER only. providerId is derived from JWT; provider can access only own inventory, offers, orders, analytics, and messages. PROVIDER only. Gift moderation approval is not required for provider inventory lookup.
Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/provider/inventory/{id}`
Summary: Fetch own provider inventory item details
Allowed role/access: PROVIDER
Notes: Access: PROVIDER. PROVIDER only. providerId is derived from JWT; provider can access only own inventory, offers, orders, analytics, and messages.
Parameters:

- `id` (path, required, string)

Response body:

```
{
  "success": true,
  "data": {
    "id": "gift_id",
    "name": "Luxury Perfume",
    "description": "Long-lasting premium fragrance.",
    "price": 99.99,
    "currency": "PKR",
    "status": "ACTIVE",
    "moderationStatus": "NOT_REQUIRED",
    "imageUrls": [
      "https://cdn.yourdomain.com/gift-images/perfume.png"
    ],
    "variants": [
      {
        "id": "variant_id",
        "name": "50ml",
        "price": 129.99,
        "originalPrice": 159.99,
        "isDefault": true,
        "isActive": true
      }
    ]
  },
  "message": "Inventory item fetched successfully"
}
```

PATCH `/api/v1/provider/inventory/{id}`
Summary: Update own provider inventory item and upsert variants
Allowed role/access: PROVIDER
Notes: Access: PROVIDER. PROVIDER only. providerId is derived from JWT; provider can access only own inventory, offers, orders, analytics, and messages. Provider can update only own inventory item. Variant id must belong to the provider-owned gift. Status and availability changes go through this same PATCH endpoint. Stock-only availability updates do not trigger unnecessary moderation; material changes preserve existing moderation behavior.
Parameters:

- `id` (path, required, string)

Request payload(s):
updateItem

```
{
  "name": "Luxury Perfume",
  "description": "Updated premium fragrance.",
  "replaceVariants": false,
  "variants": [
    {
      "id": "variant_id",
      "name": "50ml",
      "price": 129.99,
      "originalPrice": 159.99,
      "isPopular": true,
      "isDefault": true,
      "sortOrder": 2,
      "isActive": true
    },
    {
      "name": "150ml",
      "price": 249.99,
      "originalPrice": 299.99,
      "isPopular": false,
      "isDefault": false,
      "sortOrder": 4,
      "isActive": true
    }
  ]
}
```

activateItem

```
{
  "status": "ACTIVE",
  "isAvailable": true,
  "reason": "Item reactivated after stock update."
}
```

deactivateItem

```
{
  "status": "INACTIVE",
  "isAvailable": false,
  "reason": "Item paused by provider."
}
```

markOutOfStock

```
{
  "isAvailable": false,
  "reason": "Temporarily out of stock."
}
```

Response body:

```
{
  "success": true,
  "data": {
    "id": "gift_id",
    "name": "Luxury Perfume",
    "status": "ACTIVE",
    "isAvailable": true,
    "variants": [
      {
```

```
        "id": "variant_id",
        "name": "50ml",
        "price": 129.99,
        "isDefault": true,
        "isActive": true
      }
    ]
  },
  "message": "Inventory item updated successfully"
}
```

DELETE `/api/v1/provider/inventory/{id}`
Summary: Delete own inventory item
Allowed role/access: PROVIDER
Notes: Access: PROVIDER. PROVIDER only. providerId is derived from JWT; provider can access only own inventory, offers, orders, analytics, and messages. Permanently deletes the provider inventory item.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload

```
"<standard success envelope>"
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

03 Provider - Promotional Offers

GET `/api/v1/provider/offers`
Summary: List Provider Offers
Allowed role/access: PROVIDER
Notes: Access: PROVIDER. PROVIDER only. providerId is derived from JWT; provider can access only own inventory, offers, orders, analytics, and messages.
Parameters:

- `page` (query, optional, number)
- `limit` (query, optional, number)
- `search` (query, optional, string)
- `status` (query, optional, string)
- `itemId` (query, optional, string)
- `sortBy` (query, optional, string)
- `sortOrder` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
```

```
  "message": "Request completed successfully."
}
```

POST `/api/v1/provider/offers`
Summary: Create Provider Offers
Allowed role/access: PROVIDER
Notes: Access: PROVIDER. PROVIDER only. providerId is derived from JWT; provider can access only own inventory, offers, orders, analytics, and messages.
Request payload(s):
payload

```
{
  "itemId": "<string>",
  "title": "<string>",
  "description": "<string>",
  "discountType": "PERCENTAGE",
  "discountValue": 1.0,
  "startDate": "<string>",
  "endDate": "<string>",
  "eligibilityRules": "<string>",
  "isActive": true
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/provider/offers/{id}`
Summary: Fetch Provider Offers details
Allowed role/access: PROVIDER
Notes: Access: PROVIDER. PROVIDER only. providerId is derived from JWT; provider can access only own inventory, offers, orders, analytics, and messages.
Parameters:

- `id` (path, required, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

PATCH `/api/v1/provider/offers/{id}`
Summary: Update own provider promotional offer
Allowed role/access: PROVIDER
Notes: Access: PROVIDER. PROVIDER only. providerId is derived from JWT; provider can access only own inventory, offers, orders, analytics, and messages. PROVIDER only. Updates are scoped to the authenticated provider offer. Standard offer fields preserve current update behavior; status or isActive changes are handled through this same PATCH endpoint.

Parameters:

- `id` (path, required, string)

Request payload(s):
updateOffer

```
{
  "title": "Weekend Discount",
  "description": "Weekend-only campaign.",
  "discountType": "PERCENTAGE",
  "discountValue": 20,
  "startDate": "2026-06-01T00:00:00.000Z",
  "endDate": "2026-06-03T23:59:59.000Z"
}
```

activateOffer

```
{
  "title": "Weekend Discount",
  "isActive": true,
  "status": "ACTIVE",
  "reason": "Provider reactivated offer."
}
```

deactivateOffer

```
{
  "isActive": false,
  "status": "INACTIVE",
  "reason": "Provider paused offer."
}
```

Response body:

```
{
  "success": true,
  "data": {
    "id": "offer_id",
    "title": "Weekend Discount",
    "isActive": true,
    "status": "ACTIVE",
    "approvalStatus": "APPROVED"
  },
  "message": "Promotional offer updated successfully"
}
```

DELETE `/api/v1/provider/offers/{id}`
Summary: Delete Provider Offers
Allowed role/access: PROVIDER
Notes: Access: PROVIDER. PROVIDER only. providerId is derived from JWT; provider can access only own inventory, offers, orders, analytics, and messages.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload

```
"<standard success envelope>"
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

03 Provider - Orders

GET `/api/v1/provider/orders`
Summary: List own assigned provider orders
Allowed role/access: PROVIDER
Notes: Access: PROVIDER. PROVIDER only. providerId is derived from JWT; provider can access only own inventory, offers, orders, analytics, and messages. PROVIDER only. Returns only orders assigned to the authenticated providerId. Default status filter is PENDING.
Parameters:

- `page` (query, optional, number)
- `limit` (query, optional, number)
- `status` (query, optional, string)
- `search` (query, optional, string)
- `fromDate` (query, optional, string)
- `toDate` (query, optional, string)
- `sortBy` (query, optional, string)
- `sortOrder` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": [
    {
      "id": "provider_order_id",
      "orderId": "order_id",
      "orderNumber": "ORD-10293",
      "status": "PENDING",
      "paymentStatus": "SUCCEEDED",
      "customer": {
        "name": "Sarah Jenkins",
        "phone": "+15551234567"
      },
      "itemPreview": [
        {
          "name": "Premium Sneakers",
          "imageUrl": "https://cdn.yourdomain.com/gifts/sneaker.png"
        }
      ],
      "itemCount": 3,
      "totalPayout": 142,
      "currency": "PKR",
      "createdAt": "2026-10-24T10:45:00.000Z",
      "receivedAgoText": "5m ago"
    }
  ],
  "message": "Provider orders fetched successfully."
}
```

GET `/api/v1/provider/orders/history`
Summary: List own provider order history
Allowed role/access: PROVIDER
Notes: Access: PROVIDER. PROVIDER only. providerId is derived from JWT; provider can access only own inventory, offers, orders, analytics, and messages. PROVIDER only. Uses ProviderOrder records scoped to the authenticated provider. Status tabs map to provider order statuses.
Parameters:

- `page` (query, optional, number)
- `limit` (query, optional, number)
- `status` (query, optional, string)
- `search` (query, optional, string)
- `fromDate` (query, optional, string)
- `toDate` (query, optional, string)
- `sortBy` (query, optional, string)
- `sortOrder` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/provider/orders/summary`
Summary: Fetch own provider order summary
Allowed role/access: PROVIDER
Notes: Access: PROVIDER. PROVIDER only. providerId is derived from JWT; provider can access only own inventory, offers, orders, analytics, and messages. Route intentionally declared before :id. PROVIDER only.
Parameters:

- `fromDate` (query, optional, string)
- `toDate` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/provider/orders/reject-reasons`
Summary: List provider order reject reasons
Allowed role/access: PROVIDER
Notes: Access: PROVIDER. PROVIDER only. providerId is derived from JWT; provider can access only own inventory, offers, orders, analytics, and messages. Route intentionally declared before :id.
Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

POST `/api/v1/provider/orders/{id}/action`
Summary: Run provider order action
Allowed role/access: PROVIDER
Notes: Access: PROVIDER. PROVIDER only. providerId is derived from JWT; provider can access only own inventory, offers, orders, analytics, and messages. PROVIDER only. ACCEPT allows PENDING -> ACCEPTED. REJECT allows PENDING -> REJECTED with required reason. UPDATE_STATUS enforces provider order state machine. FULFILL stores dispatch details and moves order to shipped/fulfilled state.
Parameters:

- `id` (path, required, string)

Request payload(s):
accept

```
{
  "action": "ACCEPT",
  "comment": "Order accepted and will be processed shortly.",
  "notifyCustomer": true
}
```

reject

```
{
  "action": "REJECT",
  "reason": "OUT_OF_STOCK",
  "comment": "The selected item is currently unavailable.",
  "notifyCustomer": true
}
```

updateStatus

```
{
  "action": "UPDATE_STATUS",
  "status": "PROCESSING",
  "note": "Order is being prepared.",
  "notifyCustomer": true
}
```

fulfill

```
{
  "action": "FULFILL",
  "dispatchAt": "2026-10-25T10:00:00.000Z",
  "estimatedDeliveryAt": "2026-10-26T10:00:00.000Z",
  "carrier": "FedEx",
  "trackingNumber": "FDX-123456",
  "notifyCustomer": true,
  "note": "Package handed to courier."
}
```

Response body:

```
{
  "success": true,
  "data": {
    "id": "provider_order_id",
    "status": "PROCESSING",
    "orderId": "order_id",
    "orderNumber": "ORD-10293"
  },
```

```
  "message": "Provider order action completed successfully."
}
```

GET `/api/v1/provider/orders/{id}/timeline`
Summary: Fetch own provider order timeline
Allowed role/access: PROVIDER
Notes: Access: PROVIDER. PROVIDER only. providerId is derived from JWT; provider can access only own inventory, offers, orders, analytics, and messages. PROVIDER only. Timeline is scoped to the authenticated provider order.
Parameters:

- `id` (path, required, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/provider/orders/{id}/checklist`
Summary: Fetch own provider order checklist
Allowed role/access: PROVIDER
Notes: Access: PROVIDER. PROVIDER only. providerId is derived from JWT; provider can access only own inventory, offers, orders, analytics, and messages. PROVIDER only. Checklist is operational and does not change status automatically.
Parameters:

- `id` (path, required, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

PATCH `/api/v1/provider/orders/{id}/checklist`
Summary: Update own provider order checklist
Allowed role/access: PROVIDER
Notes: Access: PROVIDER. PROVIDER only. providerId is derived from JWT; provider can access only own inventory, offers, orders, analytics, and messages. PROVIDER only. Checklist updates do not directly change order status.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload

```
{
  "itemsPacked": true,
  "giftMessageAttached": true,
  "addressVerified": true,
  "customerContactChecked": true,
  "readyForCourier": false,
  "customItems": [
    {
```

```
      "id": "checklist_item_id",
      "label": "Include gift wrap",
      "isCompleted": true
    }
  ]
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/provider/orders/{id}`
Summary: Fetch own provider order details
Allowed role/access: PROVIDER
Notes: Access: PROVIDER. PROVIDER only. providerId is derived from JWT; provider can access only own inventory, offers, orders, analytics, and messages. PROVIDER only. Does not expose customer card/payment secrets or admin-only order fields.
Parameters:

- `id` (path, required, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

03 Provider - Payout Methods

GET `/api/v1/provider/payout-methods`
Summary: List own provider payout methods
Allowed role/access: PROVIDER
Notes: Access: PROVIDER. PROVIDER only. providerId is derived from JWT; provider can access only own inventory, offers, orders, analytics, and messages. PROVIDER only. providerId is derived from JWT. Returns masked payout metadata only.
Response body:

```
{
  "success": true,
  "data": {
    "primary": {
      "id": "payout_method_id",
      "type": "BANK_ACCOUNT",
      "bankName": "Chase Bank",
      "maskedAccount": "Checking Account **** 5678",
      "accountHolderName": "Sylvia Bond",
      "payerId": "SB-4491-001",
      "verificationStatus": "VERIFIED",
      "isDefault": true,
      "isActive": true
    },
```

```
    "methods": [
      {
        "id": "payout_method_id",
        "type": "BANK_ACCOUNT",
        "bankName": "Chase Bank",
        "maskedAccount": "Checking Account **** 6789",
        "accountHolderName": "Sylvia Bond",
        "verificationStatus": "VERIFIED",
        "isDefault": true,
        "isActive": true
      }
    ]
  },
  "message": "Provider payout methods fetched successfully."
}
```

POST `/api/v1/provider/payout-methods/bank-accounts`
Summary: Add provider bank payout method
Allowed role/access: PROVIDER
Notes: Access: PROVIDER. PROVIDER only. providerId is derived from JWT; provider can access only own inventory, offers, orders, analytics, and messages. PROVIDER only. Stores masked metadata only; raw routing/account/IBAN values are never returned or persisted.
Request payload(s):
bank

```
{
  "accountHolderName": "Sylvia Bond",
  "bankName": "Chase Bank",
  "accountType": "CHECKING",
  "country": "US",
  "currency": "USD",
  "routingNumber": "110000000",
  "accountNumber": "000123456789",
  "iban": null,
  "isDefault": true
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/provider/payout-methods/{id}`
Summary: Fetch own provider payout method details
Allowed role/access: PROVIDER
Notes: Access: PROVIDER. PROVIDER only. providerId is derived from JWT; provider can access only own inventory, offers, orders, analytics, and messages. PROVIDER only. Full account number, IBAN, and routing number are never returned.
Parameters:

- `id` (path, required, string)

Response body:

```
{
  "success": true,
```

```
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

PATCH /api/v1/provider/payout-methods/{id}
Summary: Update provider payout method display metadata
Allowed role/access: PROVIDER
Notes: Access: PROVIDER. PROVIDER only. providerId is derived from JWT; provider can access only own inventory, offers, orders, analytics, and messages. PROVIDER only. Bank/routing/account numbers cannot be edited; add a new payout method instead.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload

```
{
  "accountHolderName": "Sylvia Bond",
  "bankName": "Chase Bank Personal",
  "isActive": true
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

DELETE /api/v1/provider/payout-methods/{id}
Summary: Delete own provider payout method
Allowed role/access: PROVIDER
Notes: Access: PROVIDER. PROVIDER only. providerId is derived from JWT; provider can access only own inventory, offers, orders, analytics, and messages. PROVIDER only. Permanently deletes the payout method and blocks deletion when pending provider payout adjustments exist.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload

```
"<standard success envelope>"
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

PATCH /api/v1/provider/payout-methods/{id}/default
Summary: Set default provider payout method

Allowed role/access: PROVIDER
Notes: Access: PROVIDER. PROVIDER only. providerId is derived from JWT; provider can access only own inventory, offers, orders, analytics, and messages. PROVIDER only. Only own verified active payout methods can become default.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload

```
"<standard success envelope>"
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

POST `/api/v1/provider/payout-methods/{id}/verify`
Summary: Start provider payout method verification
Allowed role/access: PROVIDER
Notes: Access: PROVIDER. PROVIDER only. providerId is derived from JWT; provider can access only own inventory, offers, orders, analytics, and messages. PROVIDER only. If Plaid/Stripe Connect is not configured, MANUAL keeps verification pending for admin/system review.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload

```
{
  "verificationMethod": "MANUAL",
  "publicToken": "plaid_public_token"
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

03 Provider - Payouts

GET `/api/v1/provider/payouts`
Summary: List own provider payout history
Allowed role/access: PROVIDER
Notes: Access: PROVIDER. PROVIDER only. providerId is derived from JWT; provider can access only own inventory, offers, orders, analytics, and messages. PROVIDER only. providerId is derived from JWT.
Parameters:

- `page` (query, optional, number)
- `limit` (query, optional, number)

- `range` (query, optional, string)
- `fromDate` (query, optional, string)
- `toDate` (query, optional, string)
- `status` (query, optional, string)
- `sortBy` (query, optional, string)
- `sortOrder` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/provider/payouts/summary`
Summary: Fetch own provider payout summary
Allowed role/access: PROVIDER
Notes: Access: PROVIDER. PROVIDER only. providerId is derived from JWT; provider can access only own inventory, offers, orders, analytics, and messages. Route declared before :id. PROVIDER only.
Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/provider/payouts/preview`
Summary: Preview provider payout request
Allowed role/access: PROVIDER
Notes: Access: PROVIDER. PROVIDER only. providerId is derived from JWT; provider can access only own inventory, offers, orders, analytics, and messages. Route declared before :id. Validates available balance and verified payout method.
Parameters:

- `amount` (query, required, number)
- `payoutMethodId` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

POST `/api/v1/provider/payouts/request`
Summary: Request provider payout
Allowed role/access: PROVIDER
Notes: Access: PROVIDER. PROVIDER only. providerId is derived from JWT; provider can access only own inventory, offers, orders, analytics, and messages. PROVIDER only. Uses idempotencyKey to block duplicate payout requests.
Request payload(s):
payload

```
{
  "amount": 1250,
```

```
  "payoutMethodId": "payout_method_id",
  "idempotencyKey": "provider_payout_2026_001"
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/provider/payouts/{id}`
Summary: Fetch own provider payout details
Allowed role/access: PROVIDER
Notes: Access: PROVIDER. PROVIDER only. providerId is derived from JWT; provider can access only own inventory, offers, orders, analytics, and messages. PROVIDER only. Scoped to authenticated provider.
Parameters:

- `id` (path, required, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

POST `/api/v1/provider/payouts/{id}/cancel`
Summary: Cancel own pending provider payout
Allowed role/access: PROVIDER
Notes: Access: PROVIDER. PROVIDER only. providerId is derived from JWT; provider can access only own inventory, offers, orders, analytics, and messages. Can cancel only PENDING payouts and returns locked balance to AVAILABLE.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload

```
{
  "reason": "Requested by provider."
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

03 Provider - Refund Requests

GET `/api/v1/provider/refund-requests`
Summary: List own provider refund requests
Allowed role/access: PROVIDER
Notes: Access: PROVIDER. PROVIDER only. providerId is derived from JWT; provider can access only own inventory, offers, orders, analytics, and messages. PROVIDER only. Returns refund requests for provider orders assigned to the authenticated provider. Search supports order number, customer name, and customer email.
Parameters:

- `page` (query, optional, number)
- `limit` (query, optional, number)
- `status` (query, optional, string)
- `search` (query, optional, string)
- `fromDate` (query, optional, string)
- `toDate` (query, optional, string)
- `sortBy` (query, optional, string)
- `sortOrder` (query, optional, string)

Response body:


```
{
  "success": true,
  "data": [
    {
      "id": "refund_request_id",
      "providerOrderId": "provider_order_id",
      "orderNumber": "88417",
      "customer": {
        "name": "Jane Cooper",
        "email": "jane.cooper@example.com",
        "avatarUrl": "https://cdn.yourdomain.com/customer-avatar.jpg"
      },
      "requestedAmount": 45,
      "currency": "PKR",
      "status": "REQUESTED",
      "customerReason": "Item was damaged on arrival",
      "createdAt": "2026-10-23T18:10:00.000Z"
    }
  ],
  "message": "Provider refund requests fetched successfully."
}
```


GET `/api/v1/provider/refund-requests/summary`
Summary: Fetch own refund request summary
Allowed role/access: PROVIDER
Notes: Access: PROVIDER. PROVIDER only. providerId is derived from JWT; provider can access only own inventory, offers, orders, analytics, and messages. Route intentionally declared before :id. PROVIDER only.
Response body:


```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```


GET `/api/v1/provider/refund-requests/reject-reasons`

Summary: List refund rejection reasons
Allowed role/access: PROVIDER
Notes: Access: PROVIDER. PROVIDER only. providerId is derived from JWT; provider can access only own inventory, offers, orders, analytics, and messages. Route intentionally declared before :id. PROVIDER only.
Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/provider/refund-requests/{id}`
Summary: Fetch own refund request details
Allowed role/access: PROVIDER
Notes: Access: PROVIDER. PROVIDER only. providerId is derived from JWT; provider can access only own inventory, offers, orders, analytics, and messages. PROVIDER only. Refund request must belong to the authenticated provider order and never exposes Stripe secrets or raw card data.
Parameters:

- `id` (path, required, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

POST `/api/v1/provider/refund-requests/{id}/action`
Summary: Run own refund request action
Allowed role/access: PROVIDER
Notes: Access: PROVIDER. PROVIDER only. providerId is derived from JWT; provider can access only own inventory, offers, orders, analytics, and messages. PROVIDER only. APPROVE validates ownership, REQUESTED status, requested/refundable amount, creates refund transaction marker/timeline, and notifies the customer by default. REJECT validates ownership and REQUESTED status, requires reason, creates timeline, and does not create a Stripe refund.
Parameters:

- `id` (path, required, string)

Request payload(s):
approve

```
{
  "action": "APPROVE",
  "comment": "Refund approved after evidence review.",
  "refundAmount": 45,
  "notifyCustomer": true
}
```

reject

```
{
  "action": "REJECT",
  "reason": "NOT_COVERED_BY_POLICY",
  "comment": "Refund rejected after evidence review.",
```

```
  "notifyCustomer": true
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

03 Provider - Order Analytics

GET `/api/v1/provider/orders/performance`
Summary: Fetch own provider order performance
Allowed role/access: PROVIDER
Notes: Access: PROVIDER. PROVIDER only. providerId is derived from JWT; provider can access only own inventory, offers, orders, analytics, and messages. PROVIDER only. Completion rate uses completed / non-cancelled own provider orders.
Parameters:

- `range` (query, optional, string)
- `fromDate` (query, optional, string)
- `toDate` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/provider/orders/analytics/revenue`
Summary: Fetch own provider revenue analytics
Allowed role/access: PROVIDER
Notes: Access: PROVIDER. PROVIDER only. providerId is derived from JWT; provider can access only own inventory, offers, orders, analytics, and messages. PROVIDER only. Revenue uses provider totalPayout for paid active/completed provider orders.
Parameters:

- `range` (query, optional, string)
- `fromDate` (query, optional, string)
- `toDate` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/provider/orders/analytics/ratings`
Summary: Fetch own provider ratings analytics

Allowed role/access: PROVIDER
Notes: Access: PROVIDER. PROVIDER only. providerId is derived from JWT; provider can access only own inventory, offers, orders, analytics, and messages. PROVIDER only. Returns stable zero values until reviews module is available.
Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/provider/orders/recent`
Summary: List recent own provider orders
Allowed role/access: PROVIDER
Notes: Access: PROVIDER. PROVIDER only. providerId is derived from JWT; provider can access only own inventory, offers, orders, analytics, and messages. PROVIDER only. Defaults to 5 latest orders.
Parameters:

- `limit` (query, optional, number)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/provider/orders/export`
Summary: Export own provider orders as CSV
Allowed role/access: PROVIDER
Notes: Access: PROVIDER. PROVIDER only. providerId is derived from JWT; provider can access only own inventory, offers, orders, analytics, and messages. PROVIDER only. Export is scoped to logged-in provider orders.
Parameters:

- `status` (query, optional, string)
- `fromDate` (query, optional, string)
- `toDate` (query, optional, string)
- `format` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

04 Gifts - Categories

GET `/api/v1/gift-categories/lookup`
Summary: Lookup active gift categories
Allowed role/access: PUBLIC
Notes: Access: PUBLIC. Active gift category lookup. Public lookup under Gift Categories. Returns active category identifiers and media fields for lightweight selectors.

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/gift-categories`
Summary: List gift categories
Allowed role/access: SUPER_ADMIN or ADMIN with giftCategories.read
Notes: Access: SUPER_ADMIN or ADMIN with giftCategories.read. SUPER_ADMIN or ADMIN with giftCategories.read permission. RBAC permission: giftCategories.read. Returns soft-delete-filtered categories with gift counts and category media fields.
Parameters:

- `page` (query, optional, number)
- `limit` (query, optional, number)
- `search` (query, optional, string)
- `isActive` (query, optional, boolean)
- `sortBy` (query, optional, string)
- `sortOrder` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

POST `/api/v1/gift-categories`
Summary: Create gift category
Allowed role/access: SUPER_ADMIN or ADMIN with giftCategories.create
Notes: Access: SUPER_ADMIN or ADMIN with giftCategories.create. SUPER_ADMIN or ADMIN with giftCategories.create permission. RBAC permission: giftCategories.create. Slug is auto-generated and unique. backgroundColor defaults to #F3E8FF; color remains a backward-compatible alias.
Request payload(s):
create

```
{
  "name": "Perfumes",
  "description": "Premium fragrance gifts.",
  "iconKey": "perfume",
  "backgroundColor": "#E9D5FF",
  "imageUrl": "https://<YOUR_PUBLIC_BUCKET_OR_CDN_URL>/gift-category-images/
perfumes.png",
  "sortOrder": 1,
  "isActive": true
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
```

```
  "message": "Request completed successfully."
}
```

GET `/api/v1/gift-categories/stats`
Summary: Fetch gift category stats
Allowed role/access: SUPER_ADMIN or ADMIN with giftCategories.read
Notes: Access: SUPER_ADMIN or ADMIN with giftCategories.read. SUPER_ADMIN or ADMIN with giftCategories.read permission. RBAC permission: giftCategories.read. Returns admin category inventory counters.
Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/gift-categories/{id}`
Summary: Fetch gift category details
Allowed role/access: SUPER_ADMIN or ADMIN with giftCategories.read
Notes: Access: SUPER_ADMIN or ADMIN with giftCategories.read. SUPER_ADMIN or ADMIN with giftCategories.read permission. RBAC permission: giftCategories.read. Includes backgroundColor and imageUrl for customer app design support.
Parameters:

- `id` (path, required, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

PATCH `/api/v1/gift-categories/{id}`
Summary: Update gift category
Allowed role/access: SUPER_ADMIN or ADMIN with giftCategories.update
Notes: Access: SUPER_ADMIN or ADMIN with giftCategories.update. SUPER_ADMIN or ADMIN with giftCategories.update permission. RBAC permission: giftCategories.update. Slug is regenerated when name changes. Soft-deleted categories are not updated.
Parameters:

- `id` (path, required, string)

Request payload(s):
update

```
{
  "backgroundColor": "#F3E8FF",
  "imageUrl": "https://<YOUR_PUBLIC_BUCKET_OR_CDN_URL>/gift-category-images/
perfumes.png",
  "isActive": true
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

DELETE `/api/v1/gift-categories/{id}`
Summary: Delete gift category
Allowed role/access: SUPER_ADMIN or ADMIN with giftCategories.delete
Notes: Access: SUPER_ADMIN or ADMIN with giftCategories.delete. SUPER_ADMIN or ADMIN with giftCategories.delete permission. RBAC permission: giftCategories.delete. Permanently deletes the category. Categories with attached gifts cannot be deleted; delete writes an audit log.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload

```
"<standard success envelope>"
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

04 Gifts - Management

GET `/api/v1/gifts`
Summary: List admin gifts
Allowed role/access: SUPER_ADMIN or ADMIN with gifts.read
Notes: Access: SUPER_ADMIN or ADMIN with gifts.read. SUPER_ADMIN or ADMIN with gifts.read permission. SUPER_ADMIN/ADMIN with gifts.read. Supports category/provider/status/moderation filters.
Parameters:

- `page` (query, optional, number)
- `limit` (query, optional, number)
- `search` (query, optional, string)
- `categoryId` (query, optional, string)
- `providerId` (query, optional, string)
- `status` (query, optional, string)
- `moderationStatus` (query, optional, string)
- `isPublished` (query, optional, boolean)
- `sortBy` (query, optional, string)
- `sortOrder` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

POST `/api/v1/gifts`
Summary: Create admin gift with optional nested variants
Allowed role/access: SUPER_ADMIN or ADMIN with gifts.create
Notes: Access: SUPER_ADMIN or ADMIN with gifts.create. SUPER_ADMIN or ADMIN with gifts.create permission. SUPER_ADMIN/ADMIN with gifts.create. Nested variants are created in the same transaction and stored in GiftVariant.
Request payload(s):
withVariants

```
{
  "name": "Luxury Perfume",
  "description": "Long-lasting premium fragrance.",
  "shortDescription": "Premium fragrance gift.",
  "categoryId": "gift_category_id",
  "providerId": "provider_id",
  "price": 99.99,
  "currency": "PKR",
  "imageUrls": [
    "https://cdn.yourdomain.com/gift-images/perfume.png"
  ],
  "isPublished": true,
  "isFeatured": false,
  "tags": [
    "perfume",
    "luxury"
  ],
  "moderationStatus": "APPROVED",
  "variants": [
    {
      "name": "30ml",
      "price": 89.99,
      "originalPrice": 119.99,
      "isPopular": false,
```

```
      "isDefault": false,
      "sortOrder": 1,
      "isActive": true
    },
    {
      "name": "50ml",
      "price": 129.99,
      "originalPrice": 159.99,
      "isPopular": true,
      "isDefault": true,
      "sortOrder": 2,
      "isActive": true
    }
  ]
}
```

Response body:

```
{
  "success": true,
  "data": {
    "id": "gift_id",
    "name": "Luxury Perfume",
    "price": 99.99,
    "currency": "PKR",
    "variants": [
      {
        "id": "variant_id",
        "name": "50ml",
        "price": 129.99,
        "originalPrice": 159.99,
        "isPopular": true,
        "isDefault": true,
        "sortOrder": 2,
        "isActive": true
      }
    ]
  },
  "message": "Gift created successfully"
}
```

GET `/api/v1/gifts/stats`
Summary: Fetch gift inventory stats
Allowed role/access: SUPER_ADMIN or ADMIN with gifts.read
Notes: Access: SUPER_ADMIN or ADMIN with gifts.read. SUPER_ADMIN or ADMIN with gifts.read permission.
Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/gifts/export`
Summary: Export gift inventory
Allowed role/access: SUPER_ADMIN or ADMIN with gifts.export
Notes: Access: SUPER_ADMIN or ADMIN with gifts.export. SUPER_ADMIN or ADMIN with gifts.export permission.

Parameters:

- `page` (query, optional, number)
- `limit` (query, optional, number)
- `search` (query, optional, string)
- `categoryId` (query, optional, string)
- `providerId` (query, optional, string)
- `status` (query, optional, string)
- `moderationStatus` (query, optional, string)
- `isPublished` (query, optional, boolean)
- `sortBy` (query, optional, string)
- `sortOrder` (query, optional, string)
- `format` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/gifts/{id}`
Summary: Fetch admin gift details with variants
Allowed role/access: SUPER_ADMIN or ADMIN with gifts.read
Notes: Access: SUPER_ADMIN or ADMIN with gifts.read. SUPER_ADMIN or ADMIN with gifts.read permission.
Parameters:

- `id` (path, required, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

PATCH `/api/v1/gifts/{id}`
Summary: Update admin gift, nested variants, or operational catalog status
Allowed role/access: SUPER_ADMIN or ADMIN with gift-specific update permission
Notes: Access: SUPER_ADMIN or ADMIN with gift-specific update permission. SUPER_ADMIN or ADMIN with gifts.update for standard gift fields and gifts.status.update for operational status changes. SUPER_ADMIN or ADMIN with 'gifts.update' for normal fields and 'gifts.status.update' for status changes. If replaceVariants=true, omitted variants are soft-deleted. Only one default variant is allowed. Moderation decisions stay under POST /api/v1/gift-moderation/:id/action.
Parameters:

- `id` (path, required, string)

Request payload(s):
upsertVariants

```
{
  "name": "Luxury Perfume Updated",
  "replaceVariants": false,
  "variants": [
    {
      "id": "variant_id",
      "name": "50ml",
      "price": 129.99,
```

```
      "originalPrice": 159.99,
      "isPopular": true,
      "isDefault": true,
      "sortOrder": 2,
      "isActive": true
    },
    {
      "name": "150ml",
      "price": 249.99,
      "originalPrice": 299.99,
      "isPopular": false,
      "isDefault": false,
      "sortOrder": 4,
      "isActive": true
    }
  ]
}
```

activateGift

```
{
  "status": "ACTIVE",
  "reason": "Back in stock and approved by admin."
}
```

deactivateGift

```
{
  "status": "INACTIVE",
  "reason": "Temporarily disabled by admin."
}
```

markOutOfStock

```
{
  "status": "OUT_OF_STOCK",
  "reason": "Inventory is depleted."
}
```

Response body:

```
{
  "success": true,
  "data": {
    "id": "gift_id",
    "name": "Luxury Perfume Updated",
    "status": "ACTIVE",
    "variants": [
      {
        "id": "variant_id",
        "name": "50ml",
        "price": 129.99,
        "originalPrice": 159.99,
        "isDefault": true,
        "isActive": true
      }
    ]
  },
  "message": "Gift updated successfully"
}
```

DELETE `/api/v1/gifts/{id}`
Summary: Delete gift
Allowed role/access: SUPER_ADMIN or ADMIN with gifts.delete
Notes: Access: SUPER_ADMIN or ADMIN with gifts.delete. SUPER_ADMIN or ADMIN with gifts.delete permission. Permanently deletes the gift record.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload

```
"<standard success envelope>"
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

04 Gifts - Moderation

GET `/api/v1/gift-moderation`
Summary: List optional gift moderation queue
Allowed role/access: SUPER_ADMIN or ADMIN with giftModeration.read
Notes: Access: SUPER_ADMIN or ADMIN with giftModeration.read. SUPER_ADMIN or ADMIN with giftModeration.read permission. Gift moderation is optional/exception-based; default queue returns only PENDING, FLAGGED, REJECTED, or requiresManualReview=true. Provider-created inventory does not require separate Super Admin approval by default. Gift Moderation is optional/admin review workflow for flagged, reported, high-risk, or admin-forced review items. Provider-created inventory does not require separate Super Admin approval by default; provider approval is the primary marketplace trust gate. Default queue returns only PENDING, FLAGGED, REJECTED, or requiresManualReview=true. Use status=APPROVED, status=NOT_REQUIRED, or includeResolved=true to view resolved/normal inventory.
Parameters:

- `page` (query, optional, number)
- `limit` (query, optional, number)
- `status` (query, optional, string)
- `includeResolved` (query, optional, boolean) When true, includes resolved/normal statuses such as APPROVED and NOT_REQUIRED. Default queue returns only PENDING, FLAGGED, REJECTED, or requiresManualReview=true.
- `search` (query, optional, string)
- `providerId` (query, optional, string)
- `view` (query, optional, string)
- `sortBy` (query, optional, string)
- `sortOrder` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

POST `/api/v1/gift-moderation/{id}/action`

Summary: Run gift moderation action
Allowed role/access: SUPER_ADMIN or ADMIN with gift moderation action permission (APPROVE=>giftModeration.approve, REJECT=>giftModeration.reject, FLAG=>giftModeration.flag)
Notes: Access: SUPER_ADMIN or ADMIN with gift moderation action permission (APPROVE=>giftModeration.approve, REJECT=>giftModeration.reject, FLAG=>giftModeration.flag). SUPER_ADMIN or ADMIN with action-specific gift moderation permission. APPROVE requires giftModeration.approve; REJECT requires giftModeration.reject; FLAG requires giftModeration.flag. SUPER_ADMIN or ADMIN with action-specific gift moderation permission. APPROVE requires 'giftModeration.approve'; REJECT requires 'giftModeration.reject'; FLAG requires 'giftModeration.flag'.
Parameters:

- `id` (path, required, string)

Request payload(s):
approve

```
{
  "action": "APPROVE",
  "comment": "Gift content meets marketplace policy.",
  "notifyProvider": true
}
```

reject

```
{
  "action": "REJECT",
  "reason": "POLICY_VIOLATION",
  "comment": "Listing violates marketplace content policy.",
  "notifyProvider": true
}
```

flagAndHide

```
{
  "action": "FLAG",
  "reason": "POLICY_CONCERN",
  "comment": "Hide while policy team reviews images.",
  "hideFromMarketplace": true,
  "notifyProvider": true
}
```

flagReviewOnly

```
{
  "action": "FLAG",
  "reason": "NEEDS_MANUAL_REVIEW",
  "comment": "Keep visible but queue for manual review.",
  "hideFromMarketplace": false,
  "notifyProvider": false
}
```

Response body:

```
{
  "success": true,
  "data": {
    "id": "gift_id",
    "moderationStatus": "FLAGGED",
    "isPublished": false,
    "hiddenByModeration": true
  },
```

```
  "message": "Gift flagged successfully"
}
```

05 Customer / Guest - Marketplace

GET `/api/v1/customer/home`
Summary: Fetch customer app home
Allowed role/access: REGISTERED_USER or GUEST_USER
Notes: Access: REGISTERED_USER or GUEST_USER. Registered users receive personalized marketplace fields such as wishlist state, default address, and upcoming reminders where applicable. Guest users receive guest-safe marketplace data only. Guest users cannot access wishlist, cart, checkout, addresses, contacts, events, orders, payments, wallet, recurring payments, chats, reviews, or referrals. Guest response behavior includes isWishlisted=false, requiresAuthForWishlist=true, requiresAuthForCart=true, requiresAuthForCheckout=true, defaultAddress=null, upcomingReminder=null, and mode="GUEST" where applicable. Registered users receive personalized marketplace fields such as wishlist state, default address, and upcoming reminders where applicable. Guest users receive guest-safe marketplace data only. Guest users cannot access wishlist, cart, checkout, addresses, contacts, events, orders, payments, wallet, recurring payments, chats, reviews, or referrals. Guest home responses include defaultAddress=null, upcomingReminder=null, and mode="GUEST".
Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/customer/categories`
Summary: List customer marketplace categories
Allowed role/access: REGISTERED_USER or GUEST_USER
Notes: Access: REGISTERED_USER or GUEST_USER. Registered users receive personalized marketplace fields such as wishlist state, default address, and upcoming reminders where applicable. Guest users receive guest-safe marketplace data only. Guest users cannot access wishlist, cart, checkout, addresses, contacts, events, orders, payments, wallet, recurring payments, chats, reviews, or referrals. Guest response behavior includes isWishlisted=false, requiresAuthForWishlist=true, requiresAuthForCart=true, requiresAuthForCheckout=true, defaultAddress=null, upcomingReminder=null, and mode="GUEST" where applicable. Registered users receive personalized marketplace fields such as wishlist state, default address, and upcoming reminders where applicable. Guest users receive guest-safe marketplace data only. Guest users cannot access wishlist, cart, checkout, addresses, contacts, events, orders, payments, wallet, recurring payments, chats, reviews, or referrals.
Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/customer/gifts/discounted`
Summary: List discounted customer gifts
Allowed role/access: REGISTERED_USER or GUEST_USER
Notes: Access: REGISTERED_USER or GUEST_USER. Registered users receive personalized marketplace fields such as wishlist state, default address, and upcoming reminders where applicable. Guest users receive guest-safe marketplace data only. Guest users cannot access wishlist, cart, checkout, addresses, contacts, events, orders, payments, wallet, recurring payments, chats, reviews, or referrals. Guest response behavior includes isWishlisted=false, requiresAuthForWishlist=true, requiresAuthForCart=true, requiresAuthForCheckout=true, defaultAddress=null, upcomingReminder=null, and mode="GUEST" where applicable. Registered users receive personalized marketplace fields such as wishlist state, default

address, and upcoming reminders where applicable. Guest users receive guest-safe marketplace data only. Guest users cannot access wishlist, cart, checkout, addresses, contacts, events, orders, payments, wallet, recurring payments, chats, reviews, or referrals. Discounted gift cards include isWishlisted=false and auth-required flags for guests.
Parameters:

- `page` (query, optional, number)
- `limit` (query, optional, number)
- `search` (query, optional, string)
- `categoryId` (query, optional, string)
- `categorySlug` (query, optional, string)
- `providerId` (query, optional, string)
- `offerOnly` (query, optional, boolean)
- `minPrice` (query, optional, number)
- `maxPrice` (query, optional, number)
- `minRating` (query, optional, number)
- `brand` (query, optional, string)
- `deliveryOption` (query, optional, string)
- `sortBy` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/customer/gifts/filter-options`
Summary: Fetch marketplace gift filter options
Allowed role/access: REGISTERED_USER or GUEST_USER
Notes: Access: REGISTERED_USER or GUEST_USER. Registered users receive personalized marketplace fields such as wishlist state, default address, and upcoming reminders where applicable. Guest users receive guest-safe marketplace data only. Guest users cannot access wishlist, cart, checkout, addresses, contacts, events, orders, payments, wallet, recurring payments, chats, reviews, or referrals. Guest response behavior includes isWishlisted=false, requiresAuthForWishlist=true, requiresAuthForCart=true, requiresAuthForCheckout=true, defaultAddress=null, upcomingReminder=null, and mode="GUEST" where applicable. Registered users receive personalized marketplace fields such as wishlist state, default address, and upcoming reminders where applicable. Guest users receive guest-safe marketplace data only. Guest users cannot access wishlist, cart, checkout, addresses, contacts, events, orders, payments, wallet, recurring payments, chats, reviews, or referrals.
Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/customer/gifts`
Summary: List customer marketplace gifts
Allowed role/access: REGISTERED_USER or GUEST_USER
Notes: Access: REGISTERED_USER or GUEST_USER. Registered users receive personalized marketplace fields such as wishlist state, default address, and upcoming reminders where applicable. Guest users receive guest-safe marketplace data only. Guest users cannot access wishlist, cart, checkout, addresses, contacts, events, orders, payments, wallet, recurring payments, chats, reviews, or referrals. Guest response behavior includes isWishlisted=false, requiresAuthForWishlist=true, requiresAuthForCart=true, requiresAuthForCheckout=true, defaultAddress=null, upcomingReminder=null, and mode="GUEST" where applicable. Registered users receive personalized marketplace fields such as wishlist state, default address, and upcoming reminders where applicable. Guest users receive guest-safe marketplace data only. Guest users cannot access wishlist, cart, checkout, addresses, contacts, events, orders, payments, wallet, recurring payments, chats,

reviews, or referrals. Guest gift cards include isWishlisted=false, requiresAuthForWishlist=true, requiresAuthForCart=true, and requiresAuthForCheckout=true. Provider inventory does not require separate gift moderation approval.

Parameters:

- `page` (query, optional, number)
- `limit` (query, optional, number)
- `search` (query, optional, string)
- `categoryId` (query, optional, string)
- `categorySlug` (query, optional, string)
- `providerId` (query, optional, string)
- `offerOnly` (query, optional, boolean)
- `minPrice` (query, optional, number)
- `maxPrice` (query, optional, number)
- `minRating` (query, optional, number)
- `brand` (query, optional, string)
- `deliveryOption` (query, optional, string)
- `sortBy` (query, optional, string)

Response body:


```
{
  "success": true,
  "data": [
    {
      "id": "gift_id",
      "name": "Luxury Perfume",
      "price": 99.99,
      "currency": "PKR",
      "imageUrl": "https://cdn.yourdomain.com/gift-images/perfume.png",
      "rating": 4.6,
      "isWishlisted": false,
      "requiresAuthForWishlist": true,
      "requiresAuthForCart": true,
      "requiresAuthForCheckout": true,
      "shortDescription": "Premium fragrance gift.",
      "reviewCount": 24,
      "category": {
        "id": "gift_category_id",
        "name": "Perfumes",
        "slug": "perfumes"
      },
      "provider": {
        "id": "provider_id",
        "businessName": "Dcodax Gifts"
      },
      "deliveryOptions": [
        "SAME_DAY",
        "NEXT_DAY",
        "SCHEDULED"
      ],
      "activeOffer": null
    }
  ],
  "meta": {
    "page": 1,
    "limit": 10,
    "total": 1,
    "totalPages": 1
  },
  "message": "Customer gifts fetched successfully"
}
```

GET /api/v1/customer/gifts/{id}
Summary: Fetch customer-safe gift details
Allowed role/access: REGISTERED_USER or GUEST_USER
Notes: Access: REGISTERED_USER or GUEST_USER. Registered users receive personalized marketplace fields such as wishlist state, default address, and upcoming reminders where applicable. Guest users receive guest-safe marketplace data only. Guest users cannot access wishlist, cart, checkout, addresses, contacts, events, orders, payments, wallet, recurring payments, chats, reviews, or referrals. Guest response behavior includes isWishlisted=false, requiresAuthForWishlist=true, requiresAuthForCart=true, requiresAuthForCheckout=true, defaultAddress=null, upcomingReminder=null, and mode="GUEST" where applicable. Registered users receive personalized marketplace fields such as wishlist state, default address, and upcoming reminders where applicable. Guest users receive guest-safe marketplace data only. Guest users cannot access wishlist, cart, checkout, addresses, contacts, events, orders, payments, wallet, recurring payments, chats, reviews, or referrals. Guest gift details include isWishlisted=false and auth-required flags; SKU/exact stock stay hidden unless enabled in guest settings.
Parameters:

- `id` (path, required, string)

Response body:


```
{
  "success": true,
  "data": {
    "id": "gift_id",
    "name": "Luxury Perfume",
    "description": "Long-lasting premium fragrance.",
    "shortDescription": "Premium fragrance gift.",
    "price": 99.99,
    "originalPrice": 99.99,
    "currency": "PKR",
    "imageUrls": [
      "https://cdn.yourdomain.com/gift-images/perfume.png"
    ],
    "rating": 4.6,
    "reviewCount": 24,
    "isWishlisted": false,
    "requiresAuthForWishlist": true,
    "requiresAuthForCart": true,
    "requiresAuthForCheckout": true,
    "badges": [
      "AUTHENTIC"
    ],
    "category": {
      "id": "gift_category_id",
      "name": "Perfumes",
      "slug": "perfumes"
    },
    "provider": {
      "id": "provider_id",
      "businessName": "Dcodax Gifts",
      "rating": 4.6,
      "reviewCount": 24,
      "fulfillmentMethods": [
        "DELIVERY"
      ]
    },
    "variants": [
      {
        "id": "variant_id",
        "name": "50ml",
        "price": 129.99,
        "originalPrice": 159.99,
        "isPopular": true,
        "isDefault": true
      }
```

```
    ],
    "deliveryOptions": [
      "SAME_DAY",
      "NEXT_DAY",
      "SCHEDULED"
    ],
    "activeOffer": null
  },
  "message": "Gift details fetched successfully"
}
```

05 Customer - Wishlist

GET `/api/v1/customer/wishlist`
Summary: List wishlist gifts
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. REGISTERED_USER only. Returns customer-visible gifts: ACTIVE, published, not deleted, and owned by an approved active provider.
Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

POST `/api/v1/customer/wishlist/{giftId}`
Summary: Add gift to wishlist
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. REGISTERED_USER only. Gift must be customer-visible: ACTIVE, published, not deleted, and owned by an approved active provider. Duplicate wishlist entries are ignored.
Parameters:

- `giftId` (path, required, string)

Request payload(s):
payload

```
"<standard success envelope>"
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

DELETE `/api/v1/customer/wishlist/{giftId}`
Summary: Remove gift from wishlist
Allowed role/access: REGISTERED_USER

Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. REGISTERED_USER only. Removes only the current customer wishlist row.
Parameters:

- `giftId` (path, required, string)

Request payload(s):
payload

```
"<standard success envelope>"
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

05 Customer - Addresses

GET `/api/v1/customer/addresses`
Summary: List customer addresses
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. REGISTERED_USER only. Customers can only view their own non-deleted addresses.
Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

POST `/api/v1/customer/addresses`
Summary: Create customer address
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. REGISTERED_USER only. Maintains one default address per customer.
Request payload(s):
payload

```
{
  "label": "Home",
  "fullName": "Sarah Khan",
  "phone": "+923001234567",
  "line1": "House 12, Street 4, F-8/2",
  "line2": "Near Centaurus Mall",
  "city": "Islamabad",
  "state": "Islamabad Capital Territory",
  "country": "Pakistan",
  "postalCode": "44000",
  "latitude": 33.6844,
  "longitude": 73.0479,
  "deliveryInstructions": "Leave at reception.",
```

```
  "isDefault": true
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/customer/addresses/{id}`
Summary: Fetch customer address
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. REGISTERED_USER only. Address must belong to the current customer.
Parameters:

- `id` (path, required, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

PATCH `/api/v1/customer/addresses/{id}`
Summary: Update customer address
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. REGISTERED_USER only. Maintains one default address per customer.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload

```
{
  "label": "Home",
  "fullName": "Sarah Khan",
  "phone": "+923001234567",
  "line1": "House 12, Street 4, F-8/2",
  "line2": "Near Centaurus Mall",
  "city": "Islamabad",
  "state": "Islamabad Capital Territory",
  "country": "Pakistan",
  "postalCode": "44000",
  "latitude": 33.6844,
  "longitude": 73.0479,
  "deliveryInstructions": "Leave at reception.",
  "isDefault": true
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

DELETE `/api/v1/customer/addresses/{id}`
Summary: Delete customer address
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. REGISTERED_USER only. Permanently deletes the address and removes default status.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload

```
"<standard success envelope>"
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

PATCH `/api/v1/customer/addresses/{id}/default`
Summary: Set default customer address
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. REGISTERED_USER only. Clears default flag from all other customer addresses.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload

```
"<standard success envelope>"
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

05 Customer - Contacts

GET `/api/v1/customer/contacts`

Summary: List customer contacts
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. REGISTERED_USER only. Lists only contacts owned by the authenticated customer.
Parameters:

- `page` (query, optional, number)
- `limit` (query, optional, number)
- `search` (query, optional, string) Searches name, phone, email, and relationship.
- `relationship` (query, optional, string)
- `sortBy` (query, optional, string)
- `sortOrder` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

POST `/api/v1/customer/contacts`
Summary: Create customer contact
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. REGISTERED_USER only. Creates a personal gift contact owned by the authenticated customer. Requires at least one contact method: phone, email, or address.
Request payload(s):
create

```
{
  "name": "Mary Wilson",
  "relationship": "Mother",
  "phone": "+1234567890",
  "email": "mary@example.com",
  "address": "387 Merdina",
  "likes": "Glasses, makeup, dresses",
  "avatarUrl": "https://cdn.yourdomain.com/customer-contact-avatars/mary.png",
  "birthday": "1990-05-12",
  "notes": "Prefers elegant gifts."
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/customer/contacts/{id}`
Summary: Fetch customer contact
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. REGISTERED_USER only. Contact must belong to the authenticated customer.
Parameters:

- `id` (path, required, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

PATCH `/api/v1/customer/contacts/{id}`
Summary: Update customer contact
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. REGISTERED_USER only. Updates only contacts owned by the authenticated customer.
Parameters:

- `id` (path, required, string)

Request payload(s):
update

```
{
  "name": "Mary Wilson",
  "relationship": "Mother",
  "phone": "+1234567890",
  "email": "mary@example.com",
  "address": "387 Merdina",
  "likes": "Glasses, makeup, dresses",
  "avatarUrl": "https://cdn.yourdomain.com/customer-contact-avatars/mary.png",
  "birthday": "1990-05-12",
  "notes": "Prefers elegant gifts."
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

DELETE `/api/v1/customer/contacts/{id}`
Summary: Delete customer contact
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. REGISTERED_USER only. Permanently deletes only contacts owned by the authenticated customer.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload

```
"<standard success envelope>"
```

Response body:

```
{
  "success": true,
```

```
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

05 Customer - Events

GET `/api/v1/customer/events`
Summary: List customer events
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. REGISTERED_USER only. Lists only events owned by the authenticated customer.
Parameters:

- `page` (query, optional, number)
- `limit` (query, optional, number)
- `search` (query, optional, string)
- `eventType` (query, optional, string)
- `fromDate` (query, optional, string)
- `toDate` (query, optional, string)
- `recipientId` (query, optional, string)
- `sortBy` (query, optional, string)
- `sortOrder` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

POST `/api/v1/customer/events`
Summary: Create customer event
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. REGISTERED_USER only. recipientId must belong to the authenticated customer.
Request payload(s):
payload

```
{
  "eventType": "BIRTHDAY",
  "title": "Sarah's Birthday",
  "recipientId": "cmf0contactmary001",
  "eventDate": "2026-01-31T00:00:00.000Z",
  "reminderTiming": "ON_THE_DAY",
  "reminderFrequency": "ONE_TIME",
  "customAlertTime": "09:00",
  "channels": [
    "EMAIL",
    "SMS"
  ],
  "notes": "Send a birthday gift.",
  "isActive": true
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/customer/events/calendar`
Summary: Fetch monthly calendar events
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. REGISTERED_USER only. Returns marked dates and own events.
Parameters:

- `month` (query, required, number)
- `year` (query, required, number)
- `eventType` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/customer/events/upcoming`
Summary: Fetch upcoming customer events
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. REGISTERED_USER only. Defaults to 10 events within 30 days.
Parameters:

- `limit` (query, optional, number)
- `daysAhead` (query, optional, number)
- `eventType` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/customer/events/{id}/reminder-settings`
Summary: Fetch event reminder settings
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. REGISTERED_USER only. Event must belong to the authenticated customer.
Parameters:

- `id` (path, required, string)

Response body:

```
{
  "success": true,
```

```
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```


PATCH `/api/v1/customer/events/{id}/reminder-settings`
Summary: Update event reminder settings
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. REGISTERED_USER only. Event must belong to the authenticated customer.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload

```
{
  "reminderFrequency": "YEARLY",
  "reminderTiming": "ONE_DAY_BEFORE",
  "customAlertTime": "09:00",
  "channels": {
    "push": true,
    "email": true,
    "sms": false
  }
}
```

Response body:


```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```


GET `/api/v1/customer/events/{id}`
Summary: Fetch customer event details
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. REGISTERED_USER only. Event must belong to the authenticated customer.
Parameters:

- `id` (path, required, string)

Response body:


```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```


PATCH `/api/v1/customer/events/{id}`
Summary: Update customer event
Allowed role/access: REGISTERED_USER

Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. REGISTERED_USER only. Event and recipient contact must belong to the authenticated customer.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload

```
{
  "eventType": "ANNIVERSARY",
  "title": "Sarah's Anniversary",
  "recipientId": "cmf0contactmary001",
  "eventDate": "2026-01-31T00:00:00.000Z",
  "reminderTiming": "THREE_DAYS_BEFORE",
  "reminderFrequency": "YEARLY",
  "customAlertTime": "09:00",
  "channels": [
    "PUSH",
    "EMAIL"
  ],
  "notes": "Send flowers.",
  "isActive": true
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

DELETE `/api/v1/customer/events/{id}`
Summary: Delete customer event
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. REGISTERED_USER only. Permanently deletes only own event and cancels pending reminder jobs.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload

```
"<standard success envelope>"
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

05 Customer - Cart

GET `/api/v1/customer/cart`
Summary: Fetch active cart
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. REGISTERED_USER only. Totals are backend calculated from price snapshots.
Response body:


```
{
  "success": true,
  "data": {
    "id": "cart_id",
    "status": "ACTIVE",
    "items": [
      {
        "id": "cart_item_id",
        "giftId": "gift_id",
        "variantId": "variant_id",
        "name": "Luxury Perfume",
        "variantName": "50ml",
        "quantity": 1,
        "unitPrice": 129.99,
        "discountAmount": 20,
        "finalUnitPrice": 109.99,
        "lineTotal": 109.99,
        "imageUrl": "https://cdn.yourdomain.com/gift-images/perfume.png",
        "deliveryOption": "SAME_DAY",
        "recipient": {
          "contactId": "contact_id",
          "name": "Sarah Khan",
          "phone": "+923001234567",
          "addressId": "address_id"
        },
        "giftMessage": "Hope you love this special surprise!",
        "messageMediaUrls": [
          "https://cdn.yourdomain.com/gift-message-media/photo.png"
        ],
        "scheduledDeliveryAt": "2026-12-24T10:00:00.000Z"
      }
    ],
    "summary": {
      "subtotal": 129.99,
      "discountTotal": 20,
      "deliveryFee": 0,
      "tax": 0,
      "total": 109.99,
      "currency": "PKR"
    }
  },
  "message": "Cart fetched successfully"
}
```


DELETE `/api/v1/customer/cart`
Summary: Clear active cart
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. REGISTERED_USER only. Removes all items from active cart.
Request payload(s):
payload

```
"<standard success envelope>"
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

POST `/api/v1/customer/cart/items`
Summary: Add item to cart
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. REGISTERED_USER only. Backend calculates unit price, active offer discount, final price, subtotal, delivery fee placeholder, and total.
Request payload(s):
sendGift

```
{
  "giftId": "cmf0giftroses001",
  "variantId": "cmf0variant50ml001",
  "quantity": 1,
  "deliveryOption": "SAME_DAY",
  "recipientContactId": "cmf0contactmary001",
  "recipientName": "Sarah Khan",
  "recipientPhone": "+923001234567",
  "recipientAddressId": "cmf0addresshome001",
  "eventId": "cmf0eventbirthday001",
  "giftMessage": "Hope you love this special surprise!",
  "messageMediaUrls": [
    "https://cdn.yourdomain.com/gift-message-media/photo.png"
  ],
  "scheduledDeliveryAt": "2026-12-24T10:00:00.000Z"
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

PATCH `/api/v1/customer/cart/items/{id}`
Summary: Update cart item
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. REGISTERED_USER only. Validates ownership through the active customer cart.
Parameters:

- `id` (path, required, string)

Request payload(s):
updateSelection

```
{
  "variantId": "cmf0variant100ml001",
  "quantity": 2,
  "deliveryOption": "SCHEDULED",
  "recipientContactId": "cmf0contactmary001",
  "recipientName": "Sarah Khan",
  "recipientPhone": "+923001234567",
  "recipientAddressId": "cmf0addresshome001",
  "eventId": "cmf0eventbirthday001",
  "giftMessage": "Updated gift note.",
  "messageMediaUrls": [
    "https://cdn.yourdomain.com/gift-message-media/video.mp4"
  ],
  "scheduledDeliveryAt": "2026-12-25T10:00:00.000Z"
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

DELETE `/api/v1/customer/cart/items/{id}`
Summary: Delete cart item
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. REGISTERED_USER only. Deletes only items in the current customer active cart.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload

```
"<standard success envelope>"
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

05 Customer - Orders

GET `/api/v1/customer/orders`
Summary: List customer orders
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. REGISTERED_USER only. Returns orders owned by the current customer.
Parameters:

- `page` (query, optional, number)

- `limit` (query, optional, number)
- `type` (query, optional, string)
- `status` (query, optional, string)
- `fromDate` (query, optional, string)
- `toDate` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

POST `/api/v1/customer/orders`
Summary: Create order from active cart
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. REGISTERED_USER only. Prices are backend-calculated from cart/payment snapshots. STRIPE_CARD requires a successful owned paymentId; COD stays pending; PLACEHOLDER is for development only. Multiple providers are split into provider sub-orders.
Request payload(s):
stripeCard

```
{
  "cartId": "cart_id",
  "paymentId": "payment_id",
  "deliveryAddressId": "address_id",
  "paymentMethod": "STRIPE_CARD"
}
```

cod

```
{
  "cartId": "cart_id",
  "deliveryAddressId": "address_id",
  "paymentMethod": "COD"
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/customer/orders/{id}`
Summary: Fetch customer order
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. REGISTERED_USER only. Order must belong to the current customer.
Parameters:

- `id` (path, required, string)

Response body:

```
{
  "success": true,
  "data": {
    "id": "order_id",
    "orderNumber": "ORD-1760000000000",
    "status": "CONFIRMED",
    "paymentStatus": "SUCCEEDED",
    "paymentMethod": "STRIPE_CARD",
    "recipient": {
      "name": "Sarah Khan",
      "email": null,
      "phone": "+923001234567",
      "avatarUrl": null
    },
    "deliveryDate": "2026-12-24T10:00:00.000Z",
    "occasion": null,
    "giftMessage": "Hope you love this special surprise!",
    "items": [
      {
        "giftId": "gift_id",
        "name": "Luxury Perfume",
        "variantName": "50ml",
        "quantity": 1,
        "imageUrl": "https://cdn.yourdomain.com/gift-images/perfume.png",
        "total": 109.99
      }
    ],
    "summary": {
      "subtotal": 129.99,
      "discountTotal": 20,
      "deliveryFee": 0,
      "tax": 0,
      "total": 109.99,
      "currency": "PKR"
    }
  },
  "message": "Order fetched successfully."
}
```


05 Customer - Reviews

POST `/api/v1/customer/orders/{id}/reviews`
Summary: Submit provider review for an order
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. REGISTERED_USER only. Uses shared Review records consumed by provider reviews and admin review management.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload


```
{
  "providerId": "provider_id",
  "rating": 5,
  "comment": "Great service and fast delivery. The package arrived in perfect condition."
}
```

Response body:


```
{
  "success": true,
  "data": {
    "id": "review_id",
    "rating": 5,
    "comment": "Great service and fast delivery.",
    "status": "PUBLISHED",
    "providerId": "provider_id",
    "orderId": "order_id",
    "createdAt": "2026-05-11T10:00:00.000Z"
  },
  "message": "Review submitted successfully."
}
```


GET `/api/v1/customer/reviews`
Summary: List own provider reviews
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. REGISTERED_USER only. Customer sees only their own non-deleted reviews.
Parameters:

- `page` (query, optional, number)
- `limit` (query, optional, number)
- `rating` (query, optional, number)
- `providerId` (query, optional, string)
- `status` (query, optional, string)

Response body:


```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```


GET `/api/v1/customer/reviews/{id}`
Summary: Fetch Customer Reviews details
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account.
Parameters:

- `id` (path, required, string)

Response body:


```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```


PATCH `/api/v1/customer/reviews/{id}`
Summary: Update own review
Allowed role/access: REGISTERED_USER

Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. REGISTERED_USER only. Cannot update deleted/removed reviews; updated content is re-run through deterministic moderation.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload

```
{
  "rating": 4,
  "comment": "Updated review text."
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

DELETE `/api/v1/customer/reviews/{id}`
Summary: Delete own review
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. REGISTERED_USER only. Permanently deletes the customer review record.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload

```
"<standard success envelope>"
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

05 Customer - Provider Reports

GET `/api/v1/customer/provider-report-reasons`
Summary: Fetch provider report reasons
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. REGISTERED_USER only. Declared before /customer/provider-reports/:id.
Response body:

```
{
  "success": true,
```

```
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

POST `/api/v1/customer/providers/{providerId}/reports`
Summary: Report provider
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. REGISTERED_USER only. Customer must have an order, chat, or review relationship with provider. Duplicate active provider/order/reason reports are blocked.
Parameters:

- `providerId` (path, required, string)

Request payload(s):
payload

```
{
  "reason": "POOR_SERVICE_QUALITY",
  "details": "The provider did not respond and the order was delayed.",
  "orderId": "order_id",
  "evidenceUrls": []
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/customer/provider-reports`
Summary: List Customer Provider Reports
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account.
Parameters:

- `page` (query, optional, number)
- `limit` (query, optional, number)
- `status` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/customer/provider-reports/{id}`
Summary: Fetch Customer Provider Reports details
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account.
Parameters:

- `id` (path, required, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

05 Customer - Recurring Payments

GET `/api/v1/customer/recurring-payments`
Summary: List own recurring payments
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. REGISTERED_USER only. Returns recurring money/gift payment subscriptions owned by the logged-in customer.
Parameters:

- `page` (query, optional, number)
- `limit` (query, optional, number)
- `search` (query, optional, string)
- `status` (query, optional, string)
- `frequency` (query, optional, string)
- `recipientContactId` (query, optional, string)
- `sortBy` (query, optional, string)
- `sortOrder` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": [
    {
      "id": "recurring_payment_id",
      "title": "Sarah's Birthday",
      "recipient": {
        "id": "contact_id",
        "name": "Sarah Johnson",
        "email": "sarah.j@example.com",
        "avatarUrl": "https://cdn.yourdomain.com/customer-contact-avatars/sarah.png"
      },
      "amount": 50,
      "currency": "PKR",
      "frequency": "MONTHLY",
      "nextBillingAt": "2026-03-15T09:00:00.000Z",
      "status": "ACTIVE",
      "message": "Monthly flowers",
      "createdAt": "2026-05-09T10:00:00.000Z"
    }
  ],
  "meta": {
    "page": 1,
    "limit": 10,
    "total": 1,
    "totalPages": 1
  },
  "message": "Recurring payments fetched successfully."
}
```

POST `/api/v1/customer/recurring-payments`
Summary: Create recurring payment
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. REGISTERED_USER only. Recipient contact and saved Stripe payment method must both belong to the logged-in customer. Stripe card recurring payments require a saved payment method.
Request payload(s):
weeklyStripe

```
{
  "amount": 100,
  "currency": "PKR",
  "frequency": "WEEKLY",
  "schedule": {
    "dayOfWeek": "MONDAY",
    "dayOfMonth": null,
    "monthOfYear": null,
    "time": "09:00",
    "timezone": "Asia/Karachi"
  },
  "recipientContactId": "contact_id",
  "message": "Hope you love this special surprise!",
  "messageMediaUrls": [
    "https://cdn.yourdomain.com/gift-message-media/photo.png"
  ],
  "paymentMethod": "STRIPE_CARD",
  "stripePaymentMethodId": "pm_xxx",
  "startDate": "2026-05-10T00:00:00.000Z",
  "endDate": null,
  "autoSend": true
}
```

Response body:

```
{
  "success": true,
  "data": {
    "id": "recurring_payment_id",
    "amount": 100,
    "currency": "PKR",
    "frequency": "WEEKLY",
    "nextBillingAt": "2026-05-12T09:00:00.000Z",
    "status": "ACTIVE"
  },
  "message": "Recurring payment created successfully."
}
```

GET `/api/v1/customer/recurring-payments/summary`
Summary: Fetch recurring payment summary counts
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. Must stay before /customer/recurring-payments/:id route.
Response body:

```
{
  "success": true,
  "data": {
    "total": 5,
    "active": 3,
    "paused": 1,
```

```
    "cancelled": 1,
    "failed": 0
  },
  "message": "Recurring payment summary fetched successfully."
}
```


GET `/api/v1/customer/recurring-payments/{id}`
Summary: Fetch own recurring payment details
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. REGISTERED_USER only. Customer cannot access another user's recurring payment.
Parameters:

- `id` (path, required, string)

Response body:


```
{
  "success": true,
  "data": {
    "id": "recurring_payment_id",
    "title": "Monthly Flowers",
    "recipient": {
      "id": "contact_id",
      "name": "Sarah Johnson",
      "email": "sarah.j@example.com",
      "avatarUrl": "https://cdn.yourdomain.com/customer-contact-avatars/sarah.png"
    },
    "amount": 50,
    "currency": "PKR",
    "frequency": "MONTHLY",
    "nextBillingAt": "2026-03-15T09:00:00.000Z",
    "status": "ACTIVE",
    "message": "Fresh seasonal bouquet delivered to her doorstep every month",
    "messageMediaUrls": [],
    "paymentMethod": {
      "type": "STRIPE_CARD",
      "brand": "visa",
      "last4": "4242",
      "expiryMonth": 12,
      "expiryYear": 2026
    },
    "schedule": {
      "frequency": "MONTHLY",
      "dayOfMonth": 15,
      "time": "09:00",
      "timezone": "Asia/Karachi"
    },
    "createdAt": "2026-05-09T10:00:00.000Z"
  },
  "message": "Recurring payment fetched successfully."
}
```


PATCH `/api/v1/customer/recurring-payments/{id}`
Summary: Update own recurring payment
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. Cannot edit CANCELLED recurring payments. Changes apply from the next billing cycle and nextBillingAt is recalculated.

Parameters:

- `id` (path, required, string)

Request payload(s):
monthly

```
{
  "amount": 50,
  "frequency": "MONTHLY",
  "schedule": {
    "dayOfMonth": 15,
    "time": "09:00",
    "timezone": "Asia/Karachi"
  },
  "message": "Fresh flowers every month.",
  "messageMediaUrls": [],
  "stripePaymentMethodId": "pm_xxx"
}
```

Response body:

```
{
  "success": true,
  "data": {
    "id": "recurring_payment_id",
    "status": "ACTIVE",
    "nextBillingAt": "2026-03-15T09:00:00.000Z"
  },
  "message": "Recurring payment updated successfully. Changes will apply from the next
billing cycle."
}
```

POST `/api/v1/customer/recurring-payments/{id}/action`
Summary: Run own recurring payment action
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. PAUSE is allowed only from ACTIVE and moves the recurring payment to PAUSED. RESUME is allowed only from PAUSED and moves it back to ACTIVE. CANCEL is allowed from ACTIVE or PAUSED, supports cancelAtPeriodEnd, and preserves billing history.
Parameters:

- `id` (path, required, string)

Request payload(s):
pause

```
{
  "action": "PAUSE",
  "reason": "USER_REQUEST",
  "comment": "User paused payment temporarily."
}
```

resume

```
{
  "action": "RESUME",
  "reason": "USER_REQUEST",
  "comment": "User resumed recurring payment."
}
```

cancelImmediate

```
{
  "action": "CANCEL",
  "reason": "USER_REQUEST",
  "cancelAtPeriodEnd": false,
  "comment": "User no longer needs this recurring payment."
}
```

cancelAtPeriodEnd

```
{
  "action": "CANCEL",
  "reason": "USER_REQUEST",
  "cancelAtPeriodEnd": true,
  "comment": "User wants to finish the current billing cycle."
}
```

Response body:

```
{
  "success": true,
  "data": {
    "id": "recurring_payment_id",
    "status": "PAUSED",
    "action": "PAUSE"
  },
  "message": "Recurring payment paused successfully."
}
```

GET `/api/v1/customer/recurring-payments/{id}/history`
Summary: List own recurring payment billing history
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account.
Parameters:

- `id` (path, required, string)
- `page` (query, optional, number)
- `limit` (query, optional, number)
- `status` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": [
    {
      "id": "history_id",
      "paymentId": "payment_id",
      "amount": 50,
      "currency": "PKR",
      "status": "SUCCESS",
      "billingDate": "2026-02-15T09:00:00.000Z",
      "transactionId": "GFT-8829-XPL",
      "failureReason": null
    }
  ],
  "meta": {
    "page": 1,
    "limit": 10,
```

```
    "total": 1,
    "totalPages": 1
  },
  "message": "Recurring payment history fetched successfully."
}
```


05 Customer - Transactions

GET `/api/v1/customer/transactions`
Summary: List own customer transactions
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. REGISTERED_USER only. Normalizes backend Payment, Order, MoneyGift, and RecurringPayment occurrence records owned by the logged-in customer.
Parameters:

- `page` (query, optional, number)
- `limit` (query, optional, number)
- `search` (query, optional, string)
- `fromDate` (query, optional, string)
- `toDate` (query, optional, string)
- `type` (query, optional, string)
- `status` (query, optional, string)
- `paymentMethod` (query, optional, string)
- `minAmount` (query, optional, number)
- `maxAmount` (query, optional, number)
- `sortBy` (query, optional, string)
- `sortOrder` (query, optional, string)

Response body:


```
{
  "success": true,
  "data": [
    {
      "id": "payment_id",
      "transactionId": "TXN-2026-001234",
      "title": "Monthly Flowers",
      "description": "Recurring payment",
      "recipient": {
        "id": "contact_id",
        "name": "Sarah Johnson",
        "avatarUrl": "https://cdn.yourdomain.com/customer-contact-avatars/sarah.png"
      },
      "amount": 50,
      "currency": "PKR",
      "type": "RECURRING_PAYMENT",
      "status": "SUCCESS",
      "paymentMethod": "STRIPE_CARD",
      "createdAt": "2026-03-01T14:34:00.000Z"
    }
  ],
  "meta": {
    "page": 1,
    "limit": 10,
    "total": 1,
    "totalPages": 1
  },
```

```
  "message": "Transactions fetched successfully."
}
```


GET `/api/v1/customer/transactions/summary`
Summary: Fetch own transaction summary
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. Defaults to current month when no date range is provided. Uses backend-calculated payment records only.
Parameters:

- `fromDate` (query, optional, string)
- `toDate` (query, optional, string)

Response body:


```
{
  "success": true,
  "data": {
    "totalSpentThisMonth": 255,
    "currency": "PKR",
    "successfulCount": 9,
    "failedCount": 1,
    "pendingCount": 0,
    "refundedCount": 0
  },
  "message": "Transaction summary fetched successfully."
}
```


GET `/api/v1/customer/transactions/export`
Summary: Export own transactions
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. CSV is supported and returned as a file. Export is scoped to the logged-in customer only.
Parameters:

- `page` (query, optional, number)
- `limit` (query, optional, number)
- `search` (query, optional, string)
- `fromDate` (query, optional, string)
- `toDate` (query, optional, string)
- `type` (query, optional, string)
- `status` (query, optional, string)
- `paymentMethod` (query, optional, string)
- `minAmount` (query, optional, number)
- `maxAmount` (query, optional, number)
- `sortBy` (query, optional, string)
- `sortOrder` (query, optional, string)
- `format` (query, optional, string)

Response body:


```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/customer/transactions/{id}`
Summary: Fetch own transaction details
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. Includes order, money gift, recurring payment, and payment gateway references when available. Billing address returns null until available.
Parameters:

- `id` (path, required, string)

Response body:


```
{
  "success": true,
  "data": {
    "id": "payment_id",
    "transactionId": "TXN-2026-001234-ABC-XYZ",
    "status": "SUCCESS",
    "amount": 50,
    "currency": "PKR",
    "createdAt": "2026-03-01T14:34:00.000Z",
    "type": "RECURRING_PAYMENT",
    "giftInformation": {
      "giftName": "Monthly Flowers Subscription",
      "deliveryType": "Money",
      "recipient": {
        "id": "contact_id",
        "name": "Sarah Johnson",
        "avatarUrl": "https://cdn.yourdomain.com/customer-contact-avatars/sarah.png"
      },
      "orderReference": null,
      "recurringPaymentId": "recurring_payment_id"
    },
    "paymentInformation": {
      "paymentMethod": "Stripe card",
      "gatewayReference": "pi_3MmlLrLkdIwHu7ix0fhBHWqt",
      "billingAddress": null
    }
  },
  "message": "Transaction details fetched successfully."
}
```


GET `/api/v1/customer/transactions/{id}/receipt`
Summary: Download own transaction receipt
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. Receipt is generated only for the transaction owner and never exposes Stripe secret data.
Parameters:

- `id` (path, required, string)

Response body:


```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

05 Customer - Referrals & Rewards

GET `/api/v1/customer/referrals/summary`
Summary: Fetch own referral reward summary
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. REGISTERED_USER only. Customers can view only their own referral progress and ledger-derived balances.
Response body:


```
{
  "success": true,
  "data": {
    "invitedFriends": 3,
    "successfulReferrals": 2,
    "rewardsEarned": 20,
    "availableCredit": 20,
    "currency": "USD",
    "progress": {
      "totalInvited": 3,
      "joined": 2,
      "pending": 1
    }
  },
  "message": "Referral summary fetched successfully."
}
```


GET `/api/v1/customer/referrals/link`
Summary: Fetch own referral link
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. Generates a unique customer referral code when missing. The link never exposes internal user IDs.
Response body:


```
{
  "success": true,
  "data": {
    "referralCode": "SARAH-M",
    "referralLink": "https://giftapp.com/share/sarah-m",
    "shareTitle": "Invite Friends, Earn Rewards",
    "shareMessage": "Join Gift App with my referral link and we both earn rewards after
your first gift purchase.",
    "rewardText": "Get $10 credit after your friend's first gift purchase."
  },
  "message": "Referral link fetched successfully."
}
```


GET `/api/v1/customer/referrals/history`
Summary: List own referral history
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. REGISTERED_USER only. History is scoped to referrals created by the logged-in customer.
Parameters:

- `page` (query, optional, number)
- `limit` (query, optional, number)
- `status` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

POST `/api/v1/customer/referrals/redeem`
Summary: Redeem own available reward credit
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. Creates a REDEEMED reward ledger entry. Redemption cannot exceed ledger-derived available credit.
Request payload(s):
payload

```
{
  "amount": 20,
  "redeemTo": "WALLET"
}
```

Response body:

```
{
  "success": true,
  "data": {
    "redeemedAmount": 20,
    "currency": "USD",
    "walletBalance": 20
  },
  "message": "Reward redeemed successfully."
}
```

GET `/api/v1/customer/rewards/balance`
Summary: Fetch own reward balance
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. Balance is calculated from RewardLedger entries, not a mutable user balance field.
Response body:

```
{
  "success": true,
  "data": {
    "availableCredit": 20,
    "lifetimeEarned": 20,
    "lifetimeRedeemed": 0,
    "currency": "USD"
  },
  "message": "Reward balance fetched successfully."
}
```

GET `/api/v1/customer/rewards/ledger`
Summary: List own reward ledger
Allowed role/access: REGISTERED_USER

Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. REGISTERED_USER only. Returns ledger entries owned by the logged-in customer.
Parameters:

- `page` (query, optional, number)
- `limit` (query, optional, number)
- `type` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/customer/referrals/terms`
Summary: Fetch referral terms
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. Returns config/env based customer referral terms for the mobile app.
Response body:

```
{
  "success": true,
  "data": {
    "title": "Referral Terms",
    "rewardAmount": 10,
    "currency": "USD",
    "qualificationRule": "Reward is credited after your referred friend completes their first gift purchase.",
    "terms": [
      "Referral rewards are available only for registered users.",
      "Reward is credited after the referred user's first successful purchase.",
      "Cancelled or refunded orders may revoke reward eligibility.",
      "Referral abuse may result in reward cancellation."
    ]
  },
  "message": "Referral terms fetched successfully."
}
```

05 Customer - Subscriptions

GET `/api/v1/customer/subscription/plans`
Summary: List public active subscription plans
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. REGISTERED_USER only. Uses admin-created active/public Subscription Plans. Customer cannot create/update/delete plans.
Parameters:

- `billingCycle` (query, optional, string)

Response body:

```
{
  "success": true,
```

```
  "data": [
    {
      "id": "plan_premium",
      "name": "Premium",
      "monthlyPrice": 4.99,
      "yearlyPrice": 39.99,
      "currency": "USD",
      "isPopular": true,
      "yearlySavingsPercent": 30,
      "features": [
        {
          "key": "ai_gift_recommendations",
          "label": "Ai Gift Recommendations",
          "enabled": true
        }
      ],
      "limits": {
        "unlimitedCredits": true
      }
    }
  ],
  "message": "Subscription plans fetched successfully."
}
```


GET `/api/v1/customer/subscription/current`
Summary: Fetch own current subscription
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. REGISTERED_USER only. Returns FREE state when no active premium subscription exists.
Response body:


```
{
  "success": true,
  "data": {
    "tier": "PREMIUM",
    "subscription": {
      "id": "sub_id",
      "planId": "plan_premium",
      "planName": "Premium",
      "billingCycle": "MONTHLY",
      "status": "ACTIVE",
      "isPremium": true,
      "cancelAtPeriodEnd": false,
      "currentPeriodStart": "2026-04-01T00:00:00.000Z",
      "currentPeriodEnd": "2026-05-01T00:00:00.000Z"
    }
  },
  "message": "Current subscription fetched successfully."
}
```


POST `/api/v1/customer/subscription/checkout`
Summary: Create Stripe subscription checkout
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. REGISTERED_USER only. Backend calculates price from admin-created SubscriptionPlan and optional coupon. Uses Stripe subscription flow with payment_behavior=default_incomplete.
Request payload(s):
payload

```
{
  "planId": "plan_premium",
  "billingCycle": "MONTHLY",
  "paymentMethodId": "pm_123",
  "couponCode": "SUMMER25"
}
```

Response body:

```
{
  "success": true,
  "data": {
    "subscriptionId": "sub_local_id",
    "stripeSubscriptionId": "sub_stripe_123",
    "clientSecret": "seti_client_secret",
    "status": "INCOMPLETE",
    "amountDue": 4.99,
    "currency": "USD"
  },
  "message": "Subscription checkout created successfully."
}
```

POST `/api/v1/customer/subscription/confirm`
Summary: Confirm Stripe subscription activation
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. REGISTERED_USER only. Fetches Stripe subscription server-side and activates local entitlement when active/trialing.
Request payload(s):
payload

```
{
  "customerSubscriptionId": "<string>",
  "stripeSubscriptionId": "<string>",
  "idempotencyKey": "sub_confirm_2026_001"
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

POST `/api/v1/customer/subscription/action`
Summary: Run own subscription lifecycle action
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. REGISTERED_USER only. CANCEL cancels the current customer subscription and supports cancelAtPeriodEnd. REACTIVATE only works when the own subscription is scheduled for cancellation.
Request payload(s):
cancel

```
{
  "action": "CANCEL",
```

```
  "cancelAtPeriodEnd": true,
  "reason": "USER_REQUEST",
  "comment": "User requested cancellation."
}
```

reactivate

```
{
  "action": "REACTIVATE",
  "reason": "USER_REQUEST",
  "comment": "User changed their mind."
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/customer/subscription/invoices`
Summary: List own subscription invoices
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. REGISTERED_USER only. Returns invoices synced from Stripe subscription webhooks.
Parameters:

- `page` (query, optional, number)
- `limit` (query, optional, number)
- `status` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": [
    {
      "id": "invoice_id",
      "stripeInvoiceId": "in_123",
      "amountDue": 4.99,
      "amountPaid": 4.99,
      "currency": "USD",
      "status": "PAID",
      "invoicePdfUrl": "https://cdn.yourdomain.com/invoices/invoice.pdf",
      "periodStart": "2026-04-01T00:00:00.000Z",
      "periodEnd": "2026-05-01T00:00:00.000Z",
      "createdAt": "2026-04-01T00:00:00.000Z"
    }
  ],
  "meta": {
    "page": 1,
    "limit": 10,
    "total": 1,
    "totalPages": 1
  },
  "message": "Subscription invoices fetched successfully."
}
```

GET `/api/v1/customer/subscription/invoices/{id}`
Summary: Fetch own subscription invoice details
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. REGISTERED_USER only.
Parameters:

- `id` (path, required, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

POST `/api/v1/customer/subscription/apply-coupon`
Summary: Preview subscription coupon
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. REGISTERED_USER only. Validates coupon against active coupon rules and plan restrictions; frontend discount amounts are ignored.
Request payload(s):
payload

```
{
  "planId": "<string>",
  "billingCycle": "MONTHLY",
  "couponCode": "<string>"
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

05 Customer - Wallet

GET `/api/v1/customer/wallet`
Summary: Fetch own wallet
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. REGISTERED_USER only. Wallet is lazily created and balances are backed by wallet ledger entries.
Response body:

```
{
  "success": true,
  "data": {
    "totalBalance": 1240.5,
    "giftCredits": 350,
    "cashBalance": 890.5,
```

```
    "currency": "USD",
    "defaultPaymentMethod": {
      "id": "pm_xxx",
      "type": "CARD",
      "brand": "visa",
      "last4": "4242",
      "expiryMonth": 9,
      "expiryYear": 2025,
      "isDefault": true
    },
    "defaultBankAccount": {
      "id": "bank_account_id",
      "bankName": "Chase Bank",
      "last4": "8821",
      "isDefault": false
    }
  },
  "message": "Wallet fetched successfully."
}
```


POST `/api/v1/customer/wallet/add-funds`
Summary: Create wallet top-up payment
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. Uses Stripe PaymentIntent. Wallet is credited only after successful server-side confirmation/webhook.
Request payload(s):
payload


```
{
  "amount": 100,
  "currency": "USD",
  "paymentMethod": "STRIPE_CARD",
  "stripePaymentMethodId": "pm_xxx",
  "idempotencyKey": "wallet_topup_2026_001"
}
```


Response body:


```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```


GET `/api/v1/customer/wallet/history`
Summary: List own wallet history
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. Positive amounts are credits, negative amounts are debits. Results are scoped to the logged-in customer.
Parameters:

- `page` (query, optional, number)
- `limit` (query, optional, number)
- `type` (query, optional, string)
- `status` (query, optional, string)
- `fromDate` (query, optional, string)
- `toDate` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```


05 Customer - Payment Methods

GET `/api/v1/customer/bank-accounts`
Summary: List own bank accounts
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account.
Response body:


```
{
  "success": true,
  "data": [
    {
      "id": "bank_account_id",
      "accountHolderName": "John Smith",
      "bankName": "Chase Bank",
      "last4": "8821",
      "maskedAccount": "**** 8821",
      "isDefault": false
    }
  ],
  "message": "Bank accounts fetched successfully."
}
```


POST `/api/v1/customer/bank-accounts`
Summary: Link placeholder bank account
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. Stores only masked display data. Full IBAN/account number is never returned.
Request payload(s):
payload


```
{
  "accountHolderName": "John Smith",
  "bankName": "Chase Bank",
  "ibanOrAccountNumber": "1234567890",
  "isDefault": false
}
```


Response body:


```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```


PATCH `/api/v1/customer/bank-accounts/{id}/default`

Summary: Set own default bank account
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload

```
"<standard success envelope>"
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

DELETE `/api/v1/customer/bank-accounts/{id}`
Summary: Delete own bank account
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload

```
"<standard success envelope>"
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/customer/payment-methods`
Summary: List supported customer payment methods
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account.
Response body:

```
{
  "success": true,
  "data": [
    {
      "key": "STRIPE_CARD",
      "label": "Credit/Debit Card",
      "enabled": true
    },
    {
      "key": "BANK_TRANSFER",
      "label": "Bank Payment",
```

```
      "enabled": true
    },
    {
      "key": "E_WALLET",
      "label": "E-Wallet",
      "enabled": false
    }
  ],
  "message": "Payment methods fetched successfully."
}
```

POST `/api/v1/customer/payment-methods/setup-intent`
Summary: Create Stripe SetupIntent for saving card
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. Frontend confirms card with Stripe SDK. Backend never accepts raw card number or CVV.
Request payload(s):
payload

```
"<standard success envelope>"
```

Response body:

```
{
  "success": true,
  "data": {
    "setupIntentId": "seti_xxx",
    "clientSecret": "seti_xxx_secret_xxx",
    "publishableKey": "pk_test_xxx"
  },
  "message": "Setup intent created successfully."
}
```

GET `/api/v1/customer/payment-methods/saved`
Summary: List own saved payment methods
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. Returns masked Stripe card metadata only.
Response body:

```
{
  "success": true,
  "data": [
    {
      "id": "pm_xxx",
      "type": "CARD",
      "brand": "visa",
      "last4": "4242",
      "expiryMonth": 9,
      "expiryYear": 2025,
      "isDefault": true
    }
  ],
  "message": "Saved payment methods fetched successfully."
}
```

PATCH `/api/v1/customer/payment-methods/{id}/default`
Summary: Set own default payment method
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload

```
"<standard success envelope>"
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

DELETE `/api/v1/customer/payment-methods/{id}`
Summary: Delete own saved payment method
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. Rejects deletion when the method is used by an active recurring payment.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload

```
"<standard success envelope>"
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

06 Payments

POST `/api/v1/customer/payments/create-intent`
Summary: Create payment intent from active cart
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. REGISTERED_USER only. Amount is calculated from backend cart totals; frontend amount is never accepted.
Request payload(s):
stripe

```
{
  "cartId": "cmf0cartactive001",
```

```
  "paymentMethod": "STRIPE_CARD"
}
```

cod

```
{
  "cartId": "cmf0cartactive001",
  "paymentMethod": "COD"
}
```

Response body:

```
{
  "success": true,
  "data": {
    "paymentId": "payment_id",
    "stripePaymentIntentId": "pi_xxx",
    "clientSecret": "pi_xxx_secret_xxx",
    "publishableKey": "pk_live_or_test",
    "amount": 10999,
    "currency": "PKR"
  },
  "message": "Payment intent created successfully."
}
```

POST `/api/v1/customer/payments/confirm`
Summary: Confirm Stripe payment
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. REGISTERED_USER only. Retrieves Stripe PaymentIntent server-side before updating local payment status.
Request payload(s):
payload

```
{
  "paymentId": "cmf0payment001",
  "stripePaymentIntentId": "pi_3Pxxxxxxxxxxxxxxx",
  "idempotencyKey": "confirm_payment_2026_001"
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/customer/payments/{id}`
Summary: Fetch own payment details
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account.
Parameters:

- `id` (path, required, string)

Response body:

```
{
  "success": true,
  "data": {
    "paymentId": "payment_id",
    "provider": "STRIPE",
    "stripePaymentIntentId": "pi_xxx",
    "amount": 109.99,
    "currency": "PKR",
    "status": "SUCCEEDED",
    "paymentMethod": "STRIPE_CARD",
    "failureReason": null
  },
  "message": "Payment fetched successfully."
}
```


POST `/api/v1/payments/stripe/webhook`
Summary: Stripe webhook endpoint
Allowed role/access: PUBLIC
Notes: Access: PUBLIC. Verifies Stripe-Signature using the configured webhook secret before processing events.
Request payload(s):
payload


```
"<standard success envelope>"
```


Response body:


```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```


GET `/api/v1/customer/money-gifts`
Summary: List own money gifts
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account.
Response body:


```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```


POST `/api/v1/customer/money-gifts`
Summary: Send payment as gift
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account. Creates a customer-to-recipient money gift record and Stripe PaymentIntent for STRIPE_CARD.
Request payload(s):
create


```
{
  "amount": 100,
```

```
  "currency": "PKR",
  "recipientContactId": "cmf0contactmary001",
  "message": "Hope this helps. Enjoy your day!",
  "messageMediaUrls": [],
  "deliveryDate": "2026-12-24T00:00:00.000Z",
  "repeatAnnually": false,
  "paymentMethod": "STRIPE_CARD"
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/customer/money-gifts/{id}`
Summary: Fetch own money gift details
Allowed role/access: REGISTERED_USER
Notes: Access: REGISTERED_USER. REGISTERED_USER only. Endpoint is scoped to the authenticated customer account.
Parameters:

- `id` (path, required, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

06 Notifications

GET `/api/v1/notifications`
Summary: List notifications
Allowed role/access: SUPER_ADMIN, ADMIN, PROVIDER, or REGISTERED_USER
Notes: Access: SUPER_ADMIN, ADMIN, PROVIDER, or REGISTERED_USER. Access is scoped to the authenticated account. JWT auth. Returns only notifications belonging to the logged-in account. Supports All, Unread, Birthdays, Deliveries, and New Contacts filters. No group gift notifications are supported.
Parameters:

- `page` (query, optional, number)
- `limit` (query, optional, number)
- `filter` (query, optional, string)
- `type` (query, optional, string)
- `isRead` (query, optional, boolean)
- `groupByDate` (query, optional, boolean)
- `sortOrder` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": [
    {
```

```
      "id": "notification_id",
      "title": "Payment successful",
      "message": "Your payment was completed successfully.",
      "type": "PAYMENT_SUCCEEDED",
      "isRead": false,
      "metadata": {
        "paymentId": "payment_id"
      },
      "createdAt": "2026-05-09T10:00:00.000Z"
    }
  ],
  "meta": {
    "page": 1,
    "limit": 10,
    "total": 1,
    "totalPages": 1
  },
  "message": "Notifications fetched successfully"
}
```


GET `/api/v1/notifications/summary`
Summary: Fetch notification summary
Allowed role/access: SUPER_ADMIN, ADMIN, PROVIDER, or REGISTERED_USER
Notes: Access: SUPER_ADMIN, ADMIN, PROVIDER, or REGISTERED_USER. Access is scoped to the authenticated account. JWT auth. Counts only notifications belonging to the logged-in account.
Response body:


```
{
  "success": true,
  "data": {
    "total": 12,
    "unread": 3,
    "byType": {
      "PAYMENT_SUCCEEDED": 2,
      "RECURRING_PAYMENT_CHARGE_FAILED": 1,
      "BROADCAST": 9
    }
  },
  "message": "Notification summary fetched successfully"
}
```


GET `/api/v1/notifications/preferences`
Summary: Fetch notification preferences
Allowed role/access: SUPER_ADMIN, ADMIN, PROVIDER, or REGISTERED_USER
Notes: Access: SUPER_ADMIN, ADMIN, PROVIDER, or REGISTERED_USER. Access is scoped to the authenticated account. JWT auth. Preferences belong only to the logged-in account.
Response body:


```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```


PATCH `/api/v1/notifications/preferences`
Summary: Update notification preferences

Allowed role/access: SUPER_ADMIN, ADMIN, PROVIDER, or REGISTERED_USER
Notes: Access: SUPER_ADMIN, ADMIN, PROVIDER, or REGISTERED_USER. Access is scoped to the authenticated account. JWT auth. Push toggle does not delete device tokens. No group gift preference exists.
Request payload(s):
payload

```
{
  "pushEnabled": true,
  "emailEnabled": true,
  "smsEnabled": false,
  "dealUpdatesEnabled": true,
  "birthdayRemindersEnabled": true,
  "deliveryUpdatesEnabled": true,
  "newContactAlertsEnabled": true,
  "providerOrderAlerts": {
    "newOrders": true,
    "orderCancellations": true,
    "orderDelays": false
  },
  "providerAccountActivity": {
    "securityAlerts": true,
    "loginFromNewDevice": true
  },
  "providerMarketingUpdates": {
    "weeklyPerformanceSummary": true,
    "newFeatureAnnouncements": false,
    "promotionalOffers": false
  }
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

PATCH `/api/v1/notifications/read-all`
Summary: Mark all own notifications as read
Allowed role/access: SUPER_ADMIN, ADMIN, PROVIDER, or REGISTERED_USER
Notes: Access: SUPER_ADMIN, ADMIN, PROVIDER, or REGISTERED_USER. Access is scoped to the authenticated account. JWT auth. Marks only notifications belonging to the logged-in account.
Request payload(s):
payload

```
"<standard success envelope>"
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

PATCH `/api/v1/notifications/{id}/read`

Summary: Mark notification as read
Allowed role/access: SUPER_ADMIN, ADMIN, PROVIDER, or REGISTERED_USER
Notes: Access: SUPER_ADMIN, ADMIN, PROVIDER, or REGISTERED_USER. Access is scoped to the authenticated account. JWT auth. Notification must belong to the logged-in account.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload

```
"<standard success envelope>"
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

POST `/api/v1/notifications/{id}/action`
Summary: Process notification action
Allowed role/access: SUPER_ADMIN, ADMIN, PROVIDER, or REGISTERED_USER
Notes: Access: SUPER_ADMIN, ADMIN, PROVIDER, or REGISTERED_USER. Access is scoped to the authenticated account. JWT auth. Supports SEND_GIFT, REMIND_ME_LATER, VIEW_ORDER, VIEW_CONTACT. Group gift actions are not supported.
Parameters:

- `id` (path, required, string)

Request payload(s):
sendGift

```
{
  "action": "SEND_GIFT"
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

POST `/api/v1/notifications/device-tokens`
Summary: Save device token
Allowed role/access: SUPER_ADMIN, ADMIN, PROVIDER, or REGISTERED_USER
Notes: Access: SUPER_ADMIN, ADMIN, PROVIDER, or REGISTERED_USER. Access is scoped to the authenticated account. JWT auth. Token belongs only to logged-in account. Duplicate deviceId updates existing record.
Request payload(s):
payload

```
{
  "token": "ExponentPushToken[xxxxxxxxxxxxxxxxxxxxx]",
  "platform": "IOS",
```

```
  "deviceId": "ios-iphone-15-pro-device-id"
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

DELETE `/api/v1/notifications/device-tokens/{id}`
Summary: Disable device token
Allowed role/access: SUPER_ADMIN, ADMIN, PROVIDER, or REGISTERED_USER
Notes: Access: SUPER_ADMIN, ADMIN, PROVIDER, or REGISTERED_USER. Access is scoped to the authenticated account. JWT auth. Users can disable only their own device tokens.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload

```
"<standard success envelope>"
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

06 Broadcast Notifications

GET `/api/v1/broadcasts`
Summary: List Broadcasts
Allowed role/access: SUPER_ADMIN or ADMIN with broadcasts.read
Notes: Access: SUPER_ADMIN or ADMIN with broadcasts.read. SUPER_ADMIN or ADMIN with broadcasts.read permission.
Parameters:

- `page` (query, optional, number)
- `limit` (query, optional, number)
- `search` (query, optional, string)
- `status` (query, optional, string)
- `channel` (query, optional, string)
- `priority` (query, optional, string)
- `createdFrom` (query, optional, string)
- `createdTo` (query, optional, string)
- `scheduledFrom` (query, optional, string)
- `scheduledTo` (query, optional, string)
- `sortBy` (query, optional, string)
- `sortOrder` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

POST `/api/v1/broadcasts`
Summary: Create Broadcasts
Allowed role/access: SUPER_ADMIN or ADMIN with broadcasts.create
Notes: Access: SUPER_ADMIN or ADMIN with broadcasts.create. SUPER_ADMIN or ADMIN with broadcasts.create permission.
Request payload(s):
payload

```
{
  "title": "<string>",
  "message": "<string>",
  "imageUrl": "<string>",
  "ctaLabel": "<string>",
  "ctaUrl": "<string>",
  "channels": [
    "EMAIL"
  ],
  "priority": "LOW"
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/broadcasts/{id}`
Summary: Fetch Broadcasts details
Allowed role/access: SUPER_ADMIN or ADMIN with broadcasts.read
Notes: Access: SUPER_ADMIN or ADMIN with broadcasts.read. SUPER_ADMIN or ADMIN with broadcasts.read permission.
Parameters:

- `id` (path, required, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

PATCH `/api/v1/broadcasts/{id}`
Summary: Update Broadcasts
Allowed role/access: SUPER_ADMIN or ADMIN with broadcasts.update
Notes: Access: SUPER_ADMIN or ADMIN with broadcasts.update. SUPER_ADMIN or ADMIN with broadcasts.update permission.

Parameters:

- `id` (path, required, string)

Request payload(s):
payload

```
{
  "title": "<string>",
  "message": "<string>",
  "imageUrl": "<string>",
  "ctaLabel": "<string>",
  "ctaUrl": "<string>",
  "channels": [
    "EMAIL"
  ],
  "priority": "LOW"
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

PATCH `/api/v1/broadcasts/{id}/targeting`
Summary: Update Broadcasts Targeting
Allowed role/access: SUPER_ADMIN or ADMIN with broadcasts.update
Notes: Access: SUPER_ADMIN or ADMIN with broadcasts.update. SUPER_ADMIN or ADMIN with broadcasts.update permission.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload

```
{
  "mode": "ALL_USERS",
  "roles": [
    "ADMIN"
  ],
  "filters": {
    "location": "<string>",
    "onlyVerifiedEmails": true,
    "excludeUnsubscribed": true,
    "excludeSuspended": true
  }
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

POST `/api/v1/broadcasts/estimate-reach`
Summary: Create Broadcasts Estimate Reach
Allowed role/access: SUPER_ADMIN or ADMIN with broadcasts.read
Notes: Access: SUPER_ADMIN or ADMIN with broadcasts.read. SUPER_ADMIN or ADMIN with broadcasts.read permission.
Request payload(s):
payload

```
{
  "channels": [
    "EMAIL"
  ],
  "targeting": {
    "mode": "ALL_USERS",
    "roles": [
      "ADMIN"
    ],
    "filters": {
      "location": "<string>",
      "onlyVerifiedEmails": true,
      "excludeUnsubscribed": true,
      "excludeSuspended": true
    }
  }
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

PATCH `/api/v1/broadcasts/{id}/schedule`
Summary: Update Broadcasts Schedule
Allowed role/access: SUPER_ADMIN or ADMIN with broadcasts.schedule
Notes: Access: SUPER_ADMIN or ADMIN with broadcasts.schedule. SUPER_ADMIN or ADMIN with broadcasts.schedule permission.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload

```
{
  "sendMode": "NOW",
  "scheduledAt": "<string>",
  "timezone": "<string>",
  "isRecurring": true,
  "recurrence": {}
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
```

```
  "message": "Request completed successfully."
}
```

POST `/api/v1/broadcasts/{id}/cancel`
Summary: Create Broadcasts Cancel
Allowed role/access: SUPER_ADMIN or ADMIN with broadcasts.cancel
Notes: Access: SUPER_ADMIN or ADMIN with broadcasts.cancel. SUPER_ADMIN or ADMIN with broadcasts.cancel permission.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload

```
{
  "reason": "<string>"
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/broadcasts/{id}/report`
Summary: Fetch Broadcasts Report details
Allowed role/access: SUPER_ADMIN or ADMIN with broadcasts.report.read
Notes: Access: SUPER_ADMIN or ADMIN with broadcasts.report.read. SUPER_ADMIN or ADMIN with broadcasts.report.read permission.
Parameters:

- `id` (path, required, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/broadcasts/{id}/recipients`
Summary: Fetch Broadcasts Recipients details
Allowed role/access: SUPER_ADMIN or ADMIN with broadcasts.report.read
Notes: Access: SUPER_ADMIN or ADMIN with broadcasts.report.read. SUPER_ADMIN or ADMIN with broadcasts.report.read permission.
Parameters:

- `id` (path, required, string)
- `page` (query, optional, number)
- `limit` (query, optional, number)
- `channel` (query, optional, string)
- `status` (query, optional, string)
- `search` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

07 Plans & Coupons

GET `/api/v1/subscription-plans`
Summary: List Subscription Plans
Allowed role/access: SUPER_ADMIN or ADMIN with subscriptionPlans.read
Notes: Access: SUPER_ADMIN or ADMIN with subscriptionPlans.read. SUPER_ADMIN or ADMIN with subscriptionPlans.read permission.
Parameters:

- `page` (query, optional, number)
- `limit` (query, optional, number)
- `search` (query, optional, string)
- `status` (query, optional, string)
- `visibility` (query, optional, string)
- `billingCycle` (query, optional, string)
- `sortBy` (query, optional, string)
- `sortOrder` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

POST `/api/v1/subscription-plans`
Summary: Create Subscription Plans
Allowed role/access: SUPER_ADMIN or ADMIN with subscriptionPlans.create
Notes: Access: SUPER_ADMIN or ADMIN with subscriptionPlans.create. SUPER_ADMIN or ADMIN with subscriptionPlans.create permission.
Request payload(s):
payload

```
{
  "name": "<string>",
  "description": "<string>",
  "monthlyPrice": 1.0,
  "yearlyPrice": 1.0,
  "currency": "USD",
  "visibility": "PUBLIC",
  "isVisible": true,
  "status": "ACTIVE",
  "isPopular": true,
  "features": {},
  "limits": {
    "maxGiftsPerMonth": 1.0,
    "maxGroupGiftingEvents": 1.0,
    "maxTeamMembers": 1.0,
```

```
    "storageGb": 1.0
  }
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/subscription-plans/stats`
Summary: List Subscription Plans Stats
Allowed role/access: SUPER_ADMIN or ADMIN with subscriptionPlans.analytics.read
Notes: Access: SUPER_ADMIN or ADMIN with subscriptionPlans.analytics.read. SUPER_ADMIN or ADMIN with subscriptionPlans.analytics.read permission.
Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/subscription-plans/{id}`
Summary: Fetch Subscription Plans details
Allowed role/access: SUPER_ADMIN or ADMIN with subscriptionPlans.read
Notes: Access: SUPER_ADMIN or ADMIN with subscriptionPlans.read. SUPER_ADMIN or ADMIN with subscriptionPlans.read permission.
Parameters:

- `id` (path, required, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

PATCH `/api/v1/subscription-plans/{id}`
Summary: Update Subscription Plans
Allowed role/access: SUPER_ADMIN or ADMIN with subscriptionPlans.update, subscriptionPlans.status.update, or subscriptionPlans.visibility.update depending on changed fields
Notes: Access: SUPER_ADMIN or ADMIN with subscriptionPlans.update, subscriptionPlans.status.update, or subscriptionPlans.visibility.update depending on changed fields. Normal plan fields require subscriptionPlans.update; status requires subscriptionPlans.status.update or subscriptionPlans.update; visibility/isVisible requires subscriptionPlans.visibility.update or subscriptionPlans.update.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload

```
{
  "name": "<string>",
  "description": "<string>",
  "monthlyPrice": 1.0,
  "yearlyPrice": 1.0,
  "currency": "<string>",
  "visibility": "PUBLIC",
  "isVisible": true,
  "status": "ACTIVE",
  "reason": "Plan published.",
  "isPopular": true,
  "features": {},
  "limits": {
    "maxGiftsPerMonth": 1.0,
    "maxGroupGiftingEvents": 1.0,
    "maxTeamMembers": 1.0,
    "storageGb": 1.0
  }
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

DELETE `/api/v1/subscription-plans/{id}`
Summary: Delete Subscription Plans
Allowed role/access: SUPER_ADMIN or ADMIN with subscriptionPlans.delete
Notes: Access: SUPER_ADMIN or ADMIN with subscriptionPlans.delete. SUPER_ADMIN or ADMIN with subscriptionPlans.delete permission.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload

```
"<standard success envelope>"
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/subscription-plans/{id}/analytics`
Summary: Fetch Subscription Plans Analytics details
Allowed role/access: SUPER_ADMIN or ADMIN with subscriptionPlans.analytics.read
Notes: Access: SUPER_ADMIN or ADMIN with subscriptionPlans.analytics.read. SUPER_ADMIN or ADMIN with subscriptionPlans.analytics.read permission.
Parameters:

- `id` (path, required, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/plan-features/catalog`
Summary: List Plan Features Catalog
Allowed role/access: SUPER_ADMIN or ADMIN with planFeatures.read
Notes: Access: SUPER_ADMIN or ADMIN with planFeatures.read. SUPER_ADMIN or ADMIN with planFeatures.read permission.
Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/plan-features`
Summary: List Plan Features
Allowed role/access: SUPER_ADMIN or ADMIN with planFeatures.read
Notes: Access: SUPER_ADMIN or ADMIN with planFeatures.read. SUPER_ADMIN or ADMIN with planFeatures.read permission.
Parameters:

- `page` (query, optional, number)
- `limit` (query, optional, number)
- `search` (query, optional, string)
- `isActive` (query, optional, boolean)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

POST `/api/v1/plan-features`
Summary: Create Plan Features
Allowed role/access: SUPER_ADMIN or ADMIN with planFeatures.create
Notes: Access: SUPER_ADMIN or ADMIN with planFeatures.create. SUPER_ADMIN or ADMIN with planFeatures.create permission.
Request payload(s):
payload

```
{
  "key": "<string>",
  "label": "<string>",
  "description": "<string>",
  "type": "BOOLEAN",
  "isActive": true,
```

```
  "sortOrder": 1.0
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/plan-features/{id}`
Summary: Fetch Plan Features details
Allowed role/access: SUPER_ADMIN or ADMIN with planFeatures.read
Notes: Access: SUPER_ADMIN or ADMIN with planFeatures.read. SUPER_ADMIN or ADMIN with planFeatures.read permission.
Parameters:

- `id` (path, required, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

PATCH `/api/v1/plan-features/{id}`
Summary: Update Plan Features
Allowed role/access: SUPER_ADMIN or ADMIN with planFeatures.update
Notes: Access: SUPER_ADMIN or ADMIN with planFeatures.update. SUPER_ADMIN or ADMIN with planFeatures.update permission.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload

```
{
  "key": "<string>",
  "label": "<string>",
  "description": "<string>",
  "type": "BOOLEAN",
  "isActive": true,
  "sortOrder": 1.0
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

DELETE `/api/v1/plan-features/{id}`
Summary: Delete Plan Features
Allowed role/access: SUPER_ADMIN or ADMIN with planFeatures.delete
Notes: Access: SUPER_ADMIN or ADMIN with planFeatures.delete. SUPER_ADMIN or ADMIN with planFeatures.delete permission.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload

```
"<standard success envelope>"
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/coupons`
Summary: List Coupons
Allowed role/access: SUPER_ADMIN or ADMIN with coupons.read
Notes: Access: SUPER_ADMIN or ADMIN with coupons.read. SUPER_ADMIN or ADMIN with coupons.read permission.
Parameters:

- `page` (query, optional, number)
- `limit` (query, optional, number)
- `search` (query, optional, string)
- `status` (query, optional, string)
- `planId` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

POST `/api/v1/coupons`
Summary: Create Coupons
Allowed role/access: SUPER_ADMIN or ADMIN with coupons.create
Notes: Access: SUPER_ADMIN or ADMIN with coupons.create. SUPER_ADMIN or ADMIN with coupons.create permission.
Request payload(s):
payload

```
{
  "code": "<string>",
  "description": "<string>",
  "discountType": "PERCENTAGE",
  "discountValue": 1.0,
  "planIds": [
    "<string>"
  ],
  "startsAt": "<string>",
```

```
  "expiresAt": "<string>",
  "maxRedemptions": 1.0,
  "isActive": true
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/coupons/{id}`
Summary: Fetch Coupons details
Allowed role/access: SUPER_ADMIN or ADMIN with coupons.read
Notes: Access: SUPER_ADMIN or ADMIN with coupons.read. SUPER_ADMIN or ADMIN with coupons.read permission.
Parameters:

- `id` (path, required, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

PATCH `/api/v1/coupons/{id}`
Summary: Update coupon details or lifecycle status
Allowed role/access: SUPER_ADMIN or ADMIN with coupon-specific update permission
Notes: Access: SUPER_ADMIN or ADMIN with coupon-specific update permission. SUPER_ADMIN or ADMIN with coupon-specific permissions. Standard coupon fields require coupons.update; status or isActive changes require coupons.status.update; mixed payloads require both permissions. SUPER_ADMIN or ADMIN with coupon-specific permissions. Standard coupon fields require 'coupons.update'; status or isActive changes require 'coupons.status.update'; mixed payloads require both permissions.
Parameters:

- `id` (path, required, string)

Request payload(s):
updateCoupon

```
{
  "code": "SUMMER20",
  "description": "Seasonal 20% off campaign.",
  "discountType": "PERCENTAGE",
  "discountValue": 20,
  "maxRedemptions": 500
}
```

activateCoupon

```
{
  "isActive": true,
  "status": "ACTIVE",
```

```
  "reason": "Campaign enabled."
}
```

deactivateCoupon

```
{
  "isActive": false,
  "status": "INACTIVE",
  "reason": "Campaign paused after budget review."
}
```

Response body:

```
{
  "success": true,
  "data": {
    "id": "coupon_id",
    "code": "SUMMER20",
    "isActive": true,
    "status": "ACTIVE",
    "discountType": "PERCENTAGE",
    "discountValue": 20
  },
  "message": "Coupon updated successfully"
}
```

DELETE `/api/v1/coupons/{id}`
Summary: Delete Coupons
Allowed role/access: SUPER_ADMIN or ADMIN with coupons.delete
Notes: Access: SUPER_ADMIN or ADMIN with coupons.delete. SUPER_ADMIN or ADMIN with coupons.delete permission.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload

```
"<standard success envelope>"
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

07 Storage

POST `/api/v1/uploads/presigned-url`
Summary: Create presigned upload URL
Allowed role/access: SUPER_ADMIN, ADMIN, PROVIDER, or REGISTERED_USER
Notes: Access: SUPER_ADMIN, ADMIN, PROVIDER, or REGISTERED_USER. Backend derives ownerId/ownerRole from JWT. targetAccountId is forbidden for REGISTERED_USER and PROVIDER, admin-only when authorized, and allowed for SUPER_ADMIN. Backend derives ownerId/ownerRole from the authenticated JWT. targetAccountId is forbidden for

REGISTERED_USER and PROVIDER, allowed only for SUPER_ADMIN/authorized ADMIN dashboard uploads. Include giftId only for gift image uploads.

Request payload(s):

giftUpload

```
{
  "folder": "gift-images",
  "fileName": "perfume.png",
  "contentType": "image/png",
  "sizeBytes": 1048576,
  "giftId": "gift_id"
}
```

normalUpload

```
{
  "folder": "provider-avatars",
  "fileName": "avatar.png",
  "contentType": "image/png",
  "sizeBytes": 1048576
}
```

adminOnBehalf

```
{
  "folder": "provider-logos",
  "fileName": "logo.png",
  "contentType": "image/png",
  "sizeBytes": 1048576,
  "targetAccountId": "provider_user_id"
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

POST `/api/v1/uploads/complete`

Summary: Complete upload

Allowed role/access: SUPER_ADMIN, ADMIN, PROVIDER, or REGISTERED_USER

Notes: Access: SUPER_ADMIN, ADMIN, PROVIDER, or REGISTERED_USER. Completes only uploads accessible to the authenticated account. Authenticated upload completion. REGISTERED_USER and PROVIDER can complete only their own uploads. ADMIN defaults to own uploads. SUPER_ADMIN can manage dashboard/admin inspection flows.

Request payload(s):

payload

```
{
  "uploadId": "<string>",
  "sizeBytes": 1.0
}
```

Response body:

```
{
  "success": true,
```

```
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```


GET `/api/v1/uploads`
Summary: List uploads
Allowed role/access: SUPER_ADMIN, ADMIN, PROVIDER, or REGISTERED_USER
Notes: Access: SUPER_ADMIN, ADMIN, PROVIDER, or REGISTERED_USER. REGISTERED_USER/PROVIDER list only own uploads; ownerId is ignored. ADMIN lists own uploads by default and may use ownerId only for authorized managed access. SUPER_ADMIN may inspect by ownerId. REGISTERED_USER: ownerId query is ignored and only own uploads are listed. PROVIDER: ownerId query is ignored and only own uploads are listed. ADMIN: lists own uploads by default and may use ownerId only for managed dashboard access when authorized. SUPER_ADMIN: can use ownerId for dashboard/admin inspection.
Parameters:

- `page` (query, optional, number)
- `limit` (query, optional, number)
- `folder` (query, optional, string)
- `ownerId` (query, optional, string)

Response body:


```
{
  "success": true,
  "data": [
    {
      "id": "upload_id",
      "ownerId": "user_id",
      "ownerRole": "REGISTERED_USER",
      "targetAccountId": null,
      "folder": "user-avatars",
      "fileName": "avatar.png",
      "contentType": "image/png",
      "sizeBytes": 1048576,
      "fileUrl": "https://cdn.yourdomain.com/user-avatars/avatar.png",
      "storageKey": "user-avatars/user_id/uuid-avatar.png",
      "status": "COMPLETED",
      "giftId": null,
      "createdAt": "2026-04-08T09:00:00.000Z",
      "updatedAt": "2026-04-08T09:01:00.000Z",
      "completedAt": "2026-04-08T09:01:00.000Z"
    }
  ],
  "meta": {
    "page": 1,
    "limit": 10,
    "total": 1,
    "totalPages": 1
  },
  "message": "Uploads fetched successfully."
}
```


GET `/api/v1/uploads/{id}`
Summary: Fetch upload details
Allowed role/access: SUPER_ADMIN, ADMIN, PROVIDER, or REGISTERED_USER
Notes: Access: SUPER_ADMIN, ADMIN, PROVIDER, or REGISTERED_USER. REGISTERED_USER/PROVIDER can fetch only own uploads. ADMIN defaults to own uploads. SUPER_ADMIN may inspect uploads. REGISTERED_USER and PROVIDER can fetch only own uploads. ADMIN fetches own uploads by default. SUPER_ADMIN can inspect uploads for dashboard/admin operations.

Parameters:

- `id` (path, required, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

DELETE `/api/v1/uploads/{id}`
Summary: Delete upload
Allowed role/access: SUPER_ADMIN, ADMIN, PROVIDER, or REGISTERED_USER
Notes: Access: SUPER_ADMIN, ADMIN, PROVIDER, or REGISTERED_USER. REGISTERED_USER/PROVIDER can delete only own uploads. ADMIN defaults to own uploads. SUPER_ADMIN may delete inspected uploads. REGISTERED_USER and PROVIDER can delete only own uploads. ADMIN deletes own uploads by default. SUPER_ADMIN can delete inspected dashboard uploads. Deletion is permanent in the database.
Parameters:

- `id` (path, required, string)

Request payload(s):
payload

```
"<standard success envelope>"
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

08 Chat - Threads

GET `/api/v1/chats`
Summary: List chat threads
Allowed role/access: REGISTERED_USER, PROVIDER, SUPER_ADMIN, or ADMIN with chat/support permission
Notes: Access: REGISTERED_USER, PROVIDER, SUPER_ADMIN, or ADMIN with chat/support permission. Role-aware thread listing. Customers/providers see own order/support threads; SUPER_ADMIN sees support threads; ADMIN is scoped by supportChats.read/read.all or moderation chat permissions. Role-aware list for customer order chats, provider buyer chats, support chats, and admin-visible support/moderation chat views.
Parameters:

- `page` (query, optional, number)
- `limit` (query, optional, number)
- `search` (query, optional, string)
- `threadType` (query, optional, string)
- `status` (query, optional, string)
- `unreadOnly` (query, optional, boolean)
- `sourceType` (query, optional, string)
- `sourceId` (query, optional, string)
- `participantRole` (query, optional, string)
- `participantId` (query, optional, string)

- `assignedToAdminId` (query, optional, string)
- `fromDate` (query, optional, string)
- `toDate` (query, optional, string)
- `sortBy` (query, optional, string)
- `sortOrder` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/chats/quick-replies`
Summary: Fetch role-aware chat quick replies
Allowed role/access: REGISTERED_USER, PROVIDER, SUPER_ADMIN, or ADMIN
Notes: Access: REGISTERED_USER, PROVIDER, SUPER_ADMIN, or ADMIN. Role-aware quick replies.
Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

POST `/api/v1/chats/threads`
Summary: Create or get a chat thread
Allowed role/access: REGISTERED_USER, PROVIDER, SUPER_ADMIN, or ADMIN with supportChats.reply/messageModeration permission
Notes: Access: REGISTERED_USER, PROVIDER, SUPER_ADMIN, or ADMIN with supportChats.reply/messageModeration permission. Create or get an order/support/moderation chat thread based on role and sourceType.
Request payload(s):
payload

```
{
  "threadType": "ORDER_CHAT",
  "sourceType": "CUSTOMER_ORDER",
  "sourceId": "order_id",
  "subject": "Order support",
  "initialMessage": "Can you confirm delivery time?",
  "attachmentUrls": [],
  "createIfMissing": true,
  "participantId": "participant_user_id",
  "participantRole": "REGISTERED_USER"
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/chats/threads/{threadId}`
Summary: Fetch chat thread details
Allowed role/access: REGISTERED_USER, PROVIDER, SUPER_ADMIN, or ADMIN with chat/support permission
Notes: Access: REGISTERED_USER, PROVIDER, SUPER_ADMIN, or ADMIN with chat/support permission. Fetch a thread only when the authenticated role can access it.
Parameters:

- `threadId` (path, required, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/chats/threads/{threadId}/messages`
Summary: Fetch chat thread messages
Allowed role/access: REGISTERED_USER, PROVIDER, SUPER_ADMIN, or ADMIN with chat/support permission
Notes: Access: REGISTERED_USER, PROVIDER, SUPER_ADMIN, or ADMIN with chat/support permission. Fetch messages only for accessible threads.
Parameters:

- `threadId` (path, required, string)
- `page` (query, optional, number)
- `limit` (query, optional, number)
- `before` (query, optional, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

POST `/api/v1/chats/threads/{threadId}/messages`
Summary: Send a chat message
Allowed role/access: REGISTERED_USER, PROVIDER, SUPER_ADMIN, or ADMIN with supportChats.reply
Notes: Access: REGISTERED_USER, PROVIDER, SUPER_ADMIN, or ADMIN with supportChats.reply. Send a message in an accessible order/support thread.
Parameters:

- `threadId` (path, required, string)

Request payload(s):
payload

```
{
  "clientMessageId": "mobile-generated-uuid",
  "messageType": "TEXT",
  "body": "Can you confirm delivery time?",
  "attachmentUrls": []
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

PATCH `/api/v1/chats/threads/{threadId}/read`
Summary: Mark a chat thread as read
Allowed role/access: REGISTERED_USER, PROVIDER, SUPER_ADMIN, or ADMIN with supportChats.read
Notes: Access: REGISTERED_USER, PROVIDER, SUPER_ADMIN, or ADMIN with supportChats.read. Mark an accessible thread as read for the authenticated participant/admin.
Parameters:

- `threadId` (path, required, string)

Request payload(s):
payload

```
"<standard success envelope>"
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

PATCH `/api/v1/chats/threads/{threadId}/status`
Summary: Update chat thread status
Allowed role/access: REGISTERED_USER, PROVIDER, SUPER_ADMIN, or ADMIN with status-specific chat permission
Notes: Access: REGISTERED_USER, PROVIDER, SUPER_ADMIN, or ADMIN with status-specific chat permission. Unified chat thread lifecycle endpoint. Support thread RESOLVED/REOPENED requires supportChats.resolve; BLOCKED_BY_MODERATION requires messageModeration.moderate or chats.moderate. Unified thread lifecycle endpoint. RESOLVED and REOPENED require supportChats.resolve for support threads. ARCHIVED follows thread owner/admin access rules. BLOCKED_BY_MODERATION requires messageModeration.moderate or chats.moderate.
Parameters:

- `threadId` (path, required, string)

Request payload(s):
resolved

```
{
  "status": "RESOLVED",
  "reason": "ISSUE_RESOLVED",
  "comment": "Support issue resolved.",
  "notifyParticipants": true
}
```

reopened

```
{
  "status": "REOPENED",
  "reason": "CUSTOMER_REPLIED",
  "comment": "Customer needs more help.",
```

```
  "notifyParticipants": true
}
```

archived

```
{
  "status": "ARCHIVED",
  "reason": "NO_LONGER_NEEDED",
  "comment": "Archived by participant.",
  "notifyParticipants": false
}
```

blocked

```
{
  "status": "BLOCKED_BY_MODERATION",
  "reason": "POLICY_VIOLATION",
  "comment": "Blocked by moderation.",
  "notifyParticipants": true
}
```

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```

GET `/api/v1/chats/threads/{threadId}/audit-log`
Summary: Fetch chat thread audit log
Allowed role/access: SUPER_ADMIN or ADMIN with supportChats.read
Notes: Access: SUPER_ADMIN or ADMIN with supportChats.read. Fetch audit trail for an accessible support/moderation thread.
Parameters:

- `threadId` (path, required, string)

Response body:

```
{
  "success": true,
  "data": "<response returned by endpoint>",
  "message": "Request completed successfully."
}
```